

目录

unix 考试	8
要求	8
Ch1 Linux 命令基础	8
Linux bash 常用通配符	8
Linux 命令格式	8
用户与用户组	10
添加用户	10
用户密码设置	10
删除用户	11
修改用户信息	11
举例:	12
Linux 组有如下特点:	12
添加用户组	12
删除用户组	12
修改用户组属性	12
用户组切换	13
举例	13
用户切换	13
su	13
sudo	13
用户切换举例	14
文件操作命令	14
pwd	14
cd	14
ls	15
touch	15
mkdir	15
cp	15
rm	15
mv	15
rmdir	15
查看文件命令	15
cat	16
more	16
less	16
head	16
tail	16
wc	16
文件搜索命令	16
which	16
find	16
locate	17
grep	17
文件权限	17
权限管理	17
chmod	17
chown	18
chgrp	18

文件压缩解压缩	18
tar.....	18
zip/unzip.....	19
gzip/gunzip.....	19
bzip2/bunzip2.....	19
进程管理	19
ps.....	19
top.....	20
pstree.....	20
pgrep.....	20
nice.....	21
jobs.....	21
bg/fg.....	21
kill.....	21
服务与网络管理	21
Ch2 Linux shell 编程基础	22
Shell 功能	22
变量使用	22
变量赋值	22
变量引用	22
变量删除	22
变量输入命令 read.....	23
变量输出命令 echo.....	1
变量输出命令 printf.....	1
字符串常见操作表达式	1
变量使用举例	1
本地变量	23
环境变量	23
位置变量	24
特殊变量	24
变量使用举例	24
引号	25
通配符	25
连接符	25
“;”连接符.....	25
“&&”连接符.....	25
“ ”连接符.....	25
其他符号	26
Shell 脚本的基本元素	26
Shell 解释器运行脚本	26
通过 source 运行脚本	26
直接运行脚本 (有权限)	26
运算符	26
expr 指令.....	27
let 指令.....	27
条件测试语句	27
test.....	27

[] 或 [[]].....	27
[] 支持通配符和正则表达式.....	27
文件测试操作符	28
条件测试语句	28
整数测试.....	28
字符串测试.....	29
逻辑运算符测试.....	29
条件测试语句—逻辑测试举例	30
判断结构	30
if 结构.....	30
if/else 结构.....	30
if/elif/else 结构.....	30
if 语句举例.....	31
if/else 使用总结.....	32
case 结构.....	32
case 语句举例.....	32
case 使用总结.....	32
循环结构	33
for 循环 1.....	33
for 循环 2.....	33
for 循环 3.....	33
for 语句举例.....	34
循环结构	35
while 循环.....	35
while 语句举例.....	35
until 循环.....	36
until 语句举例.....	36
break 与 continue.....	36
exit 与 sleep.....	36
select 循环.....	37
select 语句举例.....	37
shell 函数语法	37
shell 函数参数	38
shell 函数返回值	39
正则表达式	39
与通配符区别.....	39
基础元字符.....	40
基础元字符举例.....	40
扩展元字符.....	40
常用字符簇.....	41
grep 命令	41
grep 常见选项.....	41
cut 命令	42
使用举例.....	42

awk.....	42
awk 语法格式.....	43
关系表达式.....	43
逻辑运算符.....	43
正则匹配模式.....	43
区间模式.....	44
awk 使用方式.....	44
awk 使用举例.....	44
awk 常用变量.....	44
awk 常用变量举例.....	44
awk 使用正则表达式.....	44
awk 流程控制.....	44
awk 流程控制举例.....	45
sed.....	45
sed 语法格式.....	45
sed 行定位.....	45
sed 操作——打印.....	46
sed 操作——追加文本.....	46
sed 操作——删除文本.....	46
sed 操作——查找替换.....	46
sed 操作——保存文件.....	47
sed 应用.....	47
shell 脚本运维实例	47
模拟用户登录.....	47
监控系统运行情况.....	47
统计进程的状态情况.....	48
Ch3 Linux 编程基础	48
GCC 编译器	48
GCC 编译器介绍.....	48
GCC 参数选项.....	48
GCC 编译过程.....	49
GCC 编译优化.....	49
头文件和库函数目录	50
GCC -I dir 参数使用.....	50
例子.....	50
GCC 创建函数库.....	50
GCC 创建动态库.....	51
Make 工具	51
Make 引入.....	51
make & makefile.....	51
make.....	51
makefile 的组成.....	51
Makefile 的显式规则.....	52
Makefile 举例.....	52
一个简单的 makefile.....	53

Make 的工作过程.....	53
Makefile 的变量.....	54
Makefile 的隐式规则(自动推导).....	54
Makefile 的伪目标.....	54
Ch4 Linux 文件 I/O.....	54
文件属性	54
stat/fstat/lstat 函数.....	55
文件类型	55
文件存取权限	56
改变文件存取权限——程序	56
chmod 函数.....	57
默认文件存取权限——umask	57
umask 函数.....	57
文件其他属性	58
1.chown/fchown/lchown 函数.....	58
2.获取文件存取时间——stat.....	58
掌握的知识点	59
两种 I/O 方式	59
文件描述符	60
无缓冲 I/O 函数	60
0.错误处理.....	60
1.creat 函数.....	60
2.Open 函数.....	62
3.close 函数.....	63
4.read/write 函数.....	63
5.lseek 函数.....	64
6.dup/dup2 函数.....	64
7.fcntl 函数.....	66
缓冲文件 I/O	67
标准 I/O 库	67
文件流	67
标准 I/O 函数	68
流打开/关闭.....	68
流读/写.....	69
流定位.....	70
流刷新.....	71
流缓冲.....	71
流和文件描述符.....	72
特殊文件	72
目录文件	72
读目录.....	73
数据结构.....	73
操作函数.....	73
目录扫描程序——ls -R 命令.....	73

mkdir/rmdir 函数.....	74
chdir/fchdir 函数.....	74
链接文件	74
symlink/readlink.....	74
设备文件	75
Ch5 进程编程	76
进程和进程环境	76
进程.....	76
进程启动.....	76
进程终止.....	76
进程分类.....	76
进程控制	77
进程创建.....	77
文件共享.....	77
fork 应用场合.....	80
vfork.....	80
相关函数.....	82
函数原型.....	82
exec 系列函数.....	82
exec 举例.....	82
fork 和 exec 一起使用.....	83
进程终止.....	83
1.exit 函数.....	83
2._exit 函数.....	83
父子进程关系	83
孤儿进程.....	84
僵尸进程 (Zombie).....	84
方法一： wait example.....	85
方法二： 两次 fork Example3-6.c.....	86
方法三： signal 函数处理.....	87
守护进程 Daemon	88
登录方式	88
终端登录.....	88
网络登录.....	88
进程组和会话期	89
getpgrp/setpgid 函数.....	89
getsid/setsid 函数.....	90
守护进程 Daemon	91
daemon 进程实现规则.....	92
编写守护进程的要点.....	92
守护进程编写.....	93
Ch6 线程编程	97
线程概念	97
基本 API 函数	97

线程创建——pthread_create 函数.....	97
线程终止——pthread_exit 函数.....	97
线程取消——pthread_cancel 函数.....	98
线程回收——pthread_join 函数.....	98
线程同步	99
线程同步的概念.....	99
线程互斥锁.....	100
条件变量.....	103
信号量.....	106
线程属性	110
1 线程属性初始化和销毁.....	110
2 设置分离状态属性.....	110
3 设置取消线程属性.....	113
Ch7 进程通信	114
信号	115
1.1 信号概念.....	115
1.2 信号相关命令.....	116
1.3 信号相关函数.....	116
pipe 管道	121
2.1 进程间通信.....	121
2.2 pipe 概念.....	122
2.3 popen & pclose 函数.....	125
2.4 FIFO: named pipe 命名管道.....	128
System V IPC.....	134
3.1 System V IPC 的共同特征.....	134
3.2 Message.....	135
3.3 Shared.....	140
3.4 信号量.....	143
网络编程	146
4.0 服务器和客户机的信息函数.....	146
4.1 Socket 基础.....	147
4.2 TCP 编程.....	149
4.3 UDP 编程.....	154
4.4 网络高级编程.....	157

unix考试

要求

考试以系统编程为主，运维知识适当减少

第一章不用去看

第二章常见的shell命令，有什么功能，文件啊、用户啊、进程啊、重定向等命令...

第三章shell编程，shell编程对shell的基本变量，运算符、通配符，脚本的执行方式，文本处理grep等等，至少能看懂，知道实现什么功能，不会让去编程

第四章ftp、dhcp...不在考试内容

第一章了解gcc的使用，预处理等等这每一阶段。如何去编写一个makefile

第二章文件IO，底层的文件IO open、dop、dop2、lseek知道怎么用，基本程序会写

第三章进程 僵尸进程、孤儿进程、守护进程能认出来。知道怎么写一个程序以守护程序进行 fork。。。

第四章线程 看到函数知道是实现了什么功能，互斥锁，条件变量

第五章进程通信 重点是信号 fifo pid popen 包括相关的程序会写，知道套接字相关的特点。

Ch1 Linux 命令基础

Linux bash常用通配符

*：匹配任意一个或多个字符

?：匹配任意一个字符

[]：匹配包含在[]内的单个字符

```
例：ls *.txt
ls ?.txt
ls ???.txt
ls /dev/sda[1-5]
ls [0-9]?.conf
ls [xyz]*.txt
```

Linux命令格式

command [options] [arguments]

命令 选项 参数

Linux是一个多用户、多任务的分时操作系统，在一台Linux主机上，可同时登录多名用户

Linux用户分为三种：

- 1.root用户 (ID 0)
- 2.系统用户 (ID 1-499)
- 3.普通用户 (ID 500以上)

用户与用户组

Linux操作系统用户的特点:

每个用户拥有一个UserID, 操作系统实际读取的是UID;

每个用户属于一个主组, 属于一个或多个附属组, 一个用户最多有31个附属组;

每个组拥有一个GroupID;

每个进程以一个用户身份运行, 该用户可对进程拥有资源控制权限;

每个可登陆用户拥有一个指定的Shell环境。

涉及到的相关配置文件:

/etc/passwd 保存用户信息

/etc/shadow 保存用户密码 (以加密形式保存)

/etc/group 保存组信息

/etc/login.defs 用户属性限制, 密码过期时间, 密码最大长度等限制

/etc/default/useradd 显示或更改默认的用户add配置文件

添加用户

useradd命令添加用户, 命令的格式如下:

useradd [选项] 用户名

选项	说明
-d	指定用户登入时的目录
-c	指定账户的备注文字
-e	指定账号的有效期限
-f	缓冲天数, 密码过期时在指定天数后关闭该账号
-g	指定用户所属组
-G	指定用户所属的附加用户组
-m	自动建立用户的登入目录
-r	创建系统账号
-s	指定用户的登陆shell
-u	指定用户的用户ID。若添加-o选项, 则用户ID可与其它用户重复

useradd user1命令默认创建用户user1, 创建过程:

读取/etc/default/useradd, 根据配置文件执行创建操作;

在/etc/passwd文件中添加用户信息;

如使用passwd命令创建密码, 密码会被加密保存在/etc/shadow中;

为用户1创建家目录: /home/user1;

将/etc/skel中的.bash开头的文件复制至/home/user1家目录;

创建与用户名相同的user1组, user1用户默认属于user1同名组;

user1组信息保存在/etc/group配置文件中。

用户密码设置

设置用户密码的命令是passwd, 该命令用于设置用户的认证信息, 包括用户密码、密码有效期等, 命令的格式如下:

passwd [选项] 用户名

示例: passwd bxg

选项	说明
-l	锁定密码，锁定后密码失效，无法登陆（新用户默认锁定）
-d	删除密码，仅系统管理员可使用
-S	列出密码相关信息，仅系统管理员可使用
-f	强制执行

说明：

①root用户可随意修改密码，即便出现警告，也可成功保存。

②普通用户在设置密码时应尽量复杂，避免与用户名相同。

③修改当前登陆账户的密码时，可缺省用户名。

④用户密码保存在文件/etc/shadow中。

删除用户

删除用户账号的命令为`userdel`，该命令可删除指定账户以及与账户相关的文件和信息。命令格式如下：

`userdel [选项] 用户名`

示例：`userdel -r kdy`

选项	说明
-f	强制删除用户，即便该用户为当前用户
-r	删除用户的同时，删除与用户相关的所有文件

修改用户信息

修改用户账号信息，即修改账号的属性，如用户ID、主目录、用户组、登陆shell等。修改用户账号信息的命令为`usermod`，其命令格式如下：

`usermod 选项 用户名`

示例：`usermod -u 678 Jim`

选项	说明
-c	修改用户账号的备注信息
-d	修改用户的登入目录
-e	修改账号的有效期限
-f	修改缓冲天数，即修改密码过期后关闭账号的时间
-g	修改用户所属组
-l	修改用户账号名称
-L	锁定用户密码，使密码失效
-s	修改用户登陆后使用的shell
-u	修改用户ID
-U	解除密码锁定

举例：

- 添加一个用户myuser，指定主用户组为mygroup，附加用户组为addgroup，用户主目录为/opt/myusers
- 添加一个用户testuser，指定UID为686，默认shell为/bin/csh，主用户组为mygroup，附加用户组为addgroup，添加用户描述为：This is test user
- 修改用户testuser主用户组为testgroup，附加用户组为testaddgroup，默认登录shell为/bin/bash

Linux组有如下特点：

- 每个组有一个组ID；
组信息保存在/etc/group中；
每个用户至少拥有一个主组，同时还可以拥有31个附属组。

添加用户组

- 默认情况下新建用户的用户组与用户名相同，在创建用户的同时被创建。主动添加用户组时使用的命令为groupadd，该命令的格式如下：groupadd [选项] 用户名
示例：groupadd -g 550 group1

选项	说明
-g	指定新建用户组的组ID
-r	创建系统用户组，组ID取值范围为1~499
-o	允许创建组ID已存在的用户组

删除用户组

- 若要删除已存在的用户组，可使用groupdel命令，该命令的格式如下：
groupdel 参数
示例：groupdel group1

修改用户组属性

- 用户组的一些属性，如组ID和组名，都可以被修改。修改用户组属性的命令为groupmod，其命令格式如下：
groupmod [选项] [用户组]
示例：groupmod group1 -g 550
groupmod -o group2 -g 550
groupmod group1 -n qust

选项	说明
-g	为用户组指定新的组ID
-n	修改用户组的组名
-o	允许创建组ID已存在的用户组

用户组切换

若用户被创建时没有指定用户组，系统会为用户创建一个与用户名相同的组，这个组就是基本组；除基本组之外，用户所在的组都是附加组，为用户指定附加组，可以使用户拥有对应组的权限。

`newgrp [用户组]`

举例

创建3个用户组group1, group2, group3
创建一个用户user1，指定主用户组为group1，附属用户组为group2, group3
切换到user1用户，创建目录dir1，分别切换到group2, group3，再创建目录dir2, dir3
查看目录

用户切换

`su: switch user`
`sudo: super user do`

su

使用su命令切换用户是最简单的用户切换方式，该命令可以在任意用户之间切换，其命令格式如下：

`su - username`

需要注意，以上的“-”为一个选项，类似于“-l”，选项与用户名之间应有一个空格。

示例：`su`

参数	说明
-c	执行完指定的指令后，切换回原来的用户
-l	切换用户的同时，切换到对应用户的工作目录，环境变量也会随之改变
-m, -p	切换用户时，不改变环境变量
-s	指定要执行的shell

说明：

- ①使用su命令切换用户时，需要输入的是目标用户的密码。
- ②选项与用户名缺省的情况下，只切换用户，但不改变用户环境。
- ③由root用户切换到其它用户，可以不输入密码。
- ④在命令行输入“exit”或“su -user”，可退出目标用户。

sudo

sudo命令的格式如下：

`sudo [选项] -u 用户 [命令]`

选项	说明
-b	在后台执行命令
-h	显示帮助
-H	将HOME环境变量设置为新身份的HOME环境变量
-k	结束密码的有效期限
-l	列出目前用户可执行与无法执行的命令
-p	改变询问密码的提示符号
-s	执行指定的shell
-u	以指定的用户作为新的身份，即切换到指定用户。默认切换到root用户

sudo说明:

①sudo命令可以视为受限的su，它可以使“部分”用户使用其它用户的身份执行命令。

②在使用sudo命令之前，需要通过修改/etc/sudoers文件，为当前用户配置要使用的权限。

③/etc/sudoers文件有一定的语法规范，为避免因修改后出现语法错误，应使用visudo命令打开文件进行修改。

④如下所示为/etc/sudoers文件中的一条配置信息，该信息设置了root用户可能任何情境下执行任何命令。

```
root    ALL=(ALL)    ALL
```

用户切换举例

su myuser和su - myuser区别，env |grep myuser查看

su - -c 用法: su - -c "tail -n 4 /etc/shadow"

sudo su -和su -区别

文件操作命令

pwd

cd

ls

touch

mkdir

cp

rm

mv

mkdir

pwd

命令原意: print working directory

命令功能: 打印当前工作目录的绝对路径

命令格式: pwd

cd

命令原意: change directory

命令功能: 切换目录

命令格式: cd 参数

参数 说明

. 切换到当前目录

.. 切换到父目录

~ 切换到用户家目录

ls

命令原意: `list`
命令功能: 列出参数的属性信息
命令格式: `ls [选项] [参数]`

参数 说明

`-l` 显示当前目录的详细信息
`-a` 显示当前目录下所有文件包括隐藏文件
`-i` 显示文件的inode编号
`-R` 递归显示当前目录下包括子目录下的所有文件信息

touch

命令功能: 更新已存在文件的时间标签, 若文件不存在则新建文件
命令格式: `touch 参数`
示例: `touch hello.c`

mkdir

命令原意: `make directory`
命令功能: 创建目录
命令格式: `mkdir [选项] 参数`

参数 说明

`-p` 递归创建目录

cp

命令原意: `copy`
命令功能: 将一个或多个源文件复制到指定目录
命令格式: `cp [选项] 源文件或目录 目标目录`

参数 说明

`-R` 递归处理, 包括子目录下的文件
`-f` 强制复制文件或目录

rm

命令原意: `remove`
命令功能: 删除目录中的文件或目录
命令格式: `rm [选项] 文件或目录`

参数 说明

`-f` 强制删除文件或目录
`-i` 删除文件前进行询问
`-rf` 强制删除所有文件和子目录, 不会询问

mv

命令原意: `move`
命令功能: 移动文件或目录
命令格式: `mv 源文件或目录 目标目录`

rmdir

命令原意: `remove directory`
命令功能: 删除目录
命令格式: `rmdir [-p] 目录`

查看文件命令

`cat`
`more`
`less`
`head`
`tail`
`wc`

cat

命令原意: concatenate and display files
命令功能: 打印文件内容到输出设备
命令格式: cat 文件名

more

命令功能: 分页显示文件内容
命令格式: more 文件名

快捷键 说明
空格键 显示下一页
回车键 显示下一行
q/Q 退出

less

命令功能: 分页显示文件内容, 可实现向上翻页
命令格式: less 文件名

head

命令功能: 查看文件的前n行
命令格式: head -n 文件名

tail

命令功能: 查看文件的后n行
命令格式: tail -n 文件名

wc

命令原意: word count
命令功能: 计算文件的字节数、字数和列数
命令格式: wc [选项] 参数
示例:
wc -l /etc/passwd #统计passwd文件的行数
wc -c /etc/passwd #统计passwd文件的字节数

参数 说明
-c 统计指定文件中文本的字节数
-l 统计指定文件中文本的行数
-w 统计指定文件中的字数

文件搜索命令

which

find

locate

grep

which

命令功能: 查看命令所在路径
命令格式: which 命令
示例: which ls

find

命令功能: 借助搜索关键字(文件名、文件大小、文件所有者等)查找文件或目录
命令格式: find 搜索路径 [选项] 搜索关键字
示例: find /etc -name passwd

-amin 按文件存取时间来查找文件，-n指n分钟以内，+n指n分钟以前
-atime 按文件存取时间来查找文件，-n指n天以内，+n指n天以前
-cmin 根据文件修改（文件属性和文件内容）时间来查找文件
-ctime
-mtime 根据文件修改（文件属性和文件内容）时间来查找文件
根据文件内容修改时间来查找文件
-type 查是块设备、目录、字符设备、管道、符号链接、普通文件

-exec 对查找的文件执行Linux命令，不询问用户
格式：-exec 要执行的命令 {} \;
-ok 对查找的文件执行Linux命令，询问用户是否执行
-follow 不包括符号链接
-uid 查找指定的用户id的文件
-user 根据文件所有者查找
-name 根据文件名查找
-size 根据文件大小查找，+n, n, -n
-perm 根据文件权限查找
-a 前后条件都要满足
-o 满足一个条件

find有很多例子，但没写出来

locate

命令功能：借助搜索关键字查找文件或目录

命令格式：locate [选项] 搜索关键字

locate与find的区别

- locate速度远胜find
- find搜索整个目录，locate搜索数据库/var/lib/locatedb
- 即便文件存在，数据库中没有记录，locate便搜索不到

grep

命令功能：在文件中搜索与字符串匹配的行并输出
命令格式：grep 指定字符 源文件
示例：grep itheima /etc/passwd

文件权限

chmod

chown

chgrp

权限管理

根据用户与文件的关系，Linux系统中将用户分为文件或目录的拥有者、同组用户、其他组用户和全部用户；又根据用户

对文件的权限，将用户权限分为读取权限（read）、写入权限（ write）和执行权限（ execute）。

权限	对应字符	文件	目录
读权限	r	可查看文件内容	可以列出目录中的内容
写权限	w	可修改文件内容	可以在目录中创建、删除文件
执行权限	x	可执行该文件	可以进入目录

使用权限管理命令要求用户具有执行相应命令的权限，为保证命令可成功执行，先使用su命令，将用户切换为root。

chmod

命令原意：change the permissions mode of file
命令功能：变更文件或目录的权限
命令格式：chmod {augo}{+ -=} 文件或目录
示例：chmod u-x, g-x file

chown

命令原意: change the owner of file
命令功能: 变更文件或目录的所有者
命令格式: chown 用户 文件或目录
示例: chown root file

chgrp

命令原意: change file group
命令功能: 变更文件或目录的所有者
命令格式: 更改文件过目录的所属组
示例: chgrp root itheima

文件压缩解压缩

tar

zip/unzip

gzip/gunzip

bzip2/bunzip2

Linux系统中压缩包的后缀有.zip、.gz、.bz2、.xz等，后缀不同表示压缩方式不同，使用的压缩命令也不同。压缩与解压成对使用，生成上述几种形式的压缩包所用的命令分别为zip、gzip、bzip2、xz，解压缩时使用的命令分别为unzip、gunzip、bunzip2、xz。此外，Linux系统中提供了专门用于打包文件的命令——tar。

tar

命令功能: 打包多个目录或文件，通常与压缩命令一起使用
命令格式: tar [选项] 目录

选项	说明
-c	产生.tar打包文件
-v	打包时显示详细信息
-f	指定压缩后的文件名
-z	打包，同时通过gzip指令压缩备份文件，压缩后格式为.tar.gz
-x	从打包文件中还原文件。

示例:

```
#将目录test下的文件打包
tar -cvf test.tar ./test
#将目录test下的文件打包，并以gzip命令将包压缩
tar -zcvf test.tar.gz ./test
#将目录test下的文件打包，并以bzip2命令将包压缩
tar -jcvf test.tar.bz2 ./test
#从包test.tar.bz2中还原文件
tar -xvf test.tar.bz2
```

zip/unzip

命令功能：压缩文件，获得.zip格式的压缩包，压缩后保存源文件
命令格式：zip [-r] [压缩后文件名称] 文件或目录

选项	说明
-j	只保留文件名称及其内容，不存放任何目录名称
-m	文件压缩完成后，删除原始文件
-o	以压缩文件内拥有最新更改时间的文件为准，更新压缩文件的更改时间
-r	当参数为目录时，递归处理目录下的所有文件和子目录

命令功能：解压文件
命令格式：unzip [选项] 压缩包包名

选项	说明
-l	显示指定压缩包中所包含的文件
-M	将输出结果送到more程序处理
-n	解压时不要覆盖原有文件
-o	命令执行后直接覆盖原有文件
-d	指定解压后文件要存放的目录

gzip/gunzip

命令功能：压缩文件，获得.gz格式的压缩包，压缩后不保存源文件，若同时列出多个文件，则每个文件会被单独压缩。
命令格式：gzip [选项] 文件
gunzip [选项] 压缩包包名

bzip2/bunzip2

命令功能：压缩文件，获得.bz2格式的压缩包，使用选项-k时保留源文件
命令格式：bzip2 [选项] 文件或目录
bunzip2 [选项] 压缩包包名

示例：

```
# bzip2 a.txt
# bzip2 -d a.txt.bz2
# bunzip2 a.txt.bz2
```

进程管理

ps

ps是“Process Status”的缩写。在命令行输入“ps”后回车就能查看当前系统中正在运行的进程。ps的命令格式如下：
ps [选项] [参数]

选项	说明
a	显示当前终端机下的所有进程，包括其它用户启动的进程
u	以用户的形式，显示系统中的进程
x	忽视终端机，显示所有进程
e	显示每个进程使用的环境变量
r	只列出当前终端机中正在执行的进程

top

top的命令格式如下：

top [选项]

top命令可以实时观察系统的整体运行情况，默认时间间隔为3秒，即每3秒更新一次页面，类似Windows系统中的任务管理器，是一个很实用的系统性能监测工具。

pstree

pstree命令以树状图的形式显示系统中的进程，可以直接观察到进程之间的派生关系。pstree命令的格式如下：

pstree [选项]

pstree命令中的常用选项如表所示。

选项	说明
-a	显示每个进程的完整命令（包括路径、参数等）
-c	不使用精简标识法
-h	列出树状图，特别标明当前正在执行的进程
-u	显示用户名称
-n	使用程序识别码排序（默认以程序名称排序）

pgrep

pgrep命令根据进程名从进程队列中查找进程，查找成功后默认显示进程的pid。pgrep命令的格式如下：

pgrep [选项] [参数]

选项	说明
-o	仅显示同名进程中pid最小的进程
-n	仅显示同名进程中pid最大的进程
-p	指定进程父进程的pid
-t	指定开启进程的终端
-u	指定进程的有效用户id

nice

nice命令用于设置Linux系统中进程的nice值，该命令的格式如下：

nice [选项] [参数]

nice选项常用的选项为-n，n表示优先级，是一个整数。

示例：

```
# nice -n 5 bash
```

```
# nice -n 6 top
```

jobs

使用jobs命令可以查看Linux系统中的作业列表及作业状态，进程中的作业也有编号，编号从1开始。Linux系统中作业从用户角度进行编号，进程从系统管理员的角度进行编号。jobs命令的语法格式如下：

jobs [选项] [参数]

当选项和参数缺省时，默认显示作业编号、作业状态和启动作业的命令。

选项	说明
-l	显示进程号
-p	仅显示作业对应的进程号
-n	显示作业状态的变化
-r	仅显示运行状态的任务
-s	仅显示停止状态的任务

bg/fg

Linux系统中可以使用bg命令和fg命令，使进程在前台和后台之间进行切换。

bg命令的作用是，将进程放入后台运行，使前台可以执行其它任务。其命令格式如下：

bg 参数

fg命令的作用与bg相反，是将后台的进程调往前台。其命令格式如下：

fg 作业号

kill

kill命令用来终止正在运行的进程，它的工作原理是发送某个信号到指定进程，以终止该进程。kill的命令格式如下：

kill [选项] [参数]

kill命令的选项一般是“-信号编号”，参数一般是PID。除管理进程外，kill命令也可用于查看系统中的信号。

服务与网络管理

Linux系统提供了特定的命令，以便管理各种服务，在CentOS7之前的版本中，用于管理服务的命令为service和chkconfig，CentOS7及其之后的版本统一使用systemctl命令管理服务。

systemctl命令实际上是service和chkconfig命令的组合，该命令的格式如下：

systemctl 功能 服务名称

systemctl命令功能的基础取值与含义如下：

enable：使指定服务开机自启；

disable：取消指定服务开机自启；

start：启动指定服务；

stop：停止指定服务；

status：检查指定服务运行情况，列出该服务的详细信息；

restart：重启指定服务；

reload：重新加载指定服务的配置文件

- 1、关闭http服务
systemctl stop httpd
- 2、开启http服务
systemctl start httpd
- 3、取消http服务开机自启
systemctl disable httpd
- 4、使http服务开机自启
systemctl enable httpd
- 5、查看http服务的运行状态
systemctl status httpd

Ch2 Linux shell编程基础

Shell功能

命令解释器：连接用户和内核、用户应用程序
文件名替换，命令替换，变量替换
历史替换，别名替换
流程控制的内部命令

变量使用

变量命名
首个字符不能以数字开头
其余字符可以是字母，数字，下划线
中间不能有空格，可以使用下划线 _。
不能使用标点符号。
不能使用bash里的关键字（可用help命令查看保留关键字）

变量赋值

使用格式：NAME=value
使用举例：
addr=20.1.1.254
_school="qust"

变量引用

使用格式：\$NAME或 \${NAME}
使用举例：
\$addr
\${_school}.edu.cn

变量删除

使用格式：unset NAME
变量被删除后不能再次使用，不能删除只读变量
使用举例：
unset addr

变量输入命令read

从标准输入读入一行内容赋值给变量

使用格式: read name

参数: -p 设置提示信息

使用举例:

```
read -p "input your name" NAME
```

变量输出命令echo

使用格式: echo arg1 arg2 arg3 ...

使用举例:

```
echo "Hello world"
echo "\"Hello world\""
echo "$name Hello world"
echo -e "Hello! \n"
echo -e "Hello! \c"
```

变量输出命令printf

使用格式: printf format-string [arguments...]

格式字符:

%s 输出一个字符串

%d 整型输出

%c 输出一个字符

%f 输出实数, 以小数形式输出

使用举例:

```
printf "Hello, Shell\n"
printf "%d %s\n" 1 "abc"
```

字符串常见操作表达式

字符变量	含义
\${#var}	返回字符串变量var的长度
\${var:m}	返回\${var}中从第m个字符之后的所有字符
\${var:m:len}	返回\${var}中从第m个字符之后, 长度为len的子串
\${var#pattern}	删除\${var}中开头部分与pattern匹配的子串
\${var%pattern}	删除\${var}中结尾部分与pattern匹配的子串

字符变量	含义
\${var/old/new}	用new替换\${var}中第一次出现的old子串
\${var//old/new}	用new替换\${var}中所有的old子串
\${var/#old/new}	用new替换\${var}中开头与old匹配的子串
\${var%old/new}	用new替换\${var}中结尾与old匹配的子串

变量使用举例

```
echo "===examples for testing read==="
echo -e "What is your name?\c"
read NAME
echo "Hello $NAME"
echo
echo -n "Where are you work?"
read
echo "I guess $REPLY keeps you busy!"
read -p "Enter your job title:"
echo "I thought you might be an $REPLY"

#=====

===examples for testing read===
What is your name?liuxinyu
Hello liuxinyu

Where are you work?QUST
I guess QUST keeps you busy!
Enter your job title:TTT
I thought you might be an TTT
```

```
var="Hello shell,this is a testing of string"
echo "The string is:  ${var}"
echo "The sting length is:  ${#var}"
echo "The No. 5 character is:  ${var:4:1}"
echo "From No.2 character is:  ${var:1}"
echo "Delete 'H' is :  ${var#H}"
echo "Replace the first 'l' with 'L' is :  ${var/l/L}"
echo "Replace all 'l' with 'L' is :  ${var//l/L}"

#=====
The string is:  Hello shell,this is a testing of string
The sting length is:  39
The No. 5 character is:  o
From No.2 character is:  ello shell,this is a testing of string
Delete 'H' is :  ello shell,this is a testing of string
Replace the first 'l' with 'L' is :  HeLlo shell,this is a testing of string
Replace all 'l' with 'L' is :  HeLLo sheLL,this is a testing of string
```

本地变量

在脚本或命令中定义，仅在当前shell实例中有效，其他shell启动的程序不能访问局部变量。

环境变量

环境变量定义

NAME=value（定义）

export NAME（导出，非常重要）

echo \$NAME（使用）

unset NAME（清除）

env命令查看所有环境变量

常见环境变量

PATH

用于帮助Shell找到用户所输入的命令。

PWD和OLDPWD

PWD记录当前系统目录；OLDPWD记录旧有的工作目录。

HOME

HOME记录当前用户的根目录，用于保存用户自己的文件。

SHELL

SHELL变量保存Shell的值，默认的Shell值/bin/bash。

USER和UID

USER和UID是保存已登录用户的名字和ID。

PS1和PS2

PS1和PS2称为提示符变量，用于设置提示符格式

环境变量配置文件

系统级：对所用用户有效

/etc/profile

/etc/profile.d

/etc/bashrc

用户级：对当前用户有效

~/.bash_profile

~/.bashrc

位置变量

位置变量用来向Shell脚本传递参数

它是以单个整数来命名的：\$n

\$n表示第n个位置参数

注意：\$符号不能少

例：\$0, \$1, \${10}

特殊变量

\$#: 传递到脚本的参数个数。

\$*和\$@ : 传递到脚本的所有参数。

\$? : 命令退出状态：0无错误，非0错误。

\$\$: 表示本程序的进程PID。

!: 上一个后台进程的PID

变量使用举例

```
echo "Hello,$USER,the output of this script are as follows:"
echo "The script name is: $0"
echo "The first param of this script is: $1"
echo "All the params you input are:$@"
echo "All the params you input are:$*"
echo "The number of the params you input are:$#"
echo "The process ID of the params you input are:$$"
echo "The exit status of this script is:$?"
```

```
#=====
Hello,root,the output of this script are as follows:
The script name is: test.sh
The first param of this script is:
All the params you input are:
All the params you input are:
The number of the params you input are:0
The process ID of the params you input are:3164718
The exit status of this script is:0
```

引号

- 单引号：它可以屏蔽Shell中元字符的功能

```
[root@node01 ~]# echo 'hello $PATH'
hello $PATH
```

- 双引号：屏蔽，但不会屏蔽\$、\、`符号

```
[root@node01 ~]# echo "$NUM"
10
```

- 反引号：命令替换，可与双引号结合使用。

```
[root@node01 ~]# echo "date:`date`"
date:Wed Dec 15 15:32:04 CST 2021
```

通配符

符号	含义
通配符 “*”	将与零个或多个字符匹配
通配符 “?”	与任何单个字符匹配
通配符 “[]”	与[]中的任一字符相匹配
通配符 “[!]”	与[]之外的任一字符匹配

连接符

“;”连接符

使用“;”运算符，命令会按照先后次序依次执行。比如源码安装gdb包，可使用连接符连接安装命令。

```
#tar -xzf gdb-7.11.1.tar.gz ;cd gdb-7.11.1;
./configure;make;make install;gdb -v
```

“&&”连接符

使用“&&”运算符，其前后命令的执行遵循逻辑与关系，只有该连接符前面的命令执行成功后，它后面的命令才被执行。

“||”连接符

使用“||”运算符，其前后命令的执行遵循逻辑或关系，只有该连接符之前的命令执行失败时，才会执行后面的命令。

其他符号

```
;;case分支终止符  
:空命令, 相当于true  
.相当于source命令  
#注释
```

Shell脚本的基本元素

```
第一行:  
    #!开头, 指定命令解释器。  
    可调用任何解释器。  
第二行:  
    #注释  
第三行:  
    可执行语句。  
  
test.sh:  
  
#!/bin/bash  
#This is an example  
echo"Hello,Bash Shell!"
```

Shell解释器运行脚本

将脚本文件作为参数传递给解释器。这种运行方式只要对文件有读取权限就可以。使用-x参数可以进行调试例:

```
bash test.sh  
bash -x test.sh a b  
sh test.sh
```

通过source运行脚本

source是Shell内部命令, 它的功能是读取指定Shell程序文件, 并依次执行其中的所有语句。

```
例: source test.sh  
    . test.sh
```

直接运行脚本 (有权限)

赋予脚本执行权限, 然后直接运行脚本。

它的运行原理与第一种方式相同, 都是先调用解释器创建一个子Shell, 然后由解释器解释执行脚本程序。

```
例:  
    chmod u+x test.sh  
    ./test.sh
```

运算符

算术运算符
expr指令
let指令
自增自减运算符++、--
算术运算符：+ - * / %
算术表达式求值：\$[表达式] 或者 \$((表达式))
使用举例：
 num1=\$((4+1))
 num2=\$((2+\$num1*5-3))

expr指令

求表达式变量的值，一般用于整数值，也可用于字符串
用空格隔开每个项；用反斜杠 \ 放在 shell 特定的字符前
使用举例：
 expr length "this is a test"
 expr 14 % 9
 expr 5 * 9

let指令

用于执行一个或多个表达式，变量计算中不需要加上 \$ 来表示变量
使用举例：
 let a=5+4
 let b=9-3
 echo \$a \$b
 let a++

条件测试语句

文件测试
整数测试
字符串测试
逻辑运算
命令返回值：为0表示真、非0表示假 \$?

test

语法格式：test 空格<测试表达式>
例：test -f file && echo true || echo false

[] 或 [[]]

语法格式：[<测试表达式>]
注意：方括号前后都要留空格
例：[-f file] && echo true || echo false

[[]] 支持通配符和正则表达式

语法格式：[[<测试表达式>]]
注意：方括号前后都要留空格
例：[[\$name = [Tt]?*]] && echo true || echo false

文件测试操作符

操作符	说明	举例
-b file	检测文件是否是块设备文件	[-b \$file]
-c file	检测文件是否是字符设备文件	[-c \$file]
-d file	检测文件是否是目录	[-d \$file]
-f file	检测文件是否是普通文件	[-f \$file]
-L file	检测文件是否是链接文件	[-L \$file]
-p file	检测文件是否有名管道	[-p \$file]
-r file	检测文件是否可读	[-r \$file]
-w file	检测文件是否可写	[-w \$file]
-x file	检测文件是否可执行	[-x \$file]
-s file	检测文件是否为空（文件大小是否大于0）	[-s \$file]
-e file	检测文件（包括目录）是否存在	[-e \$file]

条件测试语句

```
touch shellfile
ll shellfile
[ -f shellfile ]
echo "File shellfile is a regular file?:$?"      File shellfile is a regular file?:0
[ -e shellfile1 ]
echo "File shellfile1 exists?:$?"              File shellfile1 exists?:130
[ -d shellfile ]
echo "File shellfile is a directory?:$?"      File shellfile is a directory?:0
[ -s shellfile ]
echo "File shellfile is not empty?:$?"      File shellfile is not empty?:0
[ -w shellfile ]
echo "File shellfile can write?:$?"      File shellfile can write?:130
```

整数测试

```
整数测试通常用于数值之间的运算
语法格式：[ 整数1 操作符 整数2 ]
例：$i -eq 0
```

整数测试操作符，中括号和双中括号[]、[[]]

操作符	含义
-eq	等于 (Equal)
-ne	不等于 (Not Equal)
-gt	大于 (Greater Than)
-lt	小于 (Lesser Than)
-le	小于或等于 (Lesser or Equal)
-ge	大于或等于 (Greater or Equal)

也可以使用C语言中的关系运算符，用于比较两个变量的大小，比较的结果是一个布尔值，即true或false。注意要用双小括号(())。

符号	含义
==	等于 (Equal)
!=	不等于 (Not Equal)
>	大于 (Greater Than)
<	小于 (Lesser Than)
<=	小于或等于 (Lesser or Equal)
>=	大于或等于 (Greater or Equal)

字符串测试

字符串测试操作符的作用包括比较字符串是否相同、测试字符串的长度是否为0。
语法格式：[字符串1 = 字符串2]、[字符串1 != 字符串2]或[-z 字符串]
例：\$user !=root

符号	含义
-z	判断字符串长度是否为0
-n	判断字符串长度是否为非0
!=	判断两个字符串是否不相等
=/=	判断两个字符串是否相等

逻辑运算符测试

语法格式：[表达式1] 操作符 [表达式2]
或：命令1 操作符 命令2
例：[\$x -eq 1 -a -n \$name]

运算符	含义
-a或 &&	判断操作符两边均为真，结果为真，否则为假，“逻辑与”
-o或	判断操作符两边一边为真，结果为真，否则为假，“逻辑或”
!	判断操作符两边均为假，结果为真，否则为假，“逻辑否”

条件测试语句—逻辑测试举例

```
x=1;name=Tom;[ $x -eq 1 -a -n $name ] && echo T|| echo F
n=5;m=7;[ $n -gt $m ] && echo T|| echo F
n=5;m=7;((($n < $m ))&& echo T|| echo F
#=====
T
F
T
```

判断结构

if结构

if语句是最简单的判断结构，它对条件进行判断，如果判断结果为真。

```
if expression      if expression; then
then              命令语句
    命令语句      fi
fi
```

if/else结构

if/else结构指如果满足条件则执行then后面的操作，如果不满足条件，则执行else后面的操作。

```
if expression; then
    命令语句1
else
    命令语句2
fi
```

if/elif/else结构

if/elif/else结构可以进行多种情况的判断。

```
if expression1; then
    命令语句1
elif expression2; then
    命令语句2
.....
else
    命令语句n
fi
```

if语句举例

```
if [[ $answer = [Yy] || $answer = [Mm]aybe ]]
then
    echo "Glad to hear it."
fi
```

```
if [ $# -eq 1 ]
then
    if who|grep $1>/dev/null
    then echo "$1 is active"
    else echo "$1 is not active"
    fi
else
    echo "Usage:$0:<username>"
    exit
fi
```

```
#!/bin/bash
echo "please input score(0-100):"
read score
if [ $score -lt 0 -o $score -gt 100 ]; then
    echo error score
elif [ $score -ge 90 ]; then
    echo A
elif [ $score -ge 80 ]; then
    echo B
elif [ $score -ge 70 ]; then
    echo C
else
    echo D
fi
```

if/else使用总结

if语句必须以if开头，以fi结尾
elif可以有任意个（0个或多个）
else最多只有一个（0个或1个）
命令语句如果为空，使用冒号，返回退出状态0
expression如果为命令，以最后一个命令的退出状态为条件值

case结构

case结构也用于进行判断。

```
case "$var" in
var1)
    命令语句1;;
.....
*)
    命令语句n;;
esac
```

case语句举例

```
echo "What is your preferred scripting language?"
read -p "1)bash 2)perl 3)python 4)ruby:" lang
case $lang in
    1)echo "you select bash" ;;
    2)echo "you select perl" ;;
    3)echo "you select python" ;;
    4)echo "you select ruby" ;;
    *)echo "I do not know" ;;
esac
```

```
echo "Which is your preferred scripting language?"
read -p "1)bash 2)perl 3)python 4)ruby:" lang
case $lang in
    [Bb])echo "you select bash" ;;
    [Pp])echo "you select perl or python" ;;
    [Rr])echo "you select ruby" ;;
    *)echo "I do not know" ;;
esac
```

case使用总结

按顺序匹配，匹配成功则执行该分支的所有命令
没有匹配的模式，则执行默认值*)后的命令，可以没有
每个分支最后必须以双分号结束

循环结构

for循环1

for循环是Shell编程中最常用的循环，它通常从一个列表中读取元素赋予变量，然后执行循环体中的命令语句，执行完毕之后再次从列表中读取元素赋予变量，再次执行循环体中的命令语句，如此循环直到列表中元素被读取完毕。

```
for var in {list}
do
    命令语句
done
```

list可以是命令替换、变量替换、字符串和文件名，可以包含通配符，用空格间隔

如果用for循环遍历一个列表，则有三种写法：

- (1) for var in 1 2 3 4 5 6 #1~6列表
- (2) for var in {1 ..6} #1~6列表简写形式
- (3) for var in {1 ..6..2} #以2为步长，读取1~6列表

for循环2

```
for var
do
    命令执行语句
done
```

shell自动将命令行输入的所有参数依次组成列表

for循环3

```
for((expression1; expression2; expression3))
do
    命令执行语句
done
```

C语言语法的for循环

for语句举例

使用字符串列表

```
for x in centos ubuntu gentoo opensuse
do
    echo $x
done
```

使用命令组合列表

```
for x in ls "du -sh" "df -h"
do
    echo "=== $x ==="
    eval $x
done
```

使用当前目录下的文件名和目录名列表

```
for fname in *
do
    echo "`ls -l $fname`"
done
```

使用命令替换列表

```
for filename in `ls`
do
    echo $filename
done
```

使用命令行输入参数列表

```
i=1
for day
do
    echo -n "Positional parameter $((i++)):$day"
    case $day in
        [Mm]on|[Tt]ue|[Ww]ed|[Tt]hu|[Ff]ri)
            echo "(weekday)"
            ;;
        [Ss]at|[Ss]un)
            echo "(weekend)"
            ;;
        *)
            echo "Invalid weekday"
            ;;
    esac
done
```

批量添加用户

```
for x in {1..10}
do
    useradd user${x}
    echo "centos"|passwd --stdin user${x}
    chage -d 0 user${x}
done
```

批量添加用户

```
for x in {1..50}
do
    if ((x<10))
    then name="user0"${x}
    else name="user"${x}
    fi
    useradd ${name}
done
```

循环结构

while循环

while循环要简单一些，它只有一个判断条件，条件为真则执行循环体中的命令语句，条件为假则退出循环。

```
while expression
do
    命令语句
done
```

while语句举例

```
num=1
while (( $num <= 5 ))
do
    echo $num
    let num++
done
```

while通过read读取文件内容

```
cat /etc/resolv.conf |while read line
do
    echo $line
done
```

while循环读取命令行参数

```
echo "The count of arguments is $#"  
echo "arguments list:"  
while [ $# -ne 0 ]  
do  
    echo $1  
    shift  
done
```

until循环

until循环的格式与while循环类似，但在判断循环条件时，只有循环条件为假才会去执行循环体，如果循环条件为真，则退出循环。

```
until expression  
do  
    命令语句  
done
```

until语句举例

```
#!/bin/bash  
num=1  
until [ $num -gt 5 ]  
do  
    echo $num  
    let num++  
done
```

测试IP地址是否ping通，连通则ssh登录

```
#!/bin/bash  
read -p "Enter IP address:" addr  
echo $addr  
until ping -c 1 $addr > /dev/null  
do  
    sleep 60  
done  
ssh $addr
```

break与continue

在循环过程中，如果达到某一个条件需要退出循环，就需要使用循环控制符跳出循环，Shell中提供了两个循环控制符：break和continue。

break: 退出循环

continue: 结束本次循环

exit与sleep

exit: 退出脚本或当前进程
sleep: 指定休眠时间

select循环

`select`命令是一种菜单扩展的循环方式。当程序运行到`select`语句时，会自动将列表中的所有元素生成为可用1、2、3依次类推的选择列表，并等待用户输入。

select Menu in (list) do command done

当用户输入值并回车后，`select`可根据用户输入的值判断并执行后续的命令。如果用户不做输入，直接回车的话，`select`再次生成列表等待输入。

select语句举例

```
#!/bin/bash
PS3="What is your preferred scripting language? "
select s in bash perl python php ruby quit
do
    case $s in
        bash|perl|python|php|ruby) echo "You select $s";;
        quit) exit;;
        *) echo "You select error,reply!";;
    esac
done
```

```
PS3="Select a program you want to execute: "
TOPLIST="python telnet htop atop nettop jnettop iftop ftop iotop mytop"
select prog in $TOPLIST
do
    case $REPLY in
        1|2|3|4|5|6|7|8|9|10)
            rpm -q $prog >/dev/null &&echo $prog ||echo "$prog is not installed";;
        *) exit;;
    esac
done
```

shell函数语法

```
[ function ] 函数名 [()]
{
    commands;

    [return int;]
}
```

```
test()
{
    echo "test function";
}
```

语法格式:

函数名

函数名 参数1 参数2

函数存储

函数和调用程序保存在同一个文件：函数定义在前，调用在后

函数和调用程序分别存放在不同文件，函数文件需要先source命令执行

函数显示

`declare -F` 显示shell可见函数名

`declare -f` 显示shell函数定义

shell函数参数

调用函数时可以向其传递参数。在函数体内部，通过 `$n` 的形式来获取参数的值

`$0`在主程序和函数中都表示脚本名

函数调用结束后，`$n`、`$*`将被重置为调用函数之前

```
echo "===Print positional parameters in main :"  
echo "$0: $*"
pp1(){  
    echo 'f1--Print $* parameters in fun1:'  
    echo "$0: $*"
}  
pp2(){  
    echo 'f2--Print $* parameters in fun2:'  
    echo "$0:$*"
    pp1 1st 2nd 3rd 4th 5th 6th 7th 8th 9th  
    echo 'f2--Print $* parameters in fun2:'  
    echo "$0:$*"
}  
pp1 1 2 3 4 5 6 7 8 9  
echo "===Print positional parameters in main :"  
echo "$0:$*"
pp2 I II III IV V VI VII VIII IX

#=====
===Print positional parameters in main :  
test.sh:  
f1--Print $* parameters in fun1:  
test.sh: 1 2 3 4 5 6 7 8 9  
===Print positional parameters in main :  
test.sh:  
f2--Print $* parameters in fun2:  
test.sh:I II III IV V VI VII VIII IX  
f1--Print $* parameters in fun1:  
test.sh: 1st 2nd 3rd 4th 5th 6th 7th 8th 9th  
f2--Print $* parameters in fun2:  
test.sh:I II III IV V VI VII VIII IX
```

shell函数返回值

函数的返回值为最后一条命令的退出码，保存在\$?中，return的返回值只能返回0-255之间的整数

```
max(){
  if [[ -z $1 || -z $2 ]]
  then
    echo "Need 2 parameters to the function."
    exit
  fi
  [ $1 -eq $2 ] &&
  { echo "The two numbers are equal."
    exit
  }
  [ $1 -gt $2 ] && return $1 || return $2
}
read -p "Please input two integer numbers : " n1 n2
echo "n1=$n1,n2=$n2"
max $n1 $n2
return_val=$?
echo "The larger of the two numbers is $return_val."

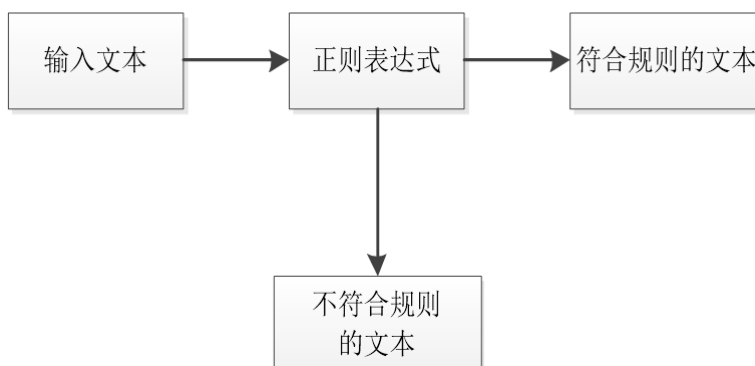
#=====
Please input two integer numbers :1 2
n1=1,n2=2
The larger of the two numbers is 2.
```

正则表达式

正则表达式是对文本进行过滤（分割、匹配、查找、替换）的工具。它定义了一系列的元字符，通过元字符配合其它的字符来表达出一种规则，只有符合规则的文本才能保留下来，而不符合规则的文本则被过滤掉。

与通配符区别

通配符：匹配文件名，完全匹配，ls,find,cp命令
正则表达式：匹配字符串，包含匹配，grep,awk,sed命令



元字符

基础元字符

- (1) 限定符“*”: 用于匹配前面一个普通字符0次或多次重复。
- (2) 单个字符匹配符号“.”: 匹配任意单个字符（除却“\n”）。
- (3) 行首定位符“^”: 用来匹配行首字符。
- (4) 行尾定位符“\$”: 定位文本行的末尾，与“^”作用相反。
- (5) 字符集匹配符号“[]”: 用于指定一个匹配的字符集集合。
- (6) 字符集不匹配符号“[^]”: 不匹配字符集的任意字符。
- (7) 转义符“\”: 屏蔽一个元字符的特殊意义。
- (8) 精确匹配符号“\<\>”: 精确匹配。 \<the\>
- (9) “\{n\}”: 匹配前面字符出现n次。 he1\{2\}o
- (10) “\{n,\}”: 匹配前面字符至少出现n次。 jo\{3,\}b
- (11) “\{n,m\}”: 匹配前面字符出现n次与m次之间 m\{2,5\}o

基础元字符举例

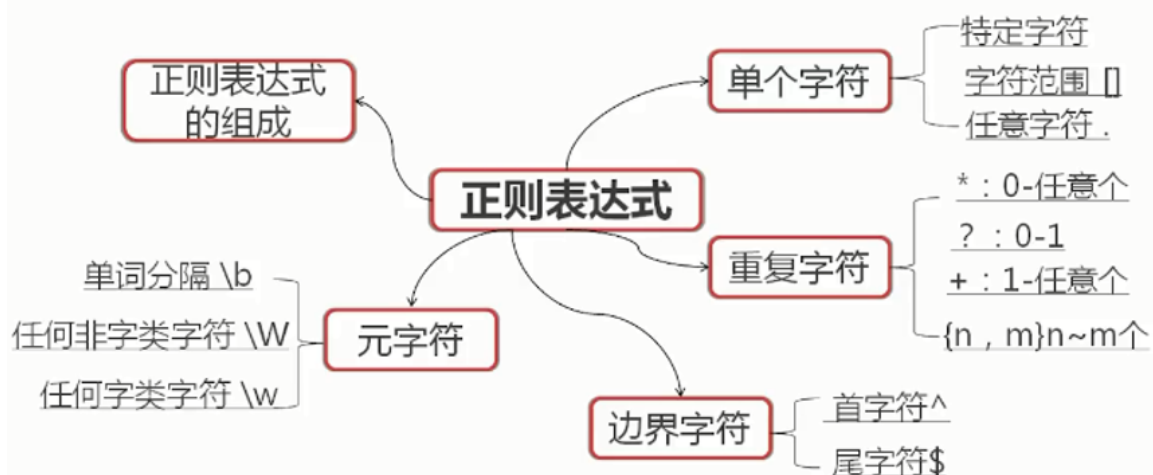
a*: 匹配字符a 0次或多次，相当于所有字符，包括空白字符
aa*: 匹配包含a的字符串
a..: 匹配a+任意两个字符（非换行）的字符串
.*: 相当于a*, 匹配所有字符串
^a: 匹配行首字符为a的字符串
a\$: 匹配行尾为字符a的字符串
^\$: 匹配空白行
a[bc]d: 匹配abd, acd
[0-9]: 匹配0-9中的一个数字
^[a-z]: 匹配行首以小写字母开头的
^[a-zA-Z]: 匹配行首以字母开头的
[^a]: 匹配不包含a的字符串
^[^a-zA-Z]: 匹配行首不是以字母开头的
[^a-z]: 匹配不包含小写字母的字符串
b\{3\}: 匹配字符b出现3次的字符串
b\{3,\}: 匹配字符b至少出现3次的字符串
ab\{3\}c: 匹配包含abbbc的字符串

```
^a$
^a\{2,4\}$
^a\{2,\}$
a\{2,\}$
[a-z]\{5\}
\<lmen\>
^rc[1-9]
```

扩展元字符

- (1) 限定符“+”: 匹配其前面的字符1次或多次。
- (2) 限定符“?”: 限定前面的字符最多出现1次（即0次或1次）。
- (3) “|”符号: 表示多个正则表达式之间的“或”关系。
表达式1 | 表达式2 | ... | 表达式n
- (4) “()”符号: 表示一组可选值的集合。“|”与“()”常结合使用。

正则表达式总结



常用字符簇

```
^[1-9][0-9]\{0,\}$
^-\{0,1\}[0-9]\{1,\}$
[0-9]\{4\}-[0-9]\{2\}-[0-9]\{2\}
[0-9]\{1,3\}\.[0-9]\{1,3\}\.[0-9]\{1,3\}\.[0-9]\{1,3\}
```

正则在线测试: <http://tool.rbtrees.cn/regtool/>

grep命令

在文件中全局查找。
输入流可以是标准输入流、管道和文件

```
ps |grep sshd
grep /bin/bash /etc/passwd
```

找到匹配的表达式:	grep 返回的退出状态为 0
没找到匹配的表达式:	grep 返回的退出状态为 1
找不到指定文件:	grep 返回的退出状态为 2

grep常见选项

- `-i`: 忽略字符大小写
- `-q`: 不输出任何信息
- `-n`: 显示匹配字符串所在行号
- `-c`: 显示成功匹配的行数
- `-v`: 显示不匹配的行
- `--color`: `.bashrc`或者`.bash_profile`文件中加入`alias grep='grep --color=auto'`, 生效后, `grep`的搜索结果自动高亮匹配。

grep命令举例

查找/etc/sysctl.conf文件中以“#”符号开头文本行，并显示每行的行号。

```
grep -n ^# /etc/sysctl.conf
```

搜索/etc/yum.conf文件中的空白行的行数。

```
grep -c ^$ /etc/yum.conf
```

搜索文件/etc/passwd中匹配root的字符串

```
grep root /etc/passwd
```

搜索文件/etc/profile中匹配不是以#开头的行

```
grep -v ^# /etc/profile
```

显示/proc/meminfo文件中以大小写s开头的行

```
grep ^[Ss] /proc/meminfo
```

显示/etc/passwd文件中不以/bin/bash结尾的行

```
grep -v /bin/bash$ /etc/passwd
```

cut命令

显示每行从开头算起 num1 到 num2 的文字

语法

```
cut [-bn] [file]
```

```
cut [-c] [file]
```

```
cut [-df] [file]
```

参数：

-b : 以字节为单位进行分割。

-c : 以字符为单位进行分割。

-d : 自定义分隔符，默认为制表符。

-f : 与-d一起使用，指定显示哪个区域。

-n : 取消分割多字节字符。仅和 -b 标志一起使用。

使用举例

```
who|cut -b 2-4
```

```
who|cut -b -3
```

```
who|cut -b 3-,-3
```

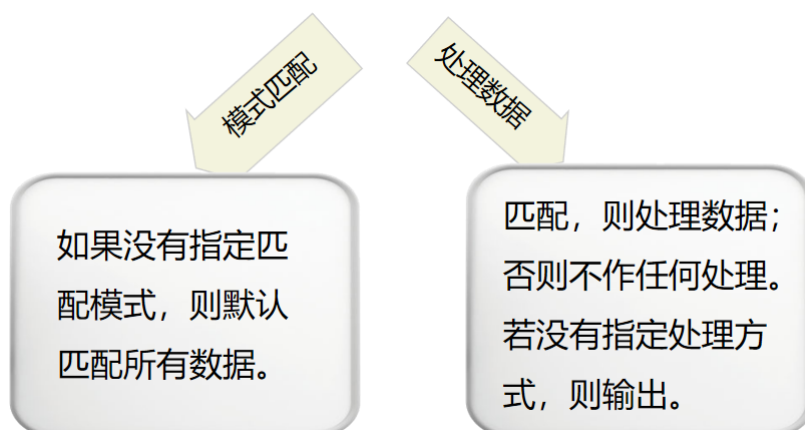
```
cat cut.txt |cut -nb 1
```

```
cat cut.txt |cut -c 1
```

```
cat /etc/passwd|head -n 5|cut -d : -f 1
```

```
cat /etc/passwd|head -n 5|cut -d : -f 1,3-5
```

awk



awk语法格式

`awk` [选项] 'BEGIN{actions} pattern {actions}END{actions}' 文件

执行流程：

第一步：执行BEGIN{actions}语句块中的语句；

第二步：从文件或标准输入（stdin）读取一行，然后执行pattern{actions}语句块，它逐行扫描文件，从第一行至最后一行；

第三步：当读至输入流末尾时，执行END{actions}语句块。

-F 选项：指定作为输入行的分隔符，默认是tab键。

-v 选项：赋值一个用户定义的变量。

-f 选项：从脚本文件读取awk指令。

pattern: 条件

一般使用关系表达式作为条件

actions: 动作

格式化输出语句

流程控制语句

关系表达式

awk允许用户使用关系表达式作为匹配模式，当文本行满足关系表达式时，将会执行相应操作。

运算符	含义	运算符	含义
>	大于	==	等于
<	小于	!=	不等于
<=	小于等于	~	匹配正则表达式
>=	大于等于	!	不匹配正则表达式

逻辑运算符

运算符	运算	范例	结果
!	非	!a	如果a为假，则!a为真
&&	与	a&& b	如果a为真，则!a为假
	或	a b	只要a、b有一个为真，则结果为真；二者都为假，结果为假。

正则匹配模式

匹配正则表达式：

全行数据正则匹配//

全行数据正则匹配取反!//

特定数据正则匹配~//

特定数据正则匹配取反!~//

区间模式

匹配一段连续的文本行。

pattern1, pattern2

从匹配pattern1的文本行开始，到匹配pattern2的文本行结束，中间连续几个文本行都会被匹配到。

awk使用方式

```
awk 'pattern' filename
awk '{action}' filename
awk 'pattern {action}' filename
```

awk使用举例

```
awk '/root/' /etc/passwd
awk '{print }' /etc/passwd
awk '/root/ {print }' /etc/passwd
awk -F: '{print}' /etc/passwd
awk -F: '/root/,/mysql/{print }' /etc/passwd
awk -F: 'BEGIN{print "User infos"}/root/{print}' /etc/passwd
```

awk常用变量

\$0: 当前行的全部字段
\$n: 当前行第n个字段
NR: 行号
NF: 当前行的字段个数
\$NF: 最后一个字段值
FILENAME: 当前文件名

awk常用变量举例

```
awk -F: '/root/{print $0 }' /etc/passwd
awk -F: '/root/{print $1,$3 }' /etc/passwd
awk -F: '/root/{print "UserName: "$1 }' /etc/passwd
awk -F: '/root/{print NR,NF,$1 }' /etc/passwd
awk -F: '/root/{print NR,NF,$1 }END{print FILENAME}' /etc/passwd
```

awk使用正则表达式

```
^,$,.,*,+,,?,[],[^],(),|,\
awk -F: 'BEGIN{count=0}$1!~/^$/{count++}END{print count}' /etc/passwd
awk '/root/' /etc/passwd
ls -l |awk '$1~/^-/'
```

awk流程控制

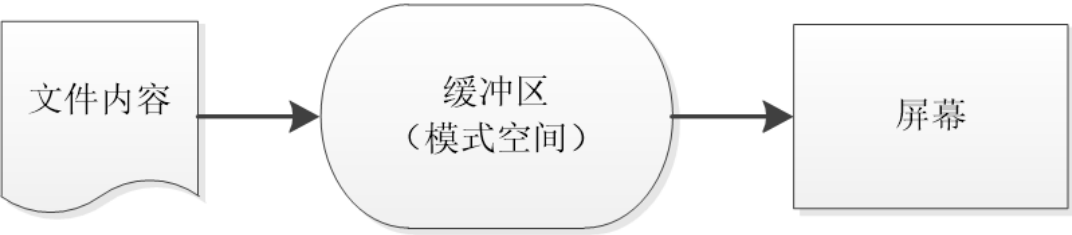
if语句
while
for

awk流程控制举例

```
awk -F: '{if ($3<1000){name="system"}else{name="user"}};{print $1,name}' /etc/passwd
awk -F: '{if ($3==0){++i}else if($3<1000)
{++j}else{k++}};END{print"admin:"i,"system:"j,"user:"k}' /etc/passwd
awk -F: '/^root/{i=1;while(i<NF) {print $i;i++}}' /etc/passwd
```

sed

sed是Linux系统中用于文本处理的一款工具，用于对文本进行过滤和替换，能够完善的结合正则表达式使用。
sed一次读取一行内容进行处理。
它通过文件或管道将内容读取到模式空间进行处理，然后输出到屏幕。



sed语法格式

语法格式: sed [选项] 'sed编辑命令' 输入文件
-n选项: 关闭默认的模式空间输出。
-e选项: 将下一个字符串解析为命令。
-f选项: 编辑脚本内容，表明正在使用脚本。
-r选项: 在脚本指令中使用扩展正则表达式。
-i选项: 直接修改原文件，此选项要慎用。

sed常用编辑命令	
选项	含义
p	打印匹配行
=	显示文件行号
a\	在定位行号之后追加文本信息
i\	在定位行号之前插入文本信息
d	删除定位行
c\	用新文本替换定位文本
s	使用替换模式替换相应模式
r	从一个文件中读文本
w	将文本写入到一个文件
}	在定位行执行命令组

sed行定位

指定行号

```
sed -n '1p' /etc/passwd
```

指定正则匹配

```
sed -n '/root/p' /etc/passwd
```

指定起始行

```
sed -n '1,3p' /etc/passwd
```

```
sed -n '1,/daemon/p' /etc/passwd
```

指定间隔行

```
sed -n '1~3p' /etc/passwd
```

指定最后一行

```
sed -n '$p' /etc/passwd
```

sed操作——打印

p命令

```
sed 'p' /etc/passwd
```

=命令

```
sed '=' /etc/passwd
```

-n选项

```
sed -n 'p' /etc/passwd
```

sed操作——追加文本

sed '行定位a\text' 输入文件

命令实现:

```
$sed '/For rain/a\add a new line!!!' poem
```

sed '行定位i\text' 输入文件

命令实现:

```
$sed '/For rain/i\add a new line!!!' poem
```

sed操作——删除文本

sed '行定位d' 输入文件

命令实现:

```
$sed '4d' poem
```

sed操作——替换文本

sed '行定位c\text' 输入文件

命令实现:

```
sed '3c -----' poem.txt
```

```
sed '3,5c -----' poem.txt
```

```
sed '3~5c -----' poem.txt
```

sed操作——查找替换

sed 's行定位/text1/text2/' 输入文件

命令实现:

```
sed 's/is/****/' poem.txt
```

```
sed 's/is/****/g' poem.txt
```

sed操作——保存文件

```
w命令
>重定向
sed '行定位w 新文件' 输入文件
命令实现:
sed '3w file' poem.txt
sed 's/is/****/g' poem.txt > file
```

sed应用

```
禁用selinux
sed -i '/SELINUX/{s/permissive/disabled/}' /etc/selinux/config
禁止root远程登录
sed -i 's/#PermitRootLogin yes/PermitRootLogin no/' /etc/ssh/sshd_config
```

shell脚本运维实例

模拟用户登录

```
#!/bin/bash
echo "user:"
read user
echo "passwd:"
read passwd
if [ "$user" = "qust" ] && [ "$passwd" = "123456" ]; then
echo "SUCCESS!"
else echo "user name or passwd ERROR!"
fi
```

监控系统运行情况

```
#!/bin/bash
echo " (1) *****"
date;
echo "Active User:"
w
echo " (2) *****"
echo "Linux process:"
top -b | head -6
echo " (3) *****"
echo "Disk and Memory use ratio:"
df -h |xargs | awk '{print "Free/total disk:"$11/"$9}'
free -m | xargs | awk '{print "Free/total memory:" $10/"$8"MB}'
echo " (4) *****"
echo "All Processes:"
ps auxf
echo " (5) *****"
echo "vmstat:"
vmstat 1 5
echo " (6) *****"
echo "socket for ssh process
ss -pl |grep sshd
```

统计进程的状态情况

```
#!/bin/bash
running=0
sleeping=0
stopped=0
zombie=0
procs=0
for pid in /proc/[1-9]*
do
    procs=$((procs+1))
    stat=$(awk '{print $3}' $pid/stat)
    case $stat in
        R)running=$((running+1));;
        T)stopped=$((stopped+1));;
        S)sleeping=$((sleeping+1));;
        Z)zombie=$((zombie+1));;
    esac
done
echo "进程统计信息如下: "
echo "总进程数为: $procs"
echo "Running 进程数为: $running"
echo "Stopped 进程数为: $stopped"
echo "Sleeping 进程数为: $sleeping"
echo "Zombie进程数为: $zombie"
```

Ch3 Linux编程基础

GCC编译器

GCC编译器介绍

GNU C Compiler -> GNU Compiler Collection

<http://gcc.gnu.org>

GCC是一个强大的工具集合，它包含了预处理器、编译器、汇编器、链接器等组件。它会在需要的时候调用其他组件。输入文件的类型和传递给gcc的参数决定了gcc调用具体的哪些组件。

GCC 参数选项

- Usage:
 - gcc [options] [filename]
- Basic options:
 - -E: 只对源程序进行预处理(调用cpp预处理器)
 - -S: 只对源程序进行预处理、编译
 - -c: 执行预处理、编译、汇编而不链接
 - -o output_file: 指定输出文件名
 - -g: 产生调试工具必需的符号信息
 - -O/On: 在程序编译、链接过程中进行优化处理
 - -Wall: 显示所有的警告信息
 - -I dir : 在头文件的搜索路径中添加dir目录
 - -L dir :在库文件的搜索路径列表中添加dir目录

GCC编译过程

一般情况下，C程序的编译过程为

- 1、预处理
- 2、编译成汇编代码
- 3、汇编成目标代码
- 4、链接

预处理

预处理：使用-E参数

```
gcc -E -o gcctest.i gcctest.c
```

使用wc命令比较预处理后的文件与源文件，可以看到两个文件的差异

编译成汇编代码

预处理文件→汇编代码

使用-S说明生成汇编代码后停止工作

```
gcc -S -o gcctest.s gcctest.i
```

直接编译到汇编代码

```
gcc -S gcctest.c
```

编译成目标代码

汇编代码→目标代码

```
gcc -x assembler -c gcctest.s
```

直接编译成目标代码

```
gcc -c gcctest.c
```

使用汇编器生成目标代码

```
as -o gcctest.o gcctest.s
```

编译成执行代码

目标代码→执行代码

```
gcc -o gcctest gcctest.o
```

直接生成执行代码

```
gcc -o gcctest gcctest.c
```

GCC编译优化

优化编译选项有：

-O0

缺省情况，不优化

-O1

-O2

-O3

等等

头文件和库函数目录

GCC -I dir 参数使用

头文件和gcc不在同一目录下，用 `-I dir`指明头文件所在的目录。
`#include <>`: 在默认路径“/usr/include”中搜索头文件
`#include ""`: 在本目录中搜索

例子

```
1-3.c //main程序

#include <my.h>      /*文件预处理，包含自定义的库文件"my.h"*/
int main()
{
    char ch;
    while((ch=getchar())!=EOF)    //ctrl+D
        putchar(ch); /
    return 1;
}

*my.h程序：自定义的头文件
#include <stdio.h>    /*文件预处理，包含标准输入输出库*/

运行结果？

解决办法：
1. gcc opt.c -o opt -I ./
2. 修改main
    #include <my.h>
    #include "my.h"
```

GCC创建函数库

函数库：公用函数定义为函数库，供其他程序使用。函数库分为静态库和动态库。
静态库：程序编译时会链接到目标代码中，程序运行时不再需要静态库。程序生成的可执行程序比较大。后缀名为“`.a`”
动态库：程序编译时不会链接到目标代码，在程序运行时载入，运行时需要动态库存在。动态库可方便多个程序共享一个函数库。后缀名为“`.so`”

函数库的生成：由编译过的`.o`文件生成。

创建静态库：

- 1.将需要生成函数库的函数执行`gcc -c`，生成`.o`文件
`gcc -c hello.c`
- 2.由`.o`文件创建静态库，静态库命名格式为：`lib静态库名.a`
`ar ars libmyhello.a hello.o`
- 3.使用静态库：在调用静态库的程序编译时指定静态库名
`gcc -o hello main.c -L./ -lmyhello.a`
`./hello`

Linux系统上的静态库是以`.a`为后缀的特殊的存档。

GCC创建动态库

1. 由.o文件生成动态库，动态库的命名：**lib**动态库名**.so**

```
gcc -shared -fPIC -o libmyhello.so hello.o
gcc -shared -fPIC -o libmyhello.so hello.c    (centos版本)
```

2. 使用动态库：用gcc命令指定动态库名进行编译，编译之前需将动态库文件复制到系统默认库函数目录/usr/lib中或者设置搜索路径。

```
gcc -o hello main.c -L. -lmyhello
```

3. 执行程序前设置（两种方式）：

把当前路径加入 /etc/ld.so.conf中，执行运行ldconfig
export 把当前路径加入环境变量LD_LIBRARY_PATH
export LD_LIBRARY_PATH=.:LD_LIBRARY_PATH

Make工具

Make 引入

文件数量太大,手工gcc编译不方便
仅需要编译已经做了修改的源代码文件;其他文件只需要重新连接
记录哪些文件已改变且需要编译, 哪些文件仅仅需要连接很难

make & makefile

Multi-file project
IDE---Eclipse
make
make & makefile
makefile描述模块间的依赖关系;
make命令根据makefile对程序进行管理和维护; make判断被维护文件的时序关系

make

```
make [-f filename] [targetname]
使用方法:
    make      自动找当前目录下名为Makefile/makefile的文件
    make -f 文件名 找当前目录下指定文件名的文件
```

makefile的组成

显式规则：明确指出目标文件的生成规则
隐式规则：需要make自动推导的规则
变量定义：声明时赋值，引用时加“\$”
文件指示：引用外部的文件
注释：#

Makefile的显式规则

规则

一条规则包含3个方面的内容，

- 1) 要创建的目标（文件），
- 2) 创建目标（文件）所依赖的文件列表；
- 3) 通过依赖文件创建目标文件的命令组

规则一般形式

```
target ... : prerequisites ...  
<tab>command  
<tab>...  
<tab>...
```

每条规则由一个带冒号的“依赖行”和一条或多条以tab开头的“命令行”组成

目标1 [目标2...]:[依赖文件列表]

[\t 命令]

...

ex:

```
make_test:make_main.o wrtlog.o  
<TAB>gcc -o make_test make_main.o wrtlog.o
```

冒号左边是目标,冒号右边是依赖文件

目标和依赖文件均是由字母、数字、句点和斜杠组成的字符串

目标或依赖文件的数目多于一个时，以空格分隔

Makefile举例

make_test:make_main.o wrtlog.o

gcc -o make_test make_main.o wrtlog.o

make_main.o:make_main.c

gcc -o make_main.o -c make_main.c

wrtlog.o:wrtlog.c

gcc -o wrtlog.o -c wrtlog.c

编译目标

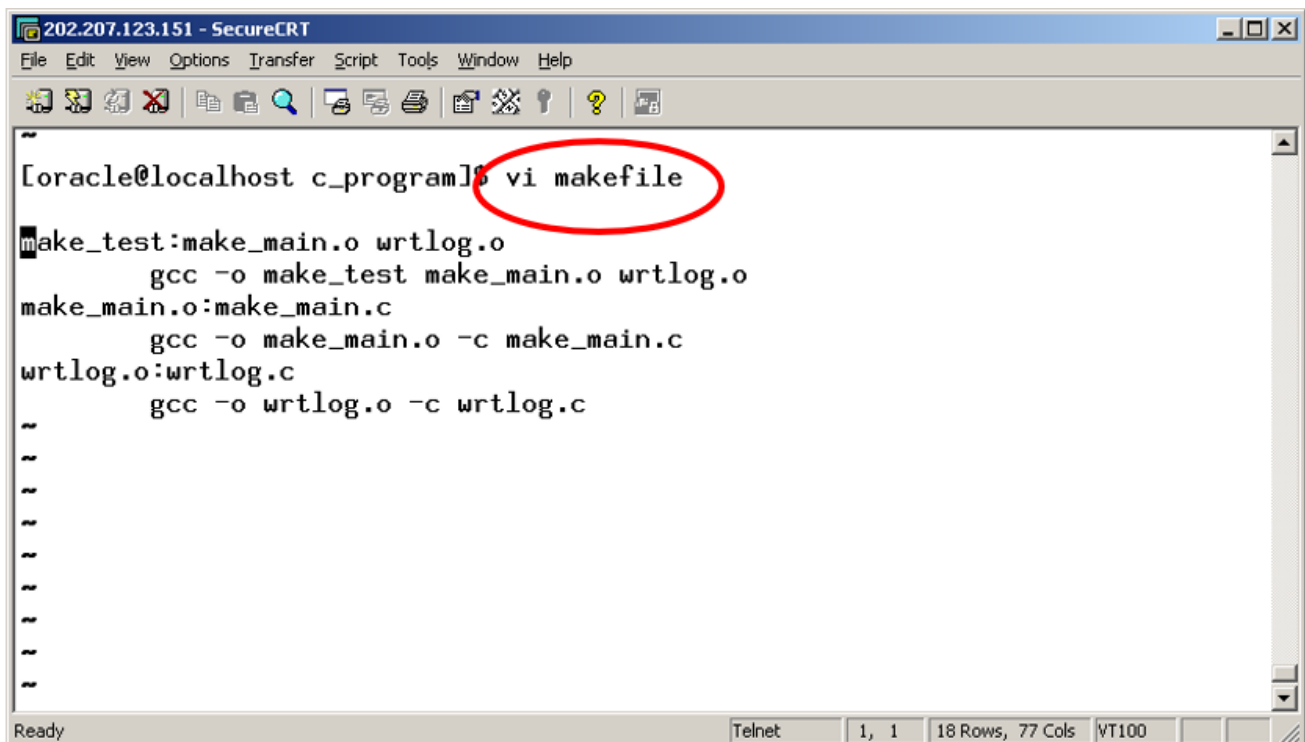
2022-6-14

命令

青岛科技大学信息科学技术学院

依赖文件

60



```
202.207.123.151 - SecureCRT
File Edit View Options Transfer Script Tools Window Help
[oracle@localhost c_program]$ vi makefile
make_test:make_main.o wrtlog.o
    gcc -o make_test make_main.o wrtlog.o
make_main.o:make_main.c
    gcc -o make_main.o -c make_main.c
wrtlog.o:wrtlog.c
    gcc -o wrtlog.o -c wrtlog.c
~
~
~
~
~
~
~
~
~
~
Ready Telnet 1, 1 18 Rows, 77 Cols VT100
```

一个简单的makefile

```
edit : main.o kbd.o command.o display.o insert.o search.o files.o utils.o
    gcc -o edit main.o kbd.o command.o display.o insert.o search.o files.o utils.o
main.o : main.c defs.h
    gcc -c main.c
kbd.o : kbd.c defs.h command.h
    gcc -c kbd.c
command.o : command.c defs.h command.h
    gcc -c command.c
display.o : display.c defs.h buffer.h
    gcc -c display.c
insert.o : insert.c defs.h buffer.h
    gcc -c insert.c
search.o : search.c defs.h buffer.h
    gcc -c search.c
files.o : files.c defs.h buffer.h command.h
    gcc -c files.c
utils.o : utils.c defs.h
    gcc -c utils.c

.PHONY : clean    #伪目标

clean :
    rm edit main.o kbd.o command.o display.o insert.o search.o files.o utils.o
```

Make的工作过程

default goal
在缺省的情况下，make从makefile中的第一个目标开始执行
Make的工作过程类似一次深度优先遍历过程

Makefile的变量

自定义变量

例：

```
object=main.o Add.o Sub.o Mul.o Div.o
exe : $(object)
gcc -o exe $(object)
```

特殊变量

\$@: 表示目标文件；

\$^: 表示所有依赖目标的集合，以空格分隔；

\$<: 表示第一个依赖文件；

Makefile的隐式规则（自动推导）

make能够自动推导文件以及文件依赖关系后面的命令

例：

```
object=main.o Add.o Sub.o Mul.o Div.o
exe : $(object)
gcc -o $@ $(object)
main.o : def.h
```

Makefile的伪目标

makefile使用.PHONY关键字来定义一个伪目标，具体格式为：

.PHONY : 伪目标名称

例：

```
.PHONY : clean
clean :
    rm $(object)
```

Ch4 Linux 文件I/O

文件属性

文件属性数据结构struct stat —文件控制块

```
struct stat {
    mode_t st_mode;      /*file type & mode*/
    ino_t   st_ino;       /*inode number (serial number)*/
    dev_t   st_rdev;      /*device number (file system)*/
    nlink_t st_nlink;     /*link count*/
    uid_t   st_uid;       /*user ID of owner*/
    gid_t   st_gid;       /*group ID of owner*/
    off_t   st_size;      /*size of file, in bytes*/
    time_t  st_atime;      /*time of last access*/
    time_t  st_mtime;     /*time of last modification*/
    time_t  st_ctime;     /*time of last file status change*/
    long    st_blksize;    /*Optimal block size for I/O*/
    long    st_blocks;     /*number 512-byte blocks allocated*/
};
```

命令查看: ls -l file

stat/fstat/lstat 函数

获取文件属性

```
#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>

int stat(const char *file_name, struct stat *buf);
int fstat(int filedes, struct stat *buf);
int lstat(const char *file_name, struct stat *buf);
(Return: 0 if success; -1 if failure)
```

文件类型

Unix/Linux系统支持的文件类型:

Directory(d): 目录文件
Link(l): 链接文件
Pipe(p): 管道文件
Block Device(b): 块设备文件
Character Device(c): 字符设备文件
Regular(-): 普通文件
Socket(s): 套接字文件

查看文件类型

- 使用命令: **ls -l /dev/sda1**

- **shell命令: 文件测试操作符**

-b file	检测文件是否是块设备文件
-c file	检测文件是否是字符设备文件
-d file	检测文件是否是目录
-f file	检测文件是否是普通文件
-L file	检测文件是否是链接文件
-p file	检测文件是否是有名管道

- **st_mode** 存储文件类型和许可权限, 形式如下:

type3 type2 type1 type0 suid sgid sticky rwx rwx rwx

- 用来确定文件类型的宏:

S_ISBLK	--	测试块文件
S_ISCHR	--	测试字符文件
S_ISDIR	--	测试目录
S_ISFIFO	--	测试FIFO
S_ISREG	--	测试普通文件
S_ISLNK	--	测试符号链接

需要带参数运行

例如 ./2-1 file

```

#include<stdio.h>
#include<sys/types.h>
#include<sys/stat.h>
int main(int argc,char *argv[])
{ int i; struct stat statbuf;
  for(i=1;i<argc;i++)
  { printf("%s:",argv[i]);
    if(lstat(argv[i],&statbuf)==-1) { printf("error\n"); continue; }
    if(S_ISDIR(statbuf.st_mode))      printf("%s is a directory\n",argv[i]);
    else if(S_ISREG(statbuf.st_mode)) printf("%s is a regular file\n",argv[i]);
    else if(S_ISBLK(statbuf.st_mode)) printf("%s is a block device\n",argv[i]);
    else if(S_ISCHR(statbuf.st_mode)) printf("%s is a character device\n",argv[i]);
    #ifdef S_ISLNK
    else if (S_ISLNK(statbuf.st_mode)) printf("%s is a link file\n",argv[i]);
    #endif
    #ifdef S_ISSOCK
    else if (S_ISSOCK(statbuf.st_mode)) printf("%s is a socket \n",argv[i]);
    #endif
    else printf("%s is an unknown type file \n",argv[i]);
  }
}
=====
[root@VM-0-4-centos /]# ./file home lib etc
home:home is a directory
lib:lib is a link file
etc:etc is a directory

```

文件存取权限

文件存取权限

修改文件存取权限：命令/程序

默认文件存取权限：umask

目录文件的权限说明：

读的权限：显示目录文件，进入目录

写的权限：目录下创建文件

执行的权限：显示目录文件，进入目录，创建文件

ls命令：读和执行

cd命令：读和执行

touch命令：写和执行

改变文件存取权限——程序

例2.2 设计一个程序，要求把系统中“/home/mylinux”目录下的myfile文件权限，设置成文件所有者可读可写，其他用户只读权限。

```

#include<sys/types.h>
#include<sys/stat.h>
int main ()
{
    chmod("/home/mylinux/myfile",S_IRUSR|S_IWUSR|S_IRGRP|S_IROTH);
    return 0;
}

```

chmod 函数

```
Change permissions of a file

#include <sys/types.h>
#include <sys/stat.h>
int chmod(const char *path, mode_t mode);
int fchmod(int fildes, mode_t mode);// 打开的文件
(Return: 0 if success; -1 if failure)
```

st_mode屏蔽	含义
S_IRUSR(00400)	Read by owner
S_IWUSR(00200)	Write by owner
S_IXUSR(00100)	Execute by owner
S_IRWXU(00700)	Read, write and execute by owner
S_IRGRP(00040)	Read by group
S_IWGRP(00020)	Write by group
S_IXGRP(00010)	Execute by group
S_IRWXG(00070)	Read, write and execute by group
S_IROTH(00004)	Read by others
S_IWOTH(00002)	Write by others
S_IXOTH(00001)	Execute by others
S_IRWXO(00007)	Read, write and execute by others

默认文件存取权限——umask

```
The default permission for newly created files and directories are(root):
新创建的文件和目录的默认权限为（root）：
File: -rw-r--r-- 644
Directory: drwxr-xr-x 755
Why?
umask: an octal number which contains the bits that are NOT set as permission bits.
一个八进制数，其中包含未设置为权限位的位。
Default 002 for normal user, 022 for root.
/etc/profile
```

umask 函数

例2.3 设计一程序，要求设置系统文件和目录的权限掩码。
注意权限掩码影响范围

```
#include<stdio.h>
#include<stdlib.h>
#include<sys/stat.h>
#include<sys/types.h>
int main()
{ mode_t new_umask,old_umask;
  new_umask=0666;
  old_umask=umask(new_umask);
  printf("系统原来的权限掩码是: %o\n",old_umask);
```

```

printf("系统新的权限掩码是: %o\n",new_umask);
system("touch liu1");
printf("创建了文件liu1\n");
new_umask=0444;
old_umask=umask(new_umask);
system("touch liu2");
printf("创建了文件liu2\n");
system("ls liu1 liu2 -l");
return 0;
}

```

#=====

系统原来的权限掩码是: 22

系统新的权限掩码是: 666

创建了文件liu1

创建了文件liu2

```
----- 1 root root 0 Jun 15 09:16 liu1
```

```
--w--w--w- 1 root root 0 Jun 15 09:16 liu2
```

为进程设置文件存取权限屏蔽字，并返回以前的值

```
#include <sys/types.h>
```

```
#include <sys/stat.h>
```

```
mode_t umask(mode_t mask);
```

建立文件，文件的权限为0666-mask;

建立目录，目录权限为0777-mask。

文件其他属性

文件主 chown系列函数

文件存取时间 stat函数

文件大小 stat函数

1.chown/fchown/lchown 函数

Change ownership of a file改变文件所有者

```
#include <sys/types.h>
```

```
#include <unistd.h>
```

```
int chown(const char *path, uid_t owner, gid_t group);
```

```
int fchown(int fd, uid_t owner, gid_t group);
```

```
int lchown(const char *path, uid_t owner, gid_t group);
```

(Return: 0 if success; -1 if failure)

2.获取文件存取时间——stat

```
#include<stdio.h>
#include<sys/stat.h>
#include<time.h>
main()
{
    char *path="/2-4.c";
    struct stat statbuf;
    if (stat(path,&statbuf)==-1)
        perror("Failed to get file status");
    else printf("%s last accessed at %s",path,ctime(&statbuf.st_atime));
}
#=====
./2-4.c last accessed at Wed Jun 15 09:16:48 2022
```

```
#include<stdio.h>
#include<unistd.h>
#include<sys/stat.h>
int main ()
{
    struct stat buf;
    stat("/etc/passwd",&buf);
    printf("\n/etc/passwd\n文件的大小是: %d\n",buf.st_size);
    return 0;
}
#=====
"/etc/passwd"文件的大小是: 3605
```

掌握的知识点

文件属性函数: stat, lstat, fstat
 常见的文件类型
 文件权限的表示方法
 修改文件权限
 设置默认文件权限
 改变文件主chown
 文件存取时间stat ->st_atime
 文件大小stat ->st_size

两种I/O方式

1. 无缓冲和缓冲I/O
 - 无缓冲I/O
 - read/write ->系统函数
 - 文件描述符
 - POSIX.1 and XPG3标准
 - 缓冲 I/O
 - 标准I/O库实现
 - 处理很多细节, 如缓存分配, 以优化长度执行I/O等。
 - 流 -> FILE类型指针
2. 无缓冲 I/O 系统调用
 - 基本 I/O
 - open/creat, close, read, write, lseek
 - dup/dup2
 - fcntl

文件描述符

1. 文件描述符概述

非负整数

```
int fd;
```

(in <unistd.h>)

```
STDIN_FILENO (0), STDOUT_FILENO (1), STDERR_FILENO (2)
```

文件操作一般步骤:

```
open-read/write-[lseek]-close
```

2. 进程打开文件的内核数据结构

无缓冲 I/O 函数

```
open/creat, close, read, write, lseek
```

```
dup/dup2
```

```
fcntl
```

```
ioctl
```

0. 错误处理

UNIX方式

Return value

“errno”变量(defined in /usr/include/errno.h)

```
extern int errno;
```

strerror & perror

```
#include <string.h>
```

```
char *strerror(int errnum);
```

```
#include <stdio.h>
```

```
void perror(const char *msg);
```

1. creat 函数

create a file or device

```
#include <sys/types.h>
```

```
#include <sys/stat.h>
```

```
#include <fcntl.h>
```

```
int creat(const char *pathname, mode_t mode);
```

(Return: a new file descriptor if success; -1 if failure)

例2.6 设计一程序，要求在“/home”目录下创建一个名称为“2-6file”的文件，并且把此文件的权限设置为所有者具有只读权限，最后显示此文件的信息。

```
#include<sys/types.h>
#include<sys/stat.h>
#include<fcntl.h>
int main()
{
    int fd;
    fd=creat("./2-6file",S_IRUSR);
    system("ls ./2-6file -l");
    return 0;
}

#=====
-r----- 1 root root 0 Jun 15 09:40 ./2-6file
```


参数 “mode”

“mode”: 指定创建的新文件的存取权限

取值	含义
S_IRUSR(00400)	Read by owner
S_IWUSR(00200)	Write by owner
S_IXUSR(00100)	Execute by owner
S_IRWXU(00700)	Read, write and execute by owner
S_IRGRP 00040	Read by group
S_IWGRP 00020	Write by group
S_IXGRP 00010	Execute by group
S_IRWXG 00070	Read, write and execute by group
S_IROTH 00004	Read by others
S_IWOTH 00002	Write by others
S_IXOTH 00001	Execute by others
S_IRWXO 00007	Read, write and execute by others

参数 “mode” & umask

- umask: 一种文件保护机制
- 新建文件的初始存取权限
 - mode & ~umask

regular files:

default permissions	rw-rw-rw-	666
umask (-)	---w--w-	022
resulting permissions	rw-r--r--	644

directories:

default permissions	rwxrwxrwx	777
umask (-)	---w--w-	022
resulting permissions	rwxr-xr-x	755

课堂练习2

假设系统的umask为0022，执行creat(“a.txt”,444)后，文件a.txt的所有者、同组用户、其他用户对文件的权限是什么？

2.Open 函数

```
Open and possibly create a file or device
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

int open(const char *pathname, int flags);
int open(const char *pathname, int flags, mode_t mode);
(Return: a new file descriptor if success; -1 if failure)
```

例2.7设计一个程序，要求在当前目录下以可读写方式打开一个名为“2-7file”的文件。如果该文件不存在，则创建此文件；如果存在，将文件清空后关闭。

```
#include<stdio.h>
#include<stdlib.h>
#include<fcntl.h>
int main ()
{   int fd;
    if((fd=open("./2-7file",O_CREAT|O_TRUNC|O_WRONLY,0600))<0)
    {
        perror("打开文件出错");
        exit(1);
    }
    else
    {
        printf("打开(创建)文件\"2-7file\", 文件描述符为: %d\n",fd);
    }
    if(close(fd)<0)
    {
        perror("关闭文件出错");
        exit(1);
    }
    system("ls ./2-7file -l");
    return 0;
}
```

```
#=====
打开(创建)文件"2-7file", 文件描述符为: 3
-rw----- 1 root root 0 Jun 15 09:55 ./2-7file
```

参数“flags”

“flags”：指定文件存取方式
One of O_RDONLY, O_WRONLY or O_RDWR which request opening the file read-only, write-only or read/write, respectively, bitwise-or'd with zero or more of the following: (All defined in /usr/include/fcntl.h)
O_APPEND: 追加方式打开
O_TRUNC: 文件存在清空
O_CREAT: 文件不存在创建。
O_EXCL: 和 O_CREAT一起用时，文件存在则出错，打开失败
...
“creat” function: 等价于open函数中指定 O_CREAT|O_WRONLY|O_TRUNC

3.close 函数

```
Close a file descriptor
#include <unistd.h>
int close(int fd);
(Return: 0 if success; -1 if failure)
```

4.read/write 函数

```
Read from a file descriptor
#include <unistd.h>
ssize_t read(int fd, void *buf, size_t count);
(返回值: 读到的字节数, 若已到文件尾为0, 若出错为-1)
Write to a file descriptor
#include <unistd.h>
ssize_t write(int fd, const void *buf, size_t count);
(返回值: 若成功为已写的字节数, 若出错为-1)
```

例2.8设计一个程序, 完成文件的复制工作。要求通过read函数和write函数复制“/etc/passwd”文件到目标文件中, 目标文件名在程序运行时从键盘输入。

```
#include<unistd.h>
#include<sys/types.h>
#include<sys/stat.h>
#include<fcntl.h>
#include<stdio.h>
#include<stdlib.h>
int main()
{
    int fdsrc,fddest,nbytes;
    int z;
    char buf[20], des[20];
    int flags=O_CREAT | O_TRUNC | O_WRONLY;
    printf("请输入目标文件名:");
    scanf("%s",des);
    fdsrc=open("/etc/passwd",O_RDONLY);
    if(fdsrc<0) { exit(1); }
    fddest=open(des,flags,0644);
    if(fddest<0) { exit(1); }
    while((nbytes=read(fdsrc,buf,20))>0)
    {
        z=write(fddest,buf,nbytes);
        if(z<0) { perror("写目标文件出错"); } }
    close(fddest);
    close(fdsrc);
    printf("复制\"/etc/passwd\"文件为\"%s\"文件成功! \n",des);
    exit(0);
}

#=====
请输入目标文件名:2-7.c
复制"/etc/passwd"文件为"2-7.c"文件成功!
```

5.lseek 函数

read/write 定位文件指针

```
#include <sys/types.h>
```

```
#include <unistd.h>
```

```
off_t lseek(int fildes, off_t offset, int whence);
```

(Return: the resulting offset location if success; -1 if failure)

The directive “whence”:

SEEK_SET: the offset is set to “offset” bytes 指针位移量为设定值

SEEK_CUR: the offset is set to its current location plus “offset” bytes 指针位移量为当前位移加设定值

SEEK_END: the offset is set to the size of the file plus “offset” bytes 指针位移量为文件尾加设定值

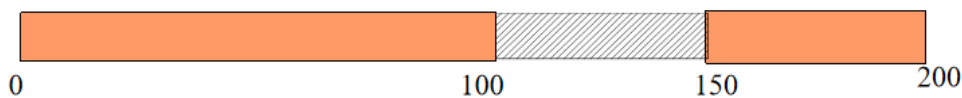
空洞文件

使用lseek修改文件偏移量后，当前文件偏移量有可能大于文件的长度

在这种情况下，对该文件的下一次写操作，将加长该文件

这样文件中形成了一个空洞。对空洞区域进行读，均返回0

`lseek(fd, 50, SEEK_END)` 文件长度是多少？
`write(fd, buf, 50)` 200? 150?



6.dup/dup2 函数

复制文件描述符

```
#include <unistd.h>
```

```
int dup(int oldfd);
```

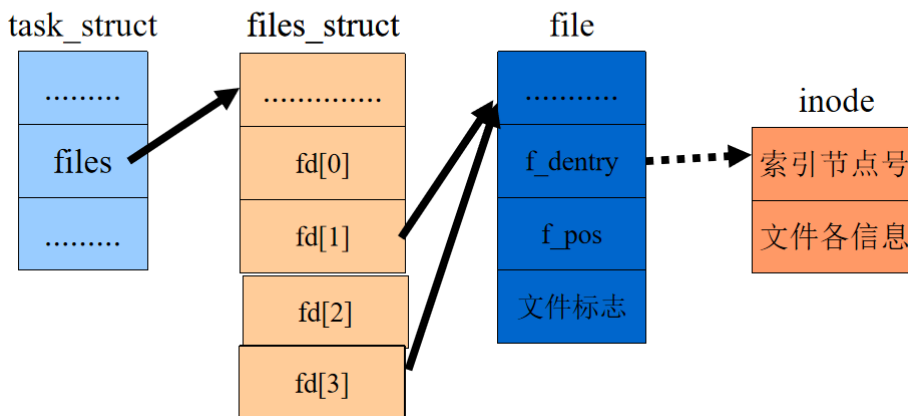
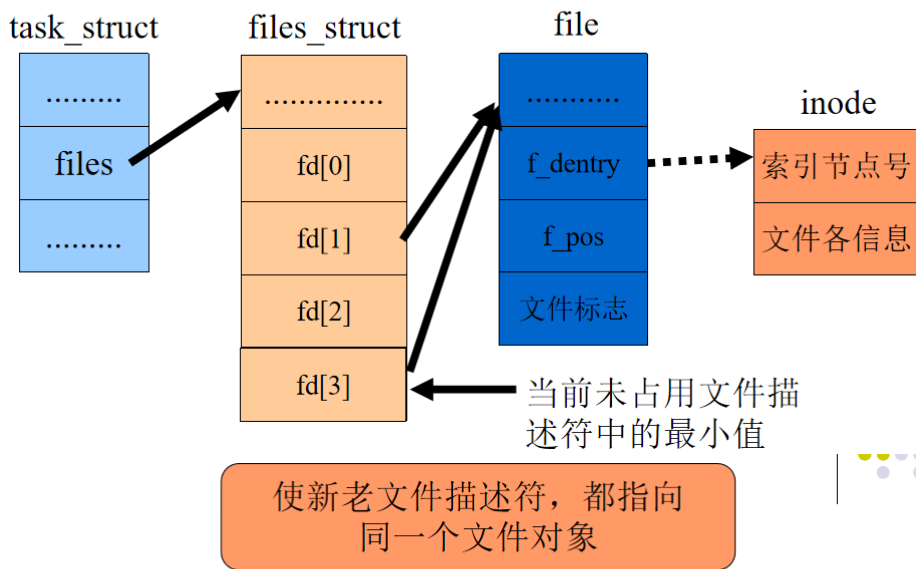
```
int dup2(int oldfd, int newfd);
```

(Return: the new file descriptor if success; -1 if failure)

6. dup/dup2 函数



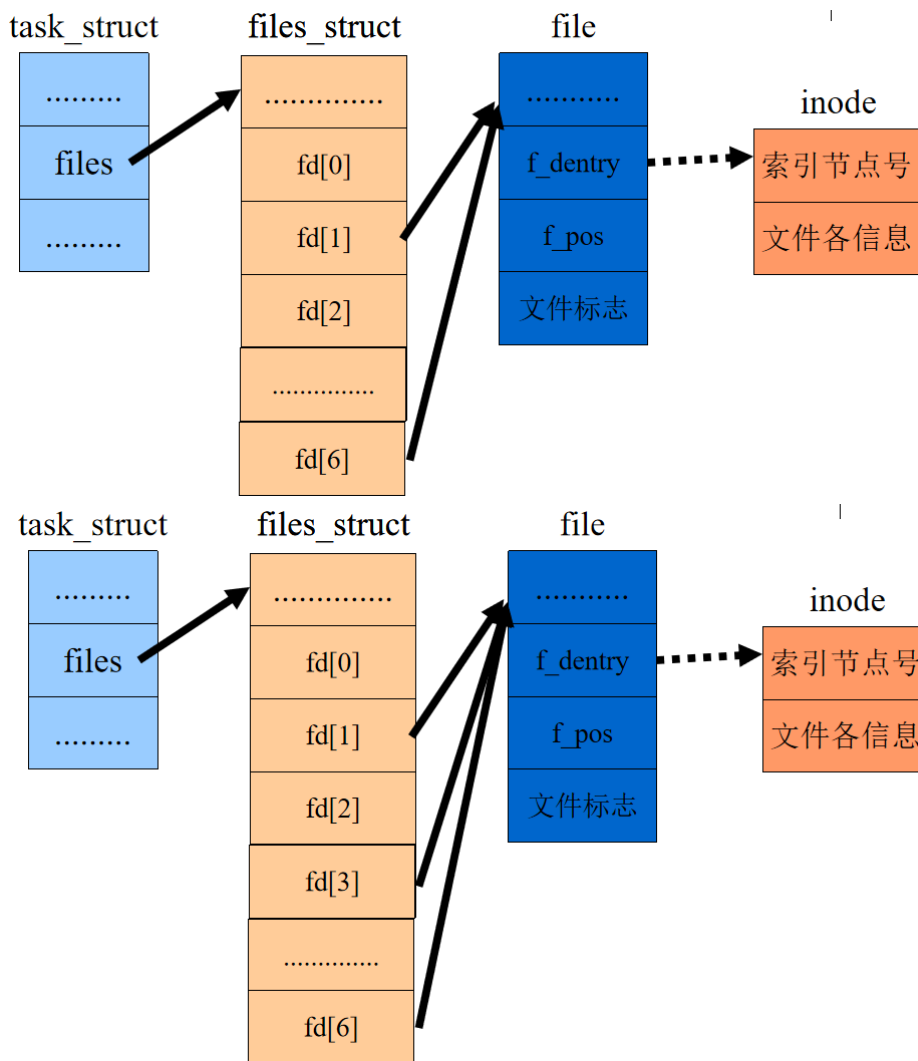
调用dup函数之前 $\longrightarrow$ `int fd = dup(1);`



假设进程已打开文件描述符0、1、2

调用`dup2(1, 6)`，`dup2`返回值是多少？

然后再调用`dup(6)`，`dup`返回值是多少？



7.fcntl函数

查看、修改打开的文件描述符

```
#include<fcntl.h>
#include<unistd.h>
#include<sys/types.h>
int result=fcntl(int fd,int cmd);
int result=fcntl(int fd,int cmd,long arg,...);
```

cmd取值:

- F_DUPFD 复制文件描述符
- F_GETFD 获得文件描述符
- F_SETFD 设置文件描述符
- F_GETFL 获取文件描述符当前模式
- F_SETFL 设置文件描述符当前模式
- F_GETLK 获得记录锁
- F_SETLK 设置记录锁

掌握的知识点

两种I/O方式区别
文件描述符概念
主要I/O函数: `open`, `read/write`, `lseek`, `dup/dup2`用法

缓冲文件I/O

标准I/O库

文件流

标准I/O函数

标准I/O库

为什么要设计标准I/O库？

直接使用API进行文件访问时，需要考虑许多细节问题

例如：read、write时，缓冲区的大小该如何确定，才能使效率最优

标准I/O库封装了诸多细节问题，包括缓冲区分配

```
#define BUFSIZE 4096
#include<stdio.h>
#include<unistd.h>
int main()
{
    int n;
    char buf[BUFSIZE];
    while((n = read(STDIN_FILENO, buf, BUFSIZE))>0)
        if(write(STDOUT_FILENO, buf,n)!= n)
            perror("write error");
    return 0;
}
好
```

程序中，影响效率的关键：
BUFSIZE的取值

4096最

原因

Linux文件系统采用了某种预读技术

当检测到正在进行顺序读取时，系统就试图读入比应用程序所要求的更多数据

并假设应用程序很快就会读这些数据

当BUFSIZE增加到一定程度后，预读就停止了

标准 I/O 缓冲

标准I/O库提供缓冲的目的：尽可能减少使用read、write调用的次数，以提高I/O效率。

通过标准I/O库进行的读写操作，数据都会被放置在标准I/O库缓冲中中转。

文件流

Buffered I/O带缓冲I/O

Stream and “FILE” structure

FILE* fp;

预定义的文件指针：stdin, stdout, stderr

```
typedef struct {
.....
char      fd;          /* File descriptor */
short     bsize;       /* Buffer size */
unsigned char *buffer; /* Data transfer buffer */
.....
} FILE;
```

标准I/O库
管理的缓冲区

标准 I/O 函数

流 open/close
流 read/write
 每次一个字符的I/O: fgetc, fputc
 每次一行的I/O: fgets, fputs, gets, puts
 直接I/O(二进制I/O): fread, fwrite
 格式化I/O: scanf, printf, fscanf, fprintf
流定位: fseek, ftell, frewind
流刷新: fflush

流打开/关闭

例2.9设计一个程序，要求用流文件I/O操作打开文件“2-9file”，如果该文件不存在，则创建文件。

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    FILE * fp;
    if((fp=fopen("./2-9file", "a+"))==NULL)
    {
        printf("打开(创建)文件出错");      exit(0);
    }
    fclose(fp);
}
```

1.流打开/关闭

Open a stream
 #include <stdio.h>
 FILE *fopen(const char *filename, const char *mode);
Parameter “mode”
 “r”: Open text file for reading.
 “w”: Truncate file to zero length or create text file for writing.
 “a”: Open for appending.
 “r+”: Open for reading and writing.
 “w+”: Open for reading and writing. The file is created if it does not exist, otherwise it is truncated.
 “a+”: Open for reading and appending. The file is created if does not exist.
Close a stream
 #include <stdio.h>
 int fclose(FILE *fp);
 (Return: 0 if success; -1 if failure)

对流有三种读写方式
每次读写一个字符
每次读写一行
每次读写任意长度的内容

(1) 输入/出一个字符

```
getc, fgetc, getchar functions
#include <stdio.h>
int getc(FILE *fp);
int fgetc(FILE *fp);
int getchar(void);
putc, fputc, putchar functions
#include <stdio.h>
int putc(int c, FILE *fp);
int fputc(int c, FILE *fp);
int putchar(int c);
(Return: the character if success; -1 if failure)
```

(2) 输入/出一行

```
fgets, gets functions
#include <stdio.h>
char *fgets(char *s, int size, FILE *stream);
char *gets(char *s);
fputs, puts functions
#include <stdio.h>
int fputs(const char *s, FILE *stream);
int puts(const char *s);
```

(3) 二进制流输入/输出

```
fread/fwrite functions
#include <stdio.h>
size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);
size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);
(Return: the number of a items successfully read or written.)
```

例2.10设计两个程序，要求一个程序把三个人的姓名和账号余额信息通过一次流文件I/O操作写入文件“2-10file”，另一个格式输出账号信息，把每个人的账号和余额——对应显示输出。

fread.c

```
/*程序：把帐号信息从文件读出*/
#include<stdio.h>
#define nmemb 3
struct test
{   char name[20];
    int pay;
}s[nmemb];
int main( )
{   FILE * fp;
    int i;
    fp = fopen("2-10file", "r");
    fread(s,sizeof(struct test),nmemb,fp);
```

```

fclose(fp);
for(i=0;i<nmemb;i++)
    printf("帐号[%d]:%-20s余额[%d]:%d\n",i,s[i].name,i,s[i].pay);
return 0;
}

```

fwrite.c

```

/*程序：把帐号信息写入文件*/
#include<stdio.h>
#include<string.h>
#define set_s(x,y,z) {strcpy(s[x].name,y);s[x].pay=z;} /*自定义宏，用于赋值*/
#define nmemb 3
struct test
{   char name[20];
    int pay;
}s[nmemb];
int main()
{   FILE * fp;
    set_s(0,"张三",12345);
    set_s(1,"李四",200);
    set_s(2,"王五",50000);
    fp=fopen("2-10file","a+"); /*打开(创建)文件*/
    fwrite(s,sizeof(struct test),nmemb,fp);
    fclose(fp);
    return 0;
}

```

```

[root@VM-0-4-centos test]# ./fwrite
[root@VM-0-4-centos test]# ./fread
帐号[0]:张三          余额[0]:12345
帐号[1]:李四          余额[1]:200
帐号[2]:王五          余额[2]:50000

```

(4) 格式化 I/O

```

scanf, fscanf, sscanf functions
#include <stdio.h>
int scanf(const char *format, ...);
int fscanf(FILE *stream, const char *format, ...);
int sscanf(const char *str, const char *format, ...);
printf, fprintf, sprintf functions
#include <stdio.h>
int printf(const char *format, ...);
int fprintf(FILE *stream, const char *format, ...);
int sprintf(char *str, const char *format, ...);

```

流定位

```

fseek, ftell, rewind functions
#include <stdio.h>
int fseek(FILE *stream, long int offset, int whence);
long ftell(FILE *stream);
void rewind(FILE *stream);

```

例2.11 设计一程序，要求用fopen函数打开系统文件“/etc/passwd”，先把位置指针移动到第10个字符前，再把位置指针移动到文件尾，最后把位置指针移动到文件头，输出三次定位的文件偏移量的值。

```

/*程序：文件的定位*/
#include<stdio.h>
int main()
{
    FILE *stream;
    long offset;
    fpos_t pos;
    stream=fopen("/etc/passwd","r");
    fseek(stream,10,SEEK_SET);
    printf("文件流的偏移量: %d\n",ftell(stream));
    fseek(stream,0,SEEK_END);
    printf("文件流的偏移量: %d\n",ftell(stream));
    rewind(stream);
    printf("文件流的偏移量: %d\n",ftell(stream));
    fclose(stream);
    return 0;
}

#=====
文件流的偏移量: 10
文件流的偏移量: 3605
文件流的偏移量: 0

```

流刷新

刷新文件流。把流里的数据立刻写入文件—fork前使用fflush

```
#include <stdio.h>
int fflush(FILE *stream);
```

自动刷新：
 流关闭fclose;
 exit终止;
 行缓冲“\n”;
 缓冲区满;
 执行输入操作读文件:printf (“hello”);scanf (“%d",&a)

流缓冲

三种类型的缓冲

块（全）缓冲block buffered (fully buffered): 一般C库函数写入文件是全缓冲的

行缓冲line buffered: 引用标准交互设备的流stdin,stdout

例:

```
for(i=1;i<=10;i++) fputc(c,stdout);
fputc("\n",stdout);
```

无缓冲Unbuffered: 标准错误流stderr

全缓冲

在填满标准I/O缓冲区后，才进行实际I/O操作（例如调用write函数）
 调用fflush函数也能强制进行实际I/O操作

行缓冲

在输入和输出遇到换行符时，标准I/O库执行I/O操作
 因为标准I/O库用来收集每一行的缓存的长度是固定的，所以，只要填满了缓存，即使没有遇到新行符，也进行I/O操作

终端（例如标准输入和标准输出），使用行缓冲

不带缓冲

标准I/O库不对字符进行缓冲存储
 标准出错是不带缓冲的，为了让出错信息尽快显示出来

流和文件描述符

确定流使用的底层文件描述符

```
#include <stdio.h>
int fileno(FILE *fp);
```

根据已打开的文件描述符创建一个流

```
#include <stdio.h>
FILE *fdopen(int fildes, const char *mode);
```

掌握的知识点

标准I/O库的作用

fopen函数的使用

三种缓冲

流和文件描述符转换

特殊文件

目录文件

链接文件

设备文件

目录文件

```
mkdir/rmdir
chdir/fchdir, getcwd
读目录操作
opendir/closedir
readdir
telldir
seekdir
```

例2.12设计一程序，要求读取当前目录文件中所有的目录结构。

```
#include<sys/types.h>
#include<dirent.h>
#include<unistd.h>
#include<stdio.h>
int main()
{
    DIR * dir;
    struct dirent * ptr;
    int i;
    dir=opendir("./");
    while((ptr = readdir(dir))!=NULL)
    {
        printf("目录: %s\n",ptr->d_name);
    }
    closedir(dir);
}

#=====
目录: f
目录: fread
目录: .
目录: 2-10file
目录: ..
目录: f.c
```

目录: `fwrite.c`
目录: `fwrite`
目录: `fread.c`

读目录

数据结构
DIR, struct dirent
操作函数
opendir/closedir
readdir
telldir
seekdir

数据结构

```
DIR
    目录流对象的数据结构
    in <dirent.h>
    typedef struct __dirstream DIR;
struct dirent
    目录项
    Defined in <dirent.h>
    ino_t d_ino;          /* inode number */
    char d_name[NAME_MAX + 1]; /* file name */
```

操作函数

```
    目录的打开、关闭、读、定位
#include <sys/types.h>
#include <dirent.h>

DIR *opendir(const char *name);
int closedir(DIR *dir);
struct dirent *readdir(DIR *dir);
off_t telldir(DIR *dir);
void seekdir(DIR *dir, off_t offset);
```

目录扫描程序——ls -R命令

```
DIR *dp;
struct dirent *entry;

if ( (dp = opendir(dir)) == NULL )
    err_sys(...);
while ( (entry = readdir(dp)) != NULL ) {
    lstat(entry->d_name, &statbuf);
    if ( S_ISDIR(statbuf.st_mode) )
        ...
    else
        ...
}
closedir(dp);
```

mkdir/rmdir 函数

创建一个空目录

```
#include <sys/stat.h>
#include <sys/types.h>
int mkdir(const char *pathname, mode_t mode);
(Return: 0 if success; -1 if failure)
```

删除一个空目录

```
#include <unistd.h>
int rmdir(const char *pathname);
(Return: 0 if success; -1 if failure)
```

chdir/fchdir 函数

Change working directory

```
#include <unistd.h>
int chdir(const char *path);
int fchdir(int fd);
(Return: 0 if success; -1 if failure)
```

当前工作目录是进程的属性，所以该函数只影响调用chdir的进程本身
cd(1) command

链接文件

ln 命令

link/unlink 函数

给一个文件创建一个链接。

```
#include <unistd.h>
int link(const char *oldpath, const char *newpath);
(Return: 0 if success; -1 if failure)
```

删除文件链接

```
#include <unistd.h>
int unlink(const char *pathname);
(Return: 0 if success; -1 if failure)
```

symlink/readlink

ln -s命令

创建一个符号链接

```
#include <unistd.h>
int symlink(const char *oldpath, const char *newpath);
(Return: 0 if success; -1 if failure)
```

读取符号链接的值

```
#include <unistd.h>
int readlink(const char *path, char *buf, size_t bufsiz);
(Return: the count of characters placed in the buffer if success; -1 if failure)
```

例2.13设计一程序，要求为“/etc/passwd”文件建立软链接“2-13link”，并查看此链接文件和“/etc/passwd”文件。

```
#include<unistd.h>
int main()
{
    symlink("/etc/passwd", "/home/mylinux/2-13link");
    system("ls 2-13link -l");
    system("ls /etc/passwd -l");
}

#=====
lrwxrwxrwx 1 root root 11 Jun 15 11:52 2-13link -> /etc/passwd
-rw-r--r-- 1 root root 3605 Jun 11 21:42 /etc/passwd
```

例2.14设计一程序，要求为“/etc/passwd”文件建立硬链接“2-14link”，并查看此链接文件和“/etc/passwd”文件。

```
#include<unistd.h>
int main()
{
    link("./myfile", "2-14link");
    system("ls 2-14link -l");
    system("ls /etc/passwd -l");
}

#=====
-rw-r--r-- 1 root root 0 Jun 15 11:57 2-14link
-rw-r--r-- 1 root root 3605 Jun 11 21:42 /etc/passwd
```

设备文件

1. 设备文件名
 - ls -C 列出当前系统加载的设备对应的文件
 - ls -li 列出当前终端设备的属性
2. 设备文件读写
 - open, read, write, close, stat

例2-15 向终端pts1写入100个“unix”。

```
#include<fcntl.h>
int main()
{
    int i, fd;
    fd=open("/dev/pts/1", O_WRONLY);
    for(i=0; i<100; i++)
        write(fd, "Unix", 4);
    close(fd);
}
```

打开两个terminal，一个运行程序，一个输入tty，观察运行结果

本章总结

文件属性
 文件属性操作
 Basic file I/O
 基本I/O系统调用(open, close, lseek, read, write, dup/dup2...)
 文件和目录
 系统调用stat/fstat/lstat; opendir, readdir, ...
 Standard I/O library: fopen, fflush

Ch5 进程编程

进程和进程环境

进程

Process: an address space with one or more threads executing within that address space, and the required system resources for those threads.

进程启动

System call “fork”
Process resources
struct task_struct
System space stack
...
System call “exec”
The entry of C programs

进程终止

进程终止的五种方式
Normal termination
Return from “main” function
Call “exit” function
Call “_exit” function
Abnormal termination
Call “abort” function
Terminated by a signal

进程分类

Foreground process 前台进程

要求用户启动它们或与它们交互的进程称为前台进程。
前台进程不结束，终端就不会出现系统提示符，直到进程终止。
缺省情况下，程序和命令作为前台进程运行。

Background process 后台进程

独立于用户运行的进程称为后台进程。
用户在输入命令行后加上“&”字符然后按<Enter>键就启动了后台进程。
Shell不等待命令终止，就立即出现系统提示符，让该命令进程在后台运行，用户可以继续执行新的命令。

Daemon 守护进程

总是运行在后台的系统进程。
守护程序通常在系统启动时启动，并且它们一直运行到系统停止。
守护进程常常用于向用户提供各种类型的服务和执行系统管理任务。
守护程序进程由 root 用户或 root shell 启动，并只能由 root 用户停止。

进程控制

进程标识符: getpid, getppid
进程创建: fork, vfork, system, exec
进程终止: exit, _exit
父子进程关系: 简单同步wait, waitpid

```
#include <sys/types.h>
```

```
#include <unistd.h>
```

```
pid_t getpid(void);
```

返回: 调用进程的进程 ID

```
pid_t getppid(void);
```

返回: 调用进程的父进程 ID

```
uid_t getuid(void);
```

返回: 调用进程的实际用户 ID

```
uid_t geteuid(void);
```

返回: 调用进程的有效用户 ID

```
gid_t getgid(void);
```

返回: 调用进程的实际组 ID

```
gid_t getegid(void);
```

返回: 调用进程的有效组 ID

进程创建

1.fork

fork: create a child process

```
#include <sys/types.h>
```

```
#include <unistd.h>
```

```
pid_t fork(void);
```

returned value:

pid of child (in the current (parent) process),

0 (in child process),

-1 (failure)

1.fork创建子进程代码结构

```
.....
```

```
pid = fork();
```

```
if (pid<0) {perror("fork()");exit(1);}
```

```
else if (pid==0) { child process }
```

```
else { parent process }
```

文件共享

所有由父进程打开的描述符都被复制到子进程中。父、子进程每个相同的打开描述符共享一个文件表项

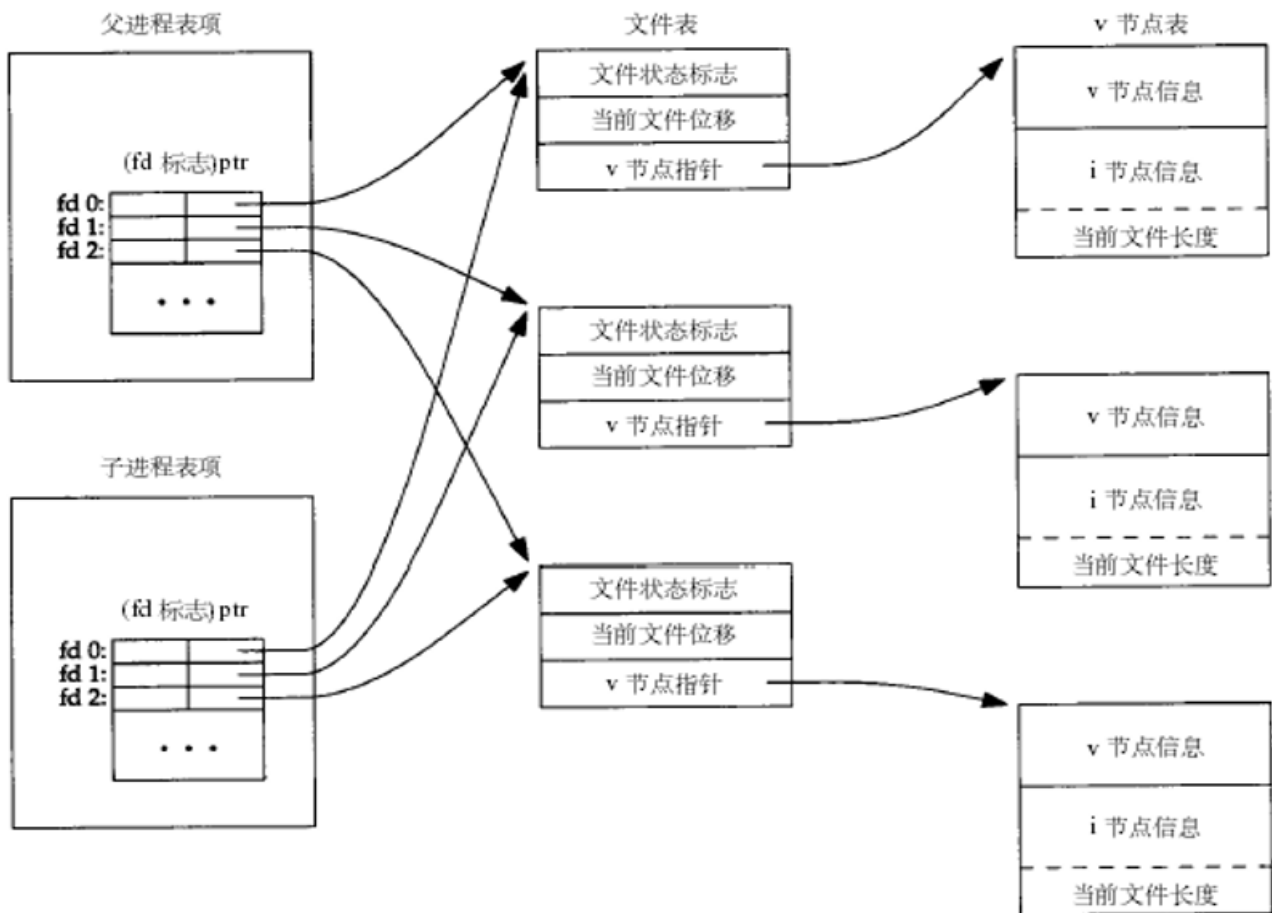


图8-1 fork之后父、子进程之间对打开文件的共享

Example3-1

```
#include<sys/types.h>
#include<unistd.h>
#include<fcntl.h>
#include<stdio.h>
#include<stdlib.h>
int main(int argc,char *argv[])
{
    pid_t pid;
    int fd;
    int i=1;
    int status;
    char *ch1="hello";
    char *ch2="world";
    char *ch3="IN";
    if((fd=open("test.txt",O_RDWR|O_CREAT,0644))==-1)
    {
        perror("parent open");
        exit(EXIT_FAILURE);
    }
    if(write(fd,ch1,strlen(ch1))==-1)
    {
        perror("parent write");
        exit(EXIT_FAILURE);
    }
    if((pid=fork())==-1)
    {
        perror("fork");
        exit(EXIT_FAILURE);
    }
}
```

```

else
    if(pid==0)
    {
        i=2;
        printf("in child\n");    //打印 i 值，以示与父亲进程区别
        printf("i=%d\n",i);
        if(write(fd,ch2,strlen(ch2))==-1)    //写文件 test.txt，与父进程共享
            perror("child write");
        return 0;
    }
    else
    {
        sleep(1);    //等待子进程先执行
        printf("in parent\n");
        printf("i=%d\n",i);    //打印 i 值，以示与子进程区别
        if(write(fd,ch3,strlen(ch3))==-1)    //写操作，结果添加到文件后
            perror("parent,write");
        wait(&status);    //等待子进程结束
        return 0;
    }
}

//-----
程序输出
in child
i=2
in parent
i=1
test.txt文件中的内容
helloworldIN

```

Example3-2——单个进程

```

#define LEFT 30000000
#define RIGHT 30000200
main()
{ int i,j,mark;
  for (i=LEFT;i<=RIGHT;i++)
  {
      mark=1;
      for (j=2;j<i/2;j++)
      {
          if (i%j==0)
          {
              mark=0;
              break;
          }
      }
      if (mark)
          printf("%d is a primer!\n",i);
  }
}

//-----
30000001 is a primer!
30000023 is a primer!
30000037 is a primer!
.....

```

Example3-2——多个进程

```

main()
{ int i,j,mark;
  pid_t pid;
  for (i=LEFT;i<=RIGHT;i++)
  { mark=1;
    pid = fork();
    if (pid==0)
    {
      for (j=2;j<i/2;j++)
      {
        if (i%j==0)
        { mark=0;
          break;
        }
      }
      if (mark)
        printf("%d is a primer!\n",i);
      exit(0);
    }
  }
  exit(0);
}
//-----
30000001 is a primer!
30000023 is a primer!
30000037 is a primer!
....

```

fork应用场合

进程复制自己，使父子进程同一时刻执行不同的代码——网络服务

进程要执行另一个不同的程序：fork-exec—shell

Question: 效率问题？父子进程各自占一段逻辑地址空间，fork之后立即exec，地址空间浪费。

“写-复制”COW

vfork

vfork

vfork

```
#include <sys/types.h>
```

```
#include <unistd.h>
```

```
pid_t vfork(void);
```

功能：类似fork，创建一个新进程，效率高。

与fork区别：

(1) vfork创建的进程与父进程共用地址空间

(2) vfork创建子进程后，阻塞父进程，直到子进程调用exec或exit，内核才唤醒父进程。

Example 3-3-1

```
#include<sys/types.h>
```

```
#include<unistd.h>
```

```
#include<fcntl.h>
```

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
int global=5;
```

```
main()
```

```
{
```

```
    int pid;
```

```

char *string="I am father: ";
int local=10;
printf("before vfork\n");
if((pid=fork())<0)
{
    perror("vfork failed");
    exit(0);
}
if(pid==0)
{
    string="I am child.";
    global++;
    printf("%s global=%d, local=%d\n",string,global,local);
    exit(0);
}
else
{
    local++;
    printf("%s my pid is %d \n"      "global=%d\n local=%d\n", string,getpid(),global,local);
    exit(0);
}
}
//-----
before vfork
I am father: my pid is 15650
global=5
local=11
I am child. global=6, local=10

```

Example 3-3-2

```

int global=5;
main()
{
    int pid;
    char *string="I am father: ";
    int local=10;
    printf("before vfork\n");
    if((pid=vfork())<0)
    {
        perror("vfork failed"); //共用进程空间，子进程先执行
        exit(0);
    }
    if(pid==0)
    {
        string="I am child.";
        global++;
        printf("%s global=%d, local=%d\n",string,global,local);
        exit(0);
    }
    else
    {
        local++;
        printf("%s my pid is %d \n"      "global=%d\n local=%d\n", string,getpid(),global,local);
        exit(0);
    }
}
//-----
before vfork
I am child. global=6, local=10

```

```
I am child. my pid is 15813
global=6
local=11
```

相关函数

sleep函数

函数原型:

```
#include <unistd.h>
```

```
unsigned int sleep(unsigned int seconds);
```

seconds: 暂停时间 (秒)

exec 系列函数

(1) Replace the current process image with a new process image.

(2) 执行新程序的进程保持原进程的一系列特征:

pid, ppid, uid, gid, working directory, root directory ...

函数原型

```
#include <unistd.h>
extern char **environ;
int execl(const char *path, const char *arg, ...);
int execlp(const char *file, const char *arg, ...);
int execl_e(const char *path, const char *arg, ..., char * const envp[]);
int execv(const char *path, char * const argv[]);
int execvp(const char *file, char * const argv[]);
int execve(const char *filename, char * const argv[], char * const envp[]);
```

exec 系列函数

1. 两大类:

execl开头: 参数以列表形式arg0,arg1...NULL结束。

execv开头: 参数以指向字符串数组argv[]指针形式, arg[0]必须为程序名。

2. 含有字母p的函数可以使用相对路径, 根据环境变量PATH查找文件; 其他函数必须使用绝对路径;

3. 含有字母e的函数, 需要通过指向envp[]数组的指针指明新的环境变量, 其他函数使用当前环境变量。

exec举例

例如: 用exec函数使新进程执行“/bin/ps” 程序。

```
#include<unistd.h>
```

```
const char*ps_argv[]={ "ps", "-af", NULL};
```

```
const char *ps_envp[] = {"PATH=/bin:/usr/bin", "TERM=console", NULL};
```

六种情况:

```
execl("/bin/ps", "ps", "-af", NULL);
```

```
execlp("ps", "ps", "-af", NULL);
```

```
execl_e("/bin/ps", "ps", "-af", NULL, ps_envp);
```

```
execv("/bin/ps", ps_argv);
```

```
execvp("ps", ps_argv);
```

```
execve("/bin/ps", ps_argv, ps_envp);
```

fork和exec一起使用

Two ways of using fork

(1)The parent process duplicates itself, and then two different pieces of codes are executed in parent process and child process.父子进程各自执行不同的代码

(2)A process want to execute another program. 进程要执行另一个程序

Example: a simple shell

```
printf("%s "); /* print prompt */
while (fgets(buf, MAXLINE, stdin) != NULL) {
    buf[strlen(buf) - 1] = 0; /* replace newline with null */
    if ( (pid = fork()) < 0 )
        err_sys("fork error");
    else if ( pid == 0 ) { /* child */
        execlp(buf, buf, (char *) 0);
        fprintf(stderr, "couldn't execute: %s", buf);
        exit(127);
    }
    if ( (pid = waitpid(pid, &status, 0)) < 0 ) /* parent */
        err_sys("waitpid error");
    printf("%s ");
}
```

进程终止

1.exit函数

函数原型:

```
#include <stdlib.h>
```

```
void exit(int status);
```

status:进程状态

功能: 正常终止目前进程的执行, 把参数status返回给父进程, 进程所有的缓冲区数据会自动写回并关闭未关闭的文件。

2._exit函数

函数原型:

```
#include <stdlib.h>
```

```
void _exit(int status);
```

status:进程状态

功能: 立刻终止目前进程的执行, 把参数status返回给父进程, 并关闭未关闭的文件。不处理标准I/O缓冲区。

父子进程关系

父进程在子进程前终止—孤儿进程

Orphan process—init

子进程在父进程前终止—可能成为僵尸进程

SIGCHLD signal 忽略SIGCHLD信号

Handled by wait/waitpid in parent 父进程中用wait/waitpid处理

Not handled by wait/waitpid in parent -> zombie父进程没有用wait/waitpid处理->僵尸进程

孤儿进程

孤儿进程：父进程终止，子进程未终止，子进程成为孤儿进程。

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
int main(int argc, const char *argv[])
{
    pid_t pid;
    pid = fork();
    if(pid < 0){
        perror("fork error");
        return -1;
    }
    else if(pid == 0){
        printf("The child process, id = %d parent id = %d\n",getpid(), getppid());
        sleep(1);
        printf("The child process, id = %d parent id = %d\n",getpid(), getppid());
        while(1);
    }
    else{
        sleep(1);
        printf("The parent process, id = %d\n", getpid());
    }
}
//-----
The child process, id = 16452 parent id = 16451
The child process, id = 16452 parent id = 16451
The parent process, id = 16451
//顺序不唯一
```

僵尸进程 (Zombie)

僵尸进程：已终止运行，但尚未被清除的进程。

子进程运行结束后（正常或异常），它并没有马上从系统的进程分配表中被删掉，而是进入僵死状态（Zombie），一直等到父进程来回收它的结束状态信息。

Example 3-4process_zombie.c

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
int main(int argc, const char *argv[])
{
    pid_t pid;
    pid = fork();
    if(pid < 0){
        perror("fork error");
        return -1;
    }
    else if(pid == 0){
        printf("The child process, id = %d parent id = %d\n", getpid(), getppid());
    }
    else{
        printf("The parent process, id = %d\n", getpid());
        while(1);
    }
}
```



```

    }
    return 0;
}
//-----
The parent process, id = 16809
The child process, id = 16810 parent id = 16809

```

僵尸进程解决方法

僵尸进程的proc结构一直存在直到父进程正常结束或系统重启。—如何消除僵尸进程？

方法一：wait/waitpid阻塞父进程，子进程先终止

方法二：父进程不阻塞，两次fork

方法三：使用signal信号处理

方法一：wait example

如果有了wait则孩子优先，若无，则parent优先。尊老爱幼。

例3.5设计一个程序，要求复制进程，子进程显示自己的进程号后暂停一段时间，父进程等待子进程正常结束，打印显示等待的进程号和等待的进程退出状态。

Example 3-5-1wait.c

```

main()
{
    int i,j,mark;
    pid_t pid;
    for (i=LEFT;i<=RIGHT;i++)
    {
        mark=1;
        pid = fork();
        if (pid==0)
        {
            for (j=2;j<i/2;j++)
                if (i%j==0){
                    mark=0;
                    break;
                }
            if (mark)
                printf("%d is a primer!\n",i);
            sleep(100);
            exit(0);
        }
    }
    for (i=LEFT;i<=RIGHT;i++)
        wait(NULL);
    exit(0);
}

```

wait & waitpid functions

```

Wait for process termination
Prototype
#include <sys/types.h>
#include <sys/wait.h>
pid_t wait(int *status);
pid_t waitpid(pid_t pid, int *status, int options);
waitpid的第一个参数pid的意义:
    pid > 0: 等待进程id为pid的子进程。
    pid == 0: 等待与自己同组的任意子进程。
    pid == -1: 等待任意一个子进程
    pid < -1: 等待进程组号为-pid的任意子进程。
    因此, wait(&stat)等价于waitpid(-1, &stat, 0)
    waitpid例子

```

Example 3-5-2waitpid.c

```

#include<stdio.h>
#include<unistd.h>
#include<sys/types.h>
#include<sys/wait.h>
#include<stdlib.h>
int main ()
{
    pid_t pid,wpid;
    int status,i;
    pid=fork();
    if(pid==0)
    {
        printf("这是子进程,进程号(pid)是:%d\n",getpid());
        sleep(5);
        exit(6);
    }
    else
    {
        printf("这是父进程,正在等待子进程.....\n");
        wpid=waitpid(pid,&status,0);
        i=WEXITSTATUS(status);
        printf("等待的进程的进程号(pid)是:%d ,结束状态:%d\n",wpid,i);
    }
}
//-----
这是父进程,正在等待子进程.....
这是子进程,进程号(pid)是:9126
等待的进程的进程号(pid)是:9126 ,结束状态:6

```

方法二：两次fork Example3-6.c

```

#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include<stdio.h>
#define LEFT 30000000
#define RIGHT 30000200
int main(void)
{pid_t pid;
    if((pid=fork())<0)
        perror("fork error");
    else if(pid==0)    /*first child*/

```

```

{
    if((pid=fork())<0)
        perror("fork error");
    else if(pid>0) /*parent of second child=first child*/
        exit(0);
    sleep(10); /*second child*/
    printf("second child,parent pid=%d\n",getppid());
    exit(0);
}
if (waitpid(pid,NULL,0)!=pid) /*wait for first child,parent(the original process)*/
    perror("waitpid error");
printf("parent exit");
exit(0);
}
//-----
parent exit
second child,parent pid=1

```

方法三：signal函数处理

Signal函数处理：

- (1) 设置信号处理函数
 signal(SIGCHLD, fun)
- (2) 忽略子进程终止信号
 signal(SIGCHLD, SIG_IGN)
 内核回收

```

#include<stdio.h>
#include<stdlib.h>
#include<sys/types.h>
#include<sys/wait.h>
main()
{
    pid_t pid;
    signal(SIGCHLD, SIG_IGN);
    if((pid=fork())==0)
    {
        sleep(2);
        printf("child process %d exit\n",getpid());
        exit(0);
    }
    else
    {
        printf("hello child\n");
        while(1)
        {
            sleep(1);
        }
    }
}

```

掌握的知识点

fork和vfork的异同
 exec系列函数的使用形式
 exit和_exit函数区别
 僵尸进程和孤儿进程概念
 僵尸进程处理方法

登录方式
进程组和会话期
守护进程

登录方式

终端登录

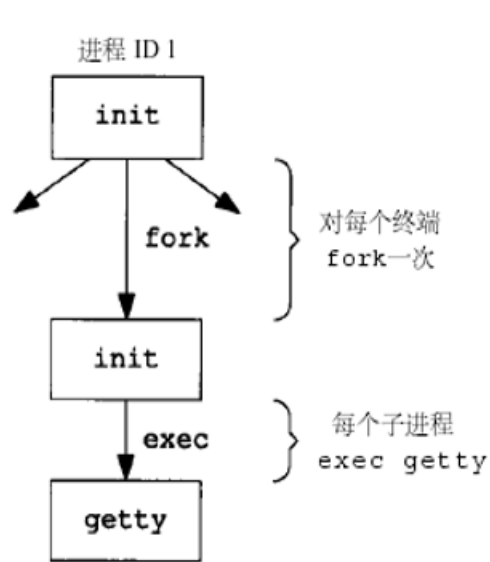


图9-1 init生成进程使终端可用于登录

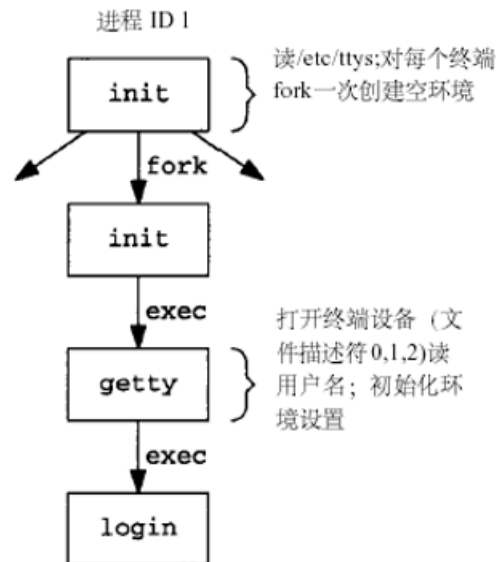


图9-2 login刚被调用后各进程的状态

网络登录

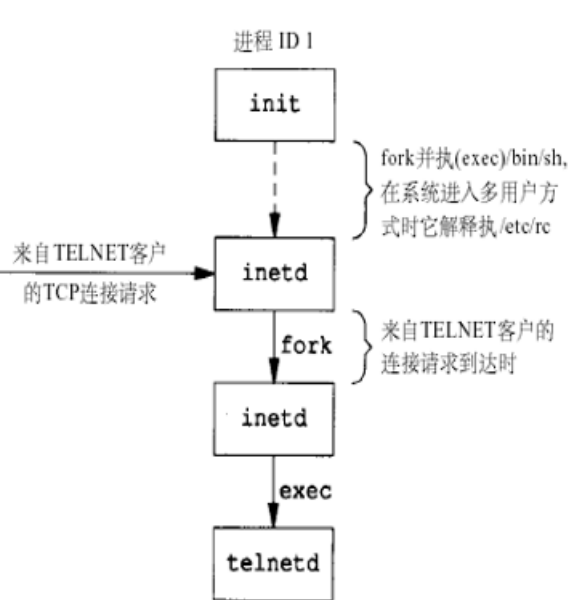


图9-4 执行TELNET服务进程中涉及的进程序列

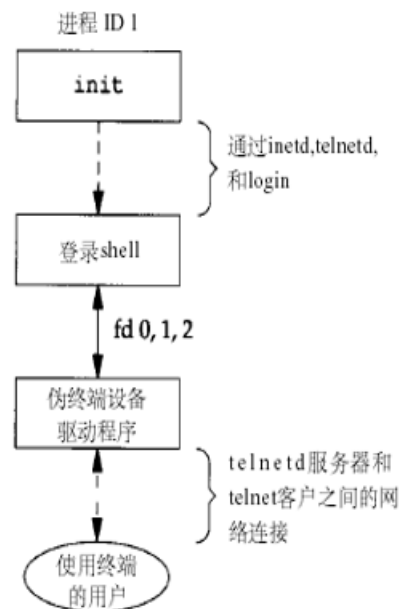


图9-5 为网络登录设置了fd 0, 1, 2后的进程安排

进程组和会话期

进程组

The set of one or more process(es).

getpgrp/setpgid functions

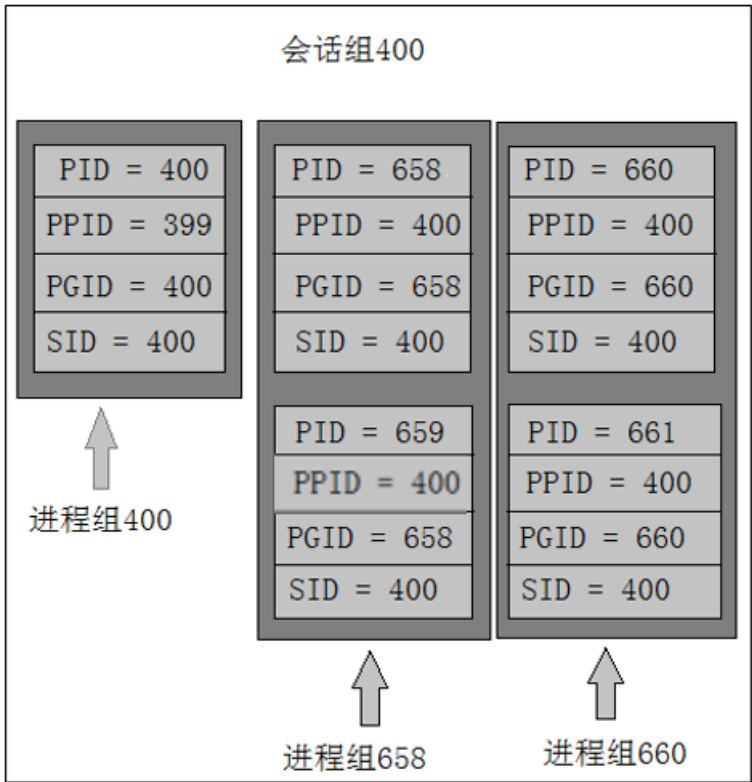
进程组长

会话组Session

The set of one or more process group(s).

getsid/setsid function

会话组长



getpgrp/setpgid函数

```
#include <unistd.h>
pid_t getpgid(pid_t pid);
pid_t getpgrp(void);           /* POSIX.1 version */
pid_t getpgrp(pid_t pid);     /* BSD version */
```

getpgid()函数返回进程号为参数pid的进程所属的进程组ID，如果参数pid为0，则返回调用进程所属的进程组ID
getpgrp()函数用于返回调用进程所属的进程组ID
getpgrp()函数则返回进程号为参数pid的进程所属的进程组ID

```
#include <unistd.h>
int setpgid(pid_t pid, pid_t pgid);
int setpgrp(void);             /* System V version */
int setpgrp(pid_t pid, pid_t pgid); /* BSD version */
```

setpgid()函数将进程号为参数pid的进程所属的进程组ID修改为参数pgid所表示的值。
setpgrp()函数用于修改调用进程所属进程组的ID为调用进程的ID（调用进程称为组长）

```
#include <unistd.h>
pid_t getsid(pid_t pid);
```

getsid()函数用于获得进程号为参数pid的进程所属的会话组的组ID。如果参数pid为0，则返回调用进程所属的会话组ID。

```
#include <unistd.h>
pid_t setsid(void);
```

setsid()函数会创建一个新会话，其前提是调用进程不能是进程组组长。

● setsid函数

所需头文件	#include<sys/types.h> #include<unistd.h>
函数功能	设置新的组进程号
函数原型	pid_t setsid(void);
函数传入值	无
函数返回值	执行成功则返回进程组号(gid)，失败则返回-1，失败原因存于errno 中。
相关函数	setregid, setegid, getsid, setregid
备注	如果调用此函数的进程不是一个进程组的组长，则此函数创建一个新对话期，结果为：(1) 此进程变成该新对话期的对话期首进程。此进程是该新对话期中的唯一进程。(2) 此进程成为一个新进程组的组长进程。新进程组号是此调用进程的进程号。(3) 此进程没有控制终端。如果在调用 setsid 之前此进程有一个控制终端，那么这种联系也被解除。 如果此调用进程已经是一个进程组的组长，则此函数返回出错。为了保证不处于这种情况，通常先调用 fork，然后使其父进程终止，而子进程则继续。

```
proc1 | proc2 &
proc3 | proc4 | proc5
```

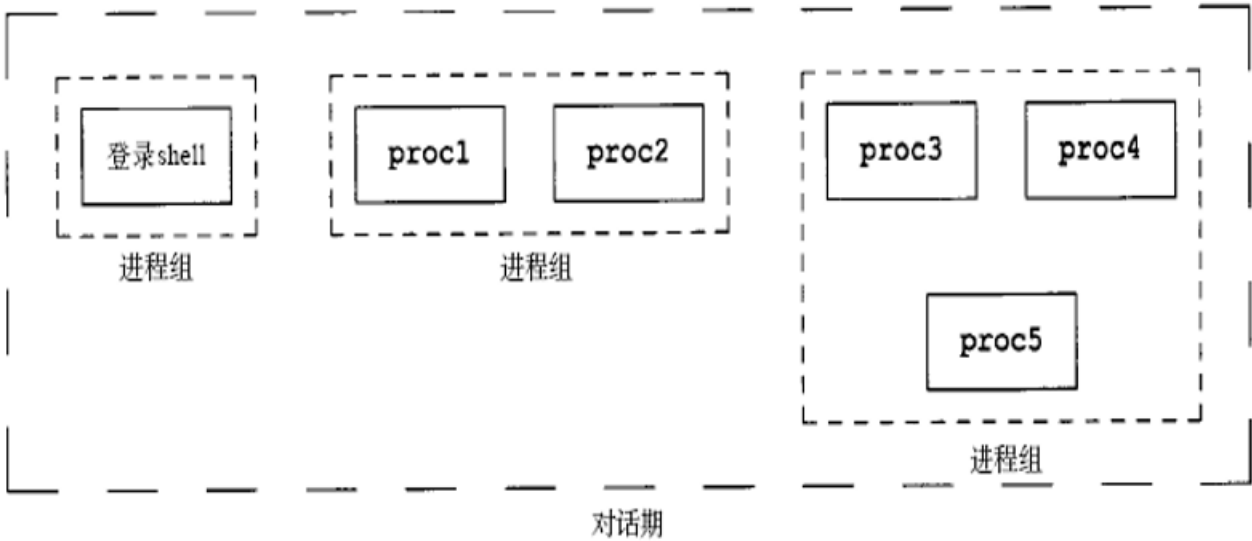
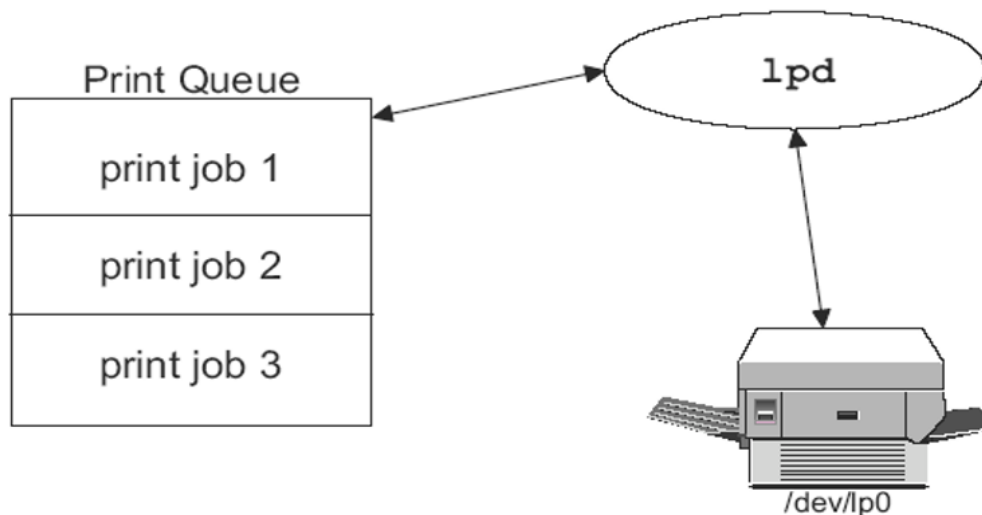


图9-6 进程组和对话期中的进程安排

守护进程Daemon

What is a daemon?
daemon进程的实现方法
编程规则
出错记录

- The word "Daemon" refers to a never-ending process, usually a system process that controls a system resource such as the printer queue or performs a network service



What is a daemon? (cont'd)

daemon
精灵进程或守护进程
后台执行，没有控制终端或登录 Shell 的进程

ps -aux 命令查看

Init:进程1, 启动系统服务
Keventd: 为内核中运行的函数提供进程上下文
Kswapd: 页面调出守护进程
bdfush, kupdated: 调整缓存中的数据写到磁盘
portmap: 将RPC程序号映射为端口号
inetd(xinetd): 侦听网络接口, 获取网络服务进程请求

注意：大多数守护进程都以超级用户（用户ID为0）特权运行。没有一个守护进程具有控制终端，其终端名设置为问号（?）。

ps axj命令查看

daemon特征：
sid, pid, pgid相同，均为pid
ppid为1
tty为?

2	691	0	0 ?
666	696	696	666 ?
666	699	699	666 ?
1	700	700	700 ?
1	817	817	817 ?
1	1012	1012	1012 ?
1	1013	1013	1013 ?
2	1026	0	0 ?
2	1027	0	0 ?
1	1038	1038	1038 ?

daemon进程实现规则

编程规则

首先调用fork，然后使父进程exit
调用setsid创建一个新的会话期 setsid()
将当前工作目录更改为特定目录chdir(./)
进程的umask设为0 umask(0)
关闭不需要的文件描述符 close()

Example

```
int daemon_init(void) {
    pid_t pid;
    if ((pid = fork()) < 0)
        return(-1);
    else if (pid != 0)
        exit(0);    /* parent goes bye-bye */

    /* child continues */
    setsid();        /* become session leader */
    chdir("/tmp");    /* change working directory */
    umask(0);        /* clear our file mode creation mask */
    for(i=0;i<MAXFILE;i++) /* close file */
        close(i);
    return(0);
}
```

编写守护进程的要点

- (1) 创建子进程，终止父进程


```
pid=fork();
if(pid>0)
{exit(0);}    /*终止父进程*/
```
- (2) 在子进程中创建新会话

setsid函数用于创建一个新的会话，并担任该会话组的组长，其作用：

 - 1) 让进程摆脱原会话的控制；
 - 2) 让进程摆脱原进程组的控制；
 - 3) 让进程摆脱原控制终端的控制。

而setsid函数能够使进程完全独立出来，从而脱离所有其他进程的控制。
- (3) 改变工作目录

改变工作目录的常见函数是chdir。
- (4) 重设文件创建掩码

文件创建掩码是指屏蔽掉文件创建时的对应位。
把文件创建掩码设置为0，可以大大增强该守护进程的灵活性。
设置文件创建掩码的函数是umask。
- (5) 关闭文件描述符

通常按如下方式关闭文件描述符：

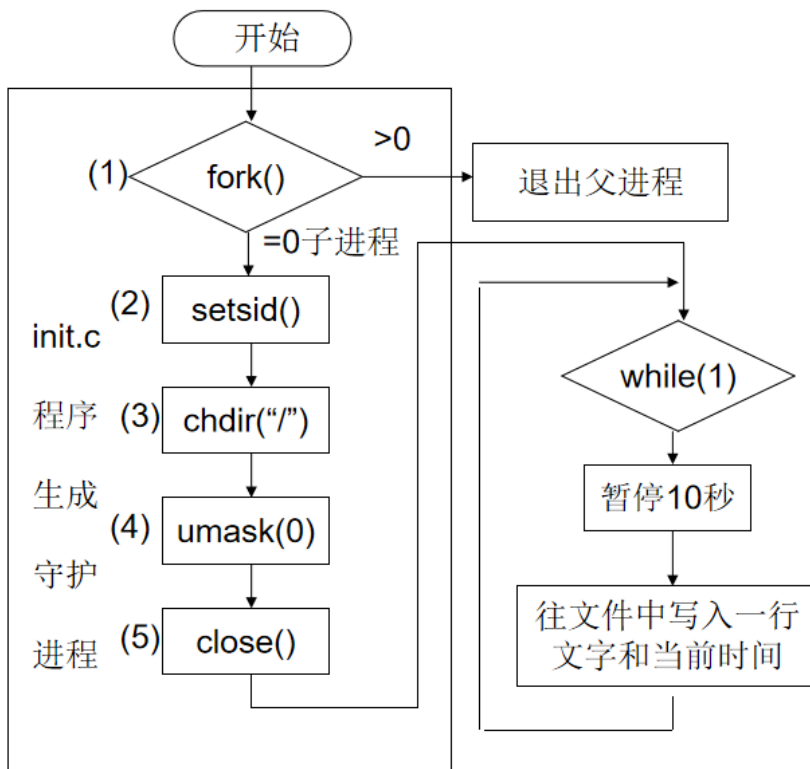

```

    for(i=0;i<NOFILE;i++)
        close(i);
或者也可以用如下方式:
    int num;
    num = getdtablesize();
    /*获取当前进程文件描述符表大小*/
    for (i = 0; i < num; i++) {
        close (i) ;
    }

```

守护进程编写

例3.7 设计两个程序，主程序和初始化程序。要求主程序每隔10秒向/tmp目录中的日志报告运行状态。初始化程序中的init_daemon函数负责生成守护进程。



```

#include <unistd.h>
#include <signal.h>
#include <sys/param.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <stdio.h>
#include <time.h>
#include <stdlib.h>
#include <unistd.h>
/*主程序每隔一分钟向/tmp目录中的日志3-7.log报告运行状态*/
void init_daemon(void);
int main()
{
    FILE *fp;
    time_t t;
    init_daemon();
    while(1)
    {
        sleep(10);
        if((fp=fopen("./3-7.log", "a+")) >=0)
        {
            t=time(0);

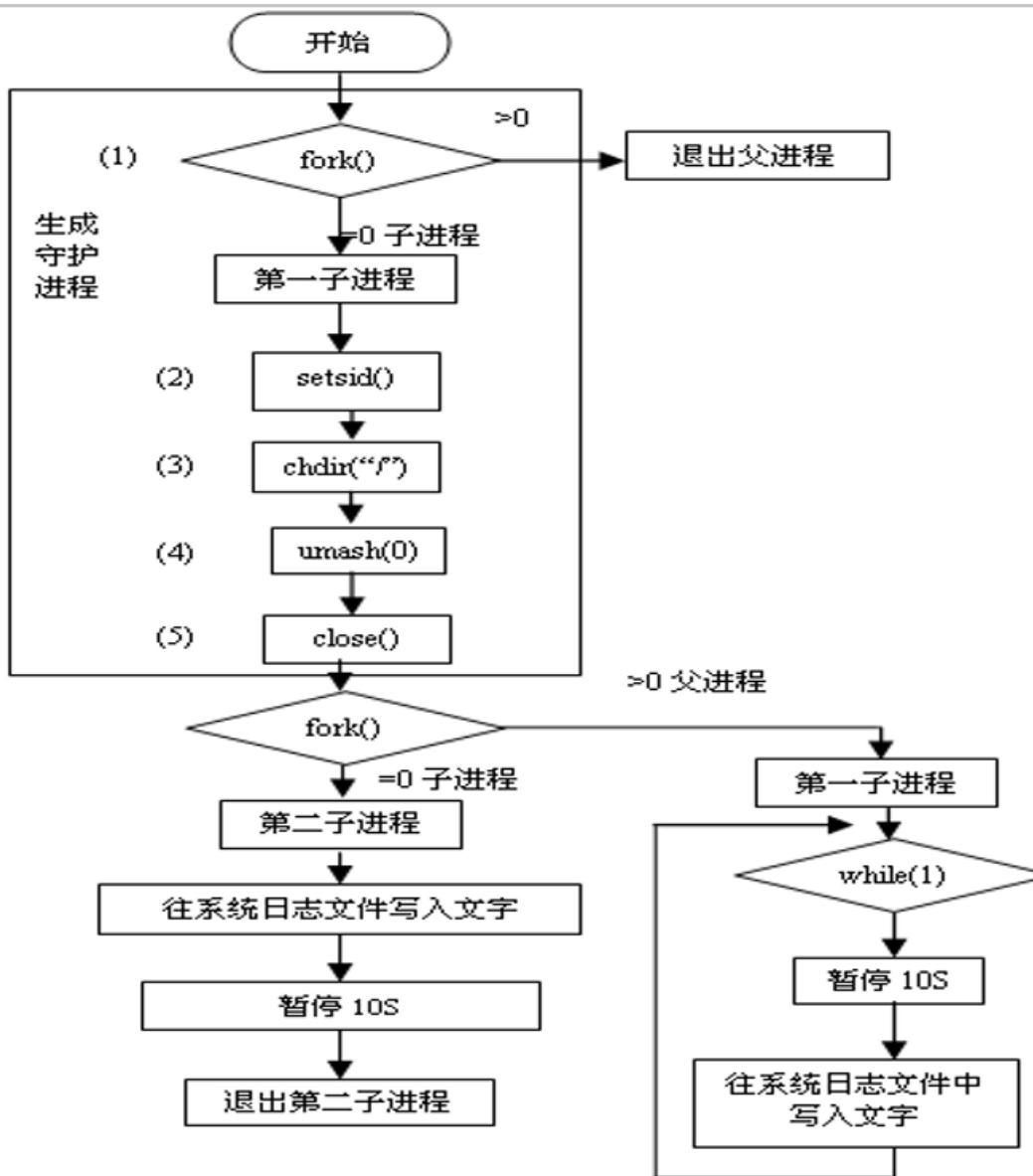
```

```

        fprintf(fp,"守护进程还在运行，时间是： %s",asctime(localtime(&t)) );
        fclose(fp);
    }
}
}
void init_daemon(void)
{
    pid_t child1,child2;
    int i;
    child1=fork();
    if(child1>0)
        exit(0);
    else if(child1< 0)
    {
        perror("创建子进程失败");
        exit(1);
    }
    setsid();
    chdir("/tmp");
    umask(0);
    for(i=0;i< NOFILE;++i)
        close(i);
    return;
}
//-----
cat 3-7.log
守护进程还在运行，时间是： Fri May 27 20:16:41 2022
守护进程还在运行，时间是： Fri May 27 20:16:51 2022
守护进程还在运行，时间是： Fri May 27 20:17:01 2022
.....
//-----
注意：fopen函数必须具有root权限。如果没有root权限，可以看到守护进程的运行，但不会在文件里写入任何字符。

```

例3-8：设计一个程序，要求运行后成为守护进程，守护进程又复制出一个子进程，守护进程和它的子进程都调用 syslog函数，把结束前的状态写入系统日志文件。



```

#include<stdio.h>
#include<stdlib.h>
#include<sys/types.h>
#include<unistd.h>
#include<sys/wait.h>
#include<syslog.h>
#include <signal.h>
#include <sys/param.h>
#include <sys/stat.h>
int main()
{
    pid_t child1,child2;
    int i;
    child1=fork();
    if(child1>0)                                /*(1)创建子进程，终止父进程*/
        exit(0);                                /*这是第一子进程，后台继续执行*/
    else if(child1< 0)
    {
        perror("创建子进程失败"); /*fork失败，退出*/
        exit(1);
    }
    setsid();                                  /*(2)在子进程中创建新会话*/
    chdir("/");                                /*(3)改变工作目录到"/"*/
    umask(0);                                  /*(4)重设文件创建掩码*/
    for(i=0;i< NOFILE;++i)                    /*(5)关闭文件描述符*/
  
```

```

        close(i);
    openlog("例3-8程序信息", LOG_PID, LOG_DAEMON); /* 调用openlog, 打开日志文件 */
    child2=fork();
    if(child2==-1)
    {
        perror("创建子进程失败");
        exit(1);
    }
    else if(child2==0)
    {
        syslog(LOG_INFO, "第二子进程暂停5秒!");
        sleep(5); /*睡眠5秒钟*/
        syslog(LOG_INFO, "第二子进程结束运行。");
        exit(0);
    }
    else
    {
        waitpid(child2, NULL, 0);
        syslog(LOG_INFO, "第一子进程在等待第二子进程结束后, 也结束运行。");
        closelog(); /*调用closelog, 关闭日志服务*/
        while(1) { sleep(10); }
    }
}

```

注意：调用openlog、syslog函数，操作的系统日志文件“/var/log/syslog”或“/var/log/messages”，必须具有root权限。

syslog函数说明

所需头文件	#include<syslog.h>
函数功能	准备做信息记录
函数原型	void openlog(char *ident,int option,int facility);
函数传入值	option 参数主要有： LOG_CONS 如果无法将信息送至 syslogd 则直接输出到控制台 LOG_PID 将信息字符串加上产生信息的进程号 (PID) facility 参数代表信息种类，主要有： LOG_CRON 由 cron 或 at 程序产生的信息 LOG_DAEMON 由系统 daemon 产生的信息
函数返回值	无
相关函数	syslog, closelog
备注	

所需头文件	#include<syslog.h>
函数功能	将信息记录至系统日志文件
函数原型	void syslog(int priority,char *format,.....);
函数传入值	priority 指定信息的种类或等级，主要有： LOG_INFO 提示相关信息 LOG_DEBUG 除错相关信息 format 参数和 printf 函数相同
函数返回值	无
相关函数	openlog, closelog
备注	

掌握的知识点

守护进程概念
守护进程编写要点
守护进程程序应用

Ch6 线程编程

线程概念
线程函数的介绍以及应用
线程互斥锁的应用
线程信号量的应用

线程概念

进程的所有信息对该进程的所有线程都是共享的，包括可执行的程序文本、程序的全局内存、堆内存、文件描述符等线程独有的：线程ID、寄存器值、栈、信号屏蔽字、`errno`值、线程私有数据

典型的UNIX进程，可以看成是只有一个线程的进程

线程：一个独立运行的函数

同进程一样，每个线程也有一个线程ID

进程ID在整个系统中是唯一的，但线程ID不同，线程ID只在它所属的进程环境中有效

线程ID的类型是`pthread_t`，在Linux中的定义：

```
在/usr/include/bits/pthreadtypes.h中
typedef unsigned long int pthread_t;
ps axm 或ps ax -L
gcc -o -pthread
```

基本API函数

线程创建——`pthread_create`函数

- (1)头文件：`#include<pthread.h>`
- (2)功能：创建一个线程
- (3)函数原型

```
int pthread_create(pthread_t *thread, pthread_attr_t *attr, void *(*start_routine)(void *), void *arg);
```
- (4)参数
 - 1)thread:线程标识符
 - 2)attr:线程属性设置，通常可设为NULL，采用默认线程属性
 - 3)start_routine:线程函数的起始地址
 - 4)arg:传递给start_routine的参数
- (5)返回值
- (6)成功：0；失败：error number

线程终止——`pthread_exit`函数

- (1)单个线程的三种退出方式
 - 1)线程从启动例程中返回，返回值是线程的退出码
 - 2)线程被同一进程中的其他线程取消
 - 3)线程调用`pthread_exit`函数
- (2)该函数让线程退出

```
#include<pthread.h>
void pthread_exit(void *rval_ptr);
```
- (3)参数
 - rval_ptr: 与线程的启动函数类似，该指针将传递给`pthread_join`函数

线程取消——pthread_cancel函数

- (1)功能：线程调用该函数可以取消同一进程中的其他线程，即让线程终止
- (2)函数原型

```
#include<pthread.h>
int pthread_cancel(pthread_t tid);
```
- (3)参数与返回值
 - tid: 需要取消的线程ID
 - 成功返回0， 出错返回错误编号
- (4)pthread_cancel并不等待线程终止，它仅仅是提出请求

线程回收——pthread_join函数

- (1)功能：该函数用于等待某个线程终止
- (2)函数原型

```
#include<pthread.h>
int pthread_join(pthread_t thread, void **rval_ptr);
```
- (3)返回值与参数
 - 1)成功返回0， 否则返回错误编号
 - 2)thread: 需要等待的线程ID
 - 3)rval_ptr:
 - 若线程从启动例程返回，rval_ptr将包含返回码
 - 若线程由于pthread_exit终止，rval_ptr即pthread_exit的参数
 - 若线程被取消，由rval_ptr指定的内存单元就置为PTHREAD_CANCELED
 - 若不关心线程返回值，可将该参数设置为NULL

例4-1线程创建

创建两个线程，线程1每秒打印全局变量的值并累计，线程2每2秒打印全局变量的值，两个线程打印5次后退出。main线程等待两个线程退出后显示两个线程已经退出。

例4-1thrdcreat.c

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
void task1(int *counter);
void task2(int *counter);
void cleanup(int counter1, int counter2);
int g1 = 0;
int g2 = 0;
int main(int argc, char *argv[])
{
    pthread_t thrd1, thrd2;
    int ret;
    void *retval;
    ret = pthread_create(&thrd1, NULL, (void *)task1, (void *)&g1);
    printf("I'm in main function\n");
    ret = pthread_create(&thrd2, NULL, (void *)task2, (void *)&g2);
    pthread_join(thrd1,&retval);
    pthread_join(thrd2,NULL);
    cleanup(g1, g2);
    getchar();
    printf("return val of task1 is %d\n", (int*)retval);
    cleanup(g1, g2);
    exit(EXIT_SUCCESS);
}
void task1(int *counter)
{
```

```

while(*counter < 5)
{
    printf("task1 count: %d\n", *counter);
    (*counter)++;
    sleep(1);
}
pthread_exit((void *)100);
}
void task2(int *counter)
{
    while(*counter < 5)
    {
        printf("task2 count: %d\n", *counter);
        (*counter)++;
        sleep(2);
    }
    pthread_exit(NULL);
}
void cleanup(int counter1, int counter2)
{
    printf("total iterations: %d\n", counter1 + counter2);
}

//-----
I'm in main function
task1 count: 0
task2 count: 0
task1 count: 1
task2 count: 1
task1 count: 2
task1 count: 3
task2 count: 2
task1 count: 4
task2 count: 3
task2 count: 4
total iterations: 10

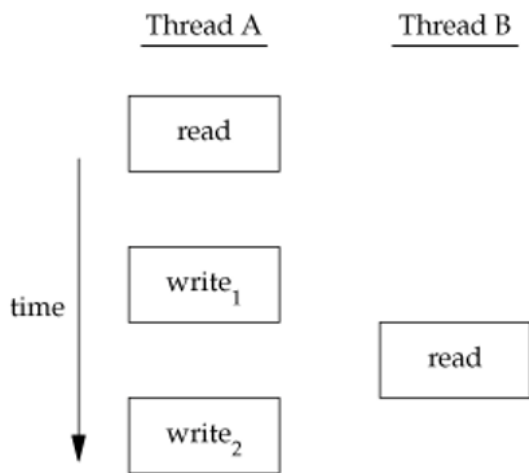
```

线程同步

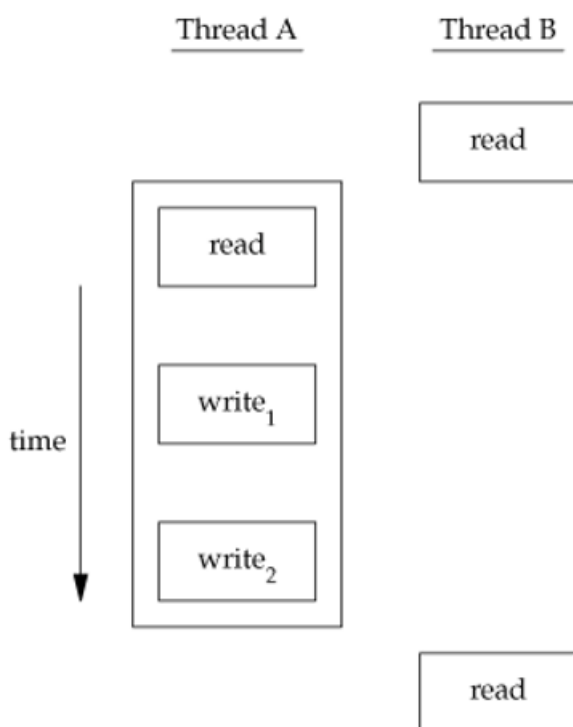
线程同步的概念

对同一个存储单元，至少存在两个执行体，其一读该单元，另一写该单元，则需要同步，避免不一致性
在处理器架构中，对存单元的修改，可能需要多个总线周期，因此读操作和写操作有可能交织在一起

假设读操作需要一个总线周期
写操作需要两个总线周期
线程B和线程A冲突



使用锁，以保证共享存储一次只能被一个线程访问
说明获取、释放锁的过程



线程同步的方法

互斥锁
读写锁
条件变量
信号量：用于线程的POSIX实时扩展

线程互斥锁

- (1) 可以通过使用pthread的互斥接口保护数据，确保同一时间里只有一个线程访问数据
- (2) 互斥锁mutex，本质上就是一把锁
 - 1) 在访问共享资源前，对互斥锁进行加锁
 - 2) 在访问完成后释放互斥锁上的锁
 - 3) 对互斥锁进行加锁后，任何其他试图再次对互斥锁加锁的线程将会被阻塞，直到锁被释放

线程互斥锁函数

头文件: #include<pthread.h>

线程互斥锁初始化pthread_mutex_init

线程互斥锁销毁pthread_mutex_destroy

线程互斥锁加锁和解锁pthread_mutex_lock pthread_mutex_unlock

1 线程互斥锁初始化

(1)互斥锁在使用前,必须要对互斥锁进行初始化

(2)函数原型

#include<pthread.h>

int pthread_mutex_init(pthread_mutex_t *mutex,const pthread_mutexattr_t *mutexattr);

(3)参数与返回值

1)mutex: 即互斥锁, 类型是pthread_mutex_t

注意: mutex必须指向有效的内存区域

2)attr: 设置互斥锁的属性, 通常可采用默认属性, 即可将attr设为NULL。

3)成功返回0, 出错返回错误码

2 线程互斥锁销毁

(1)互斥锁在使用完毕后,必须要对互斥锁进行销毁, 以释放资源

(2)函数原型

#include<pthread.h>

int pthread_mutex_destroy(pthread_mutex_t *mutex);

(3)参数与返回值

1)mutex: 即互斥锁

2)成功返回0, 出错返回错误码

4 线程互斥锁加锁和解锁

(1)在对共享资源访问之前和访问之后, 需要对互斥锁进行加锁和解锁操作

(2)函数原型

#include<pthread.h>

int pthread_mutex_lock(pthread_mutex_t *mutex);

int pthread_mutex_unlock(pthread_mutex_t *mutex);

5 尝试锁

(1)当使用pthread_mutex_lock时, 若已被加锁, 则调用线程将被阻塞。

(2)pthread_mutex_trylock让线程不阻塞, 即实现非阻塞的语义

(3)函数原型

#include<pthread.h>

int pthread_mutex_trylock(pthread_mutex_t *mutex);

(4)调用该函数时, 若互斥量未加锁, 则锁住该互斥量, 返回0; 若互斥量已加锁, 则函数直接返回失败, 即EBUSY

6 互斥锁的操作顺序

定义一个互斥锁pthread_mutex_t

调用pthread_mutex_init初始化互斥锁

调用pthread_mutex_lock或者pthread_mutex_trylock对互斥锁进行加锁操作

调用pthread_mutex_unlock对互斥锁解锁

调用pthread_mutex_destroy销毁互斥锁

例 4-2线程互斥锁4-2lock_mutex.c

创建两个线程，线程1执行时加1，线程2减1。

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
void task1(void);
void task2(void);
int sharedi=0;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
int main(void) {
    pthread_t thrd1, thrd2;
    int ret;
    ret = pthread_create(&thrd1, NULL, (void *)task1, NULL);
    if (ret) {
        perror("pthread_create: task1");
        exit(EXIT_FAILURE);
    }
    ret = pthread_create(&thrd2, NULL, (void *)task2, NULL);
    if (ret) {
        perror("pthread_create: task2");
        exit(EXIT_FAILURE);
    }
    pthread_join(thrd1, NULL);
    pthread_join(thrd2, NULL);
    pthread_mutex_destroy(&mutex);
    printf("sharedi = %d\n", sharedi);
}
void task1(void)
{
    long i,tmp;
    for(i=0; i<1000000; i++) {
/* if(pthread_mutex_lock(&mutex) != 0) {
        perror("pthread_mutex_lock");
        exit(EXIT_FAILURE);    }
*/
        tmp = sharedi;
        tmp = tmp + 1;
        sharedi = tmp;
/* if (pthread_mutex_unlock(&mutex) != 0) {
        perror("pthread_mutex_unlock");
        exit(EXIT_FAILURE);    }
*/
    }
}
void task2(void)
{
    long i,tmp;
    for(i=0; i<1000000; i++) {
/* if(pthread_mutex_lock(&mutex) != 0) {
        perror("pthread_mutex_lock");    exit(EXIT_FAILURE);    }
*/
        tmp = sharedi;
        tmp = tmp - 1;
        sharedi = tmp;
/* if (pthread_mutex_unlock(&mutex) != 0) {
        perror("pthread_mutex_unlock");
        exit(EXIT_FAILURE);    }
*/
    }
}
```

```
//-----
结果不唯一
root@ubuntu:/home/liuxinyu# ./lock_mutex
sharedi = -81302
root@ubuntu:/home/liuxinyu# ./lock_mutex
sharedi = -194593
root@ubuntu:/home/liuxinyu# ./lock_mutex
sharedi = 135428
//-----
把代码中的注释都去掉，结果唯一
root@ubuntu:/home/liuxinyu# ./lock_mutex
sharedi = 0
root@ubuntu:/home/liuxinyu# ./lock_mutex
sharedi = 0
root@ubuntu:/home/liuxinyu# ./lock_mutex
sharedi = 0
```

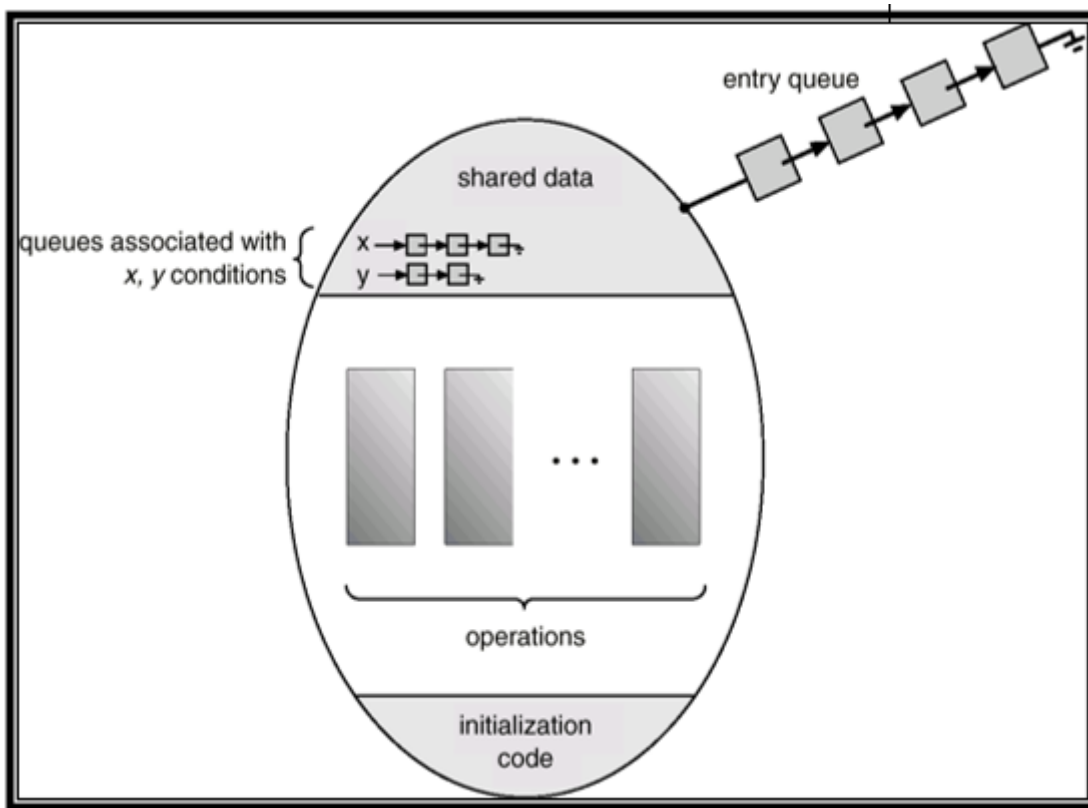
互斥锁特点

- (1)两种状态：锁定和非锁定
- (2)死锁问题：
 - 1.若线程试图对同一个互斥量加锁两次，那么它自身就会陷入死锁状态
 - 2.多个互斥量时可能出现死锁
 - 1)一个线程锁住互斥量A，等待互斥量B解锁
 - 2)另一个线程锁住B，而等待A

条件变量

因为互斥锁的状态只有锁定和非锁定两种状态，无法决定线程执行先后，有一定的局限。而条件变量通过允许线程阻塞和等待另一个线程发送信号实现线程的同步。

条件变量
`condition x, y;`
 条件变量的操作
 阻塞操作: `wait`
 唤醒操作: `signal`
`x.wait()`: 进程阻塞直到另外一个进程调用`x.signal()`
`x.signal()`: 唤醒另外一个进程



头文件: `#include <pthread.h>`

初始化 `pthread_cond_init` 函数

销毁一个条件变量 `pthread_cond_destroy` 函数

阻塞等待一个条件变量 `pthread_cond_wait` 函数

唤醒某个线程 `pthread_cond_signal` 函数

唤醒所有线程 `pthread_cond_broadcast` 函数

`int pthread_cond_init (pthread_cond_t cond, pthread_condattr_t cond_attr);`

功能: 初始化条件变量

cond: 待初始化的条件变量

cond_attr: 条件变量的属性, 通常为 NULL, 执行默认属性

`int pthread_cond_destroy(pthread_cond_t *cond);`

功能: 销毁条件变量

`int pthread_cond_wait (pthread_cond_t *cond, pthread_mutex_t *mutex)`

功能: 让调用者线程进入睡眠, 并解锁一个互斥量

cond: 线程睡眠的条件变量

mutex: 线程睡眠前的要解锁的互斥量

必须放在 `pthread_mutex_lock()` 函数和 `pthread_mutex_unlock()` 函数之间

`int pthread_cond_signal(pthread_cond_t *cond)`

功能: 唤醒条件变量中的一个线程

注意: 线程醒的前提条件是互斥量必须是解锁状态的, 线程醒前会再次加锁, 如果不能加锁就不会醒来。

例 4-3 线程条件变量 4-3 condition.c

创建三个线程, 线程1输入, 线程2读取, 线程3读取

```
pthread_mutex_lock(&lock); /*执行加锁操作*/
pthread_cond_wait(&cond, &lock); /*线程执行阻塞, 此时自动执行解锁, 当线程收到唤醒信号, 函数立即返回, 此时在进入临界区之前, 再次自动加锁*/
printf("thread2 buf:%s\n", buf); /*临界区*/
sleep(1);
pthread_mutex_unlock(&lock); /*解除互斥锁*/
```

```
#include <stdio.h>
```

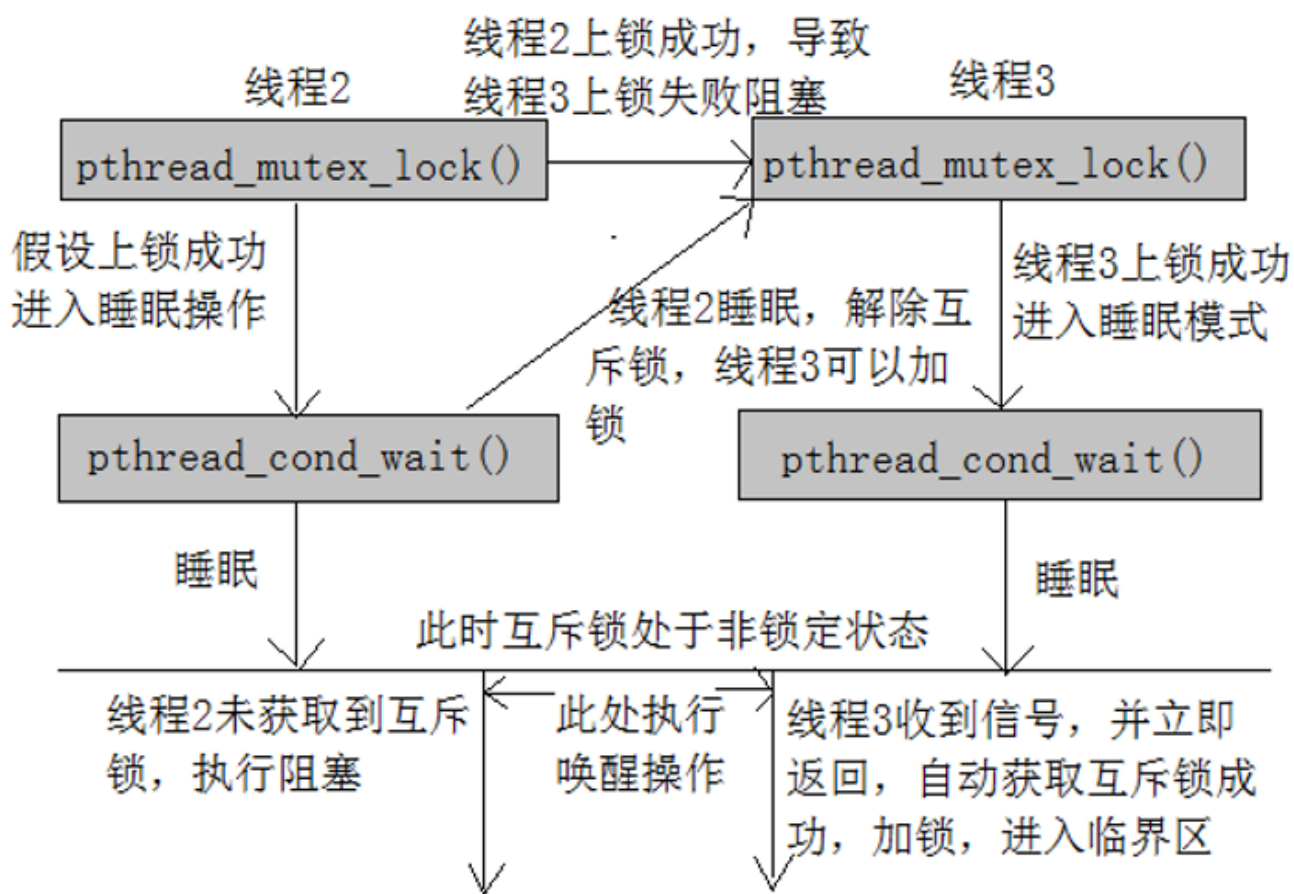
```

#include <pthread.h>
#include <semaphore.h>
#include <string.h>
char buf[20] = "";
pthread_cond_t cond;
pthread_mutex_t lock;
void *thread1_handler(void *arg){
    while(1){
        fgets(buf, N, stdin);
        buf[strlen(buf) - 1] = '\0';
        pthread_cond_signal(&cond);
    }
    pthread_exit(0);
}
void *thread2_handler(void *arg){
    while(1){
        pthread_mutex_lock(&lock);
        pthread_cond_wait(&cond, &lock);
        printf("thread2 buf:%s\n", buf);
        sleep(1);
        pthread_mutex_unlock(&lock);
    }
    pthread_exit(0);
}
void *thread3_handler(void *arg){
    while(1){
        pthread_mutex_lock(&lock);
        pthread_cond_wait(&cond, &lock);
        printf("thread3 buf:%s\n", buf);
        sleep(1);
        pthread_mutex_unlock(&lock);
    }
    pthread_exit(0);
}
int main(int argc, const char *argv[])
{
    pthread_t thread1, thread2, thread3;
    if(pthread_cond_init(&cond, NULL) != 0){
        perror("pthread_cond_init error");
    }
    if(pthread_mutex_init(&lock, NULL) != 0){
        perror("pthread_mutex_init error");
    }
    if(pthread_create(&thread1, NULL,
        thread1_handler, NULL) != 0){
        perror("pthread_create1 error");
    }
    if(pthread_create(&thread2, NULL,
        thread2_handler, NULL) != 0){
        perror("pthread_create2 error");
    }
    if(pthread_create(&thread3, NULL,
        thread3_handler, NULL) != 0){
        perror("pthread_create3 error");
    }
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);
    pthread_join(thread3, NULL);
    pthread_mutex_destroy(&lock);
    pthread_cond_destroy(&cond);
    return 0;
}

```

```
//-----
liu
thread2 buf:liu
xin
thread3 buf:xin
yu
thread2 buf:yu
liu
thread3 buf:liu
xin
thread2 buf:xin
yu
thread3 buf:yu
.....
```

例 4-3线程条件变量4-3condition.c



信号量

信号量也就是操作系统中所用到的PV 原语，它广泛用于进程或线程间的同步与互斥。信号量本质上是一个非负的整数计数器，它被用来控制对公共资源的访问。

Linux 实现了POSIX 的无名信号量，主要用于线程间的互斥同步。

(1)函数定义

```
#include <semaphore.h>
int sem_init(sem_t *sem,int pshared,unsigned int value)
```

(2)函数参数

sem: 信号量

pshared: 决定信号量能否在几个进程间共享。由于目前Linux还没有实现进程间共享信号量，所以这个值只能取0

value: 信号量初始化值

(3)函数返回值

```

    成功: 0;  出错: -1
(4)int sem_wait(sem_t *sem)
    int sem_trywait(sem_t *sem)
    int sem_post(sem_t *sem)
    int sem_getvalue(sem_t *sem,int * sval);
    int sem_destroy(sem_t *sem)
    函数传入值sem: 信号量
    函数返回值
(5)成功: 0
    出错: -1

```

信号量举例

信号量互斥 例4-4sem_mutex.c

信号量同步 例4-5sem_syn.c 4-6sem.c

例4-4sem_mutex.c

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <errno.h>
#include <sys/ipc.h>
#include <semaphore.h>
例4-4sem_mutex.c
sem_t sem;
void task1(void);
void task2(void);
int sharedi=0;
int main(void) {
    int ret;
    pthread_t id1,id2;
    ret=sem_init(&sem,0,1);
    if(ret!=0) perror("sem_init");
    ret=pthread_create(&id1,NULL,(void *)task1, NULL);
    if(ret!=0) perror("pthread create1");
    ret=pthread_create(&id2,NULL,(void *)task2, NULL);
    if(ret!=0) perror("pthread create2");
    pthread_join(id1,NULL);
    pthread_join(id2,NULL);
    sem_destroy(&sem);
    printf("sharedi = %d\n", sharedi);
}
void task1(void)
{
    long i,tmp;
    for(i=0; i<1000000; i++)
    {
        sem_wait(&sem);
        tmp = sharedi;
        tmp = tmp + 1;
        sharedi = tmp;
        sem_post(&sem);
    }
}
void task2(void)
{
    long i,tmp;
    for(i=0; i<1000000; i++)

```

```

    {
        sem_wait(&sem);
        tmp = sharedi;
        tmp = tmp + 1;
        sharedi = tmp;
        sem_post(&sem);
    }
}

//-----
sharedi = 2000000

```

例4-5sem_syn.c

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <string.h>
#include <semaphore.h>
void *thread_function(void *arg); sem_t sem;
#define WORK_SIZE 1024
char work_area[WORK_SIZE];
int main()
{
    int res;
    pthread_t a_thread; void *thread_result;
    res=sem_init(&sem,0,0);
    if(res!=0)
    {
        perror("Semaphore initialization failed");
        exit(EXIT_FAILURE);
    }
    res=pthread_create(&a_thread,NULL,thread_function,NULL);
    if(res!=0)
    {
        perror("Thread creation failed");
        exit(EXIT_FAILURE);
    }
    printf("Input some text.Enter 'end' to finish\n");
    while(strncmp("end",work_area,3)!=0)
    {
        fgets(work_area,WORK_SIZE,stdin);
        sem_post(&sem);
    }
    printf("\n Waiting for thread to finish...\n");
    res=pthread_join(a_thread,&thread_result);
    if(res!=0){
        perror("Thread join failed");
        exit(EXIT_FAILURE); }
    printf("Thread joined\n");
    sem_destroy(&sem);
    exit(EXIT_SUCCESS);
}
void *thread_function(void*arg){
    sem_wait(&sem);
    while(strncmp("end",work_area,3)!=0){
        printf("You input %d characters\n",strlen(work_area)-1);
        sem_wait(&sem);
    }
}

```



```

    pthread_exit(NULL);
}
//-----
Input some text.Enter 'end' to finish
liuxinyu
You input 8 characters
end

Waiting for thread to finish...
Thread joined

```

例4-6sem.c

```

#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>
#define MAXSIZE 10
int stack[MAXSIZE][2];
int size = 0;  sem_t sem;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
void provide_data(void)
{
    int i;
    for (i = 0; i < MAXSIZE; i++)
    {
        stack[i][0] = i;
        stack[i][1] = i;
        sem_post(&sem);
    }
}
void handle_data1(void)
{
    int i;
    while (pthread_mutex_lock(&mutex), ((i = size++) < MAXSIZE))
    {
        pthread_mutex_unlock(&mutex);
        sem_wait(&sem);
        printf("Plus:   %d + %d = %d\n", stack[i][0], stack[i][1], stack[i][0] + stack[i][1]);
    }
    pthread_mutex_unlock(&mutex);
}
void handle_data2(void)
{
    int i;
    while (pthread_mutex_lock(&mutex), ((i = size++) < MAXSIZE)) {
        pthread_mutex_unlock(&mutex);
        sem_wait(&sem);
        printf("Multiple: %d * %d = %d\n", stack[i][0], stack[i][1], stack[i][0] * stack[i][1]);
    }
    pthread_mutex_unlock(&mutex);
}
int main(void)
{
    pthread_t thrd1, thrd2, thrd3;
    sem_init(&sem, 0, 0);
    pthread_create(&thrd1, NULL, (void *)handle_data1, NULL);
    pthread_create(&thrd2, NULL, (void *)handle_data2, NULL);
    pthread_create(&thrd3, NULL, (void *)provide_data, NULL);
    pthread_join(thrd1, NULL);

```

```

pthread_join(thrd2, NULL);
pthread_join(thrd3, NULL);
sem_destroy(&sem);
return 0;
}

//-----
Plus:    0 + 0 = 0
Plus:    2 + 2 = 4
Plus:    3 + 3 = 6
Plus:    4 + 4 = 8
Plus:    5 + 5 = 10
Plus:    6 + 6 = 12
Plus:    7 + 7 = 14
Plus:    8 + 8 = 16
Plus:    9 + 9 = 18
Multiple: 1 * 1 = 1

```

线程属性

- (1)pthread_create时，针对线程属性，传入的参数都是NULL。
- (2)实际上，可以通过构建pthread_attr_t结构体，设置若干线程属性
- (3)要使用该结构体，必须首先对其进行初始化；使用完毕后，需要销毁它

1 线程属性初始化和销毁

- (1)函数原型

```

#include<pthread.h>
int pthread_attr_init(pthread_attr_t *attr);
int pthread_attr_destroy(pthread_attr_t *attr);

```
- (2)参数与返回值
 - 1)成功返回0，否则返回错误编号
 - 2)attr: 线程属性，确保attr指向的存储区域有效
 - 3)为了移植性，pthread_attr_t结构对应用程序是不可见的，应使用设置和查询等函数访问属性

2 设置分离状态属性

- (1)每个线程默认是非分离状态，线程被调用pthread_join的线程回收资源；如果一个非分离状态线程结束运行但没有被join，则 它的状态类似于进程中的Zombie Process， 还有一部分资源没有被回收。
- (2)设置为分离属性，由系统释放资源函数，不需要其他线程调用pthread_join函数。分离属性设置方法：
 - 1.pthread_attr_setdetachstate函数，使调用线程处于分离状态，以便让OS在线程退出时收回它所占的资源
 2. 调用函数pthread_detach(threadid)或pthread_detach(pthread_self())分离。

设置分离状态属性方法1

- (1)函数原型

```

#include<pthread.h>
int pthread_attr_setdetachstate(pthread_attr_t *attr, int detachstate);

```
- (2)参数与返回值
 - 1)成功返回0，否则返回错误编号
 - 2)attr: 线程属性结构体指针
 - 3)detachstate:
 - 1.PTHREAD_CREATE_DETACHED: 以分离状态启动线程
 - 2.PTHREAD_CREATE_JOINABLE: 正常启动线程（默认属性）

(1)函数原型

```
#include<pthread.h>
int pthread_attr_getdetachstate(const pthread_attr_t *attr, int *detachstate);
```

(2)参数与返回值

- 1)成功返回0，否则返回错误编号
- 2)attr: 线程属性结构体指针
- 3)detachstate指向的整数，可被设置为以下值
 - 1.PTHREAD_CREATE_DETACHED: 以分离状态启动线程
 - 2.PTHREAD_CREATE_JOINABLE: 正常启动线程（默认属性）

设置分离状态属性1举例

例4-7thread_detach1.c

```
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>
#include <stdlib.h>
void *thread_function(void *arg);
char message[]="Hello World";
int thread_finished=0;
int main()
{
    int res;
    pthread_t a_thread;
    pthread_attr_t thread_attr;
    res=pthread_attr_init(&thread_attr);
    if(res!=0)
    {
        perror("Attribute creation failed");
        exit(EXIT_FAILURE);
    }
    res=pthread_attr_setdetachstate(&thread_attr,PTHREAD_CREATE_DETACHED);
    if(res!=0)
    {
        perror("setting detached attribute failed");
        exit(EXIT_FAILURE);
    }
    res=pthread_create(&a_thread,&thread_attr,thread_function,(void*)message);
    if(res!=0)
    {
        perror("Thread creation failed");
        exit(EXIT_FAILURE);
    }
    (void)pthread_attr_destroy(&thread_attr);
    while(!thread_finished)
    {
        printf("Waiting for thread to say it's finished...\n");
        sleep(1);
    }
    printf("Other thread finished,bye\n");
    exit(EXIT_SUCCESS);
}
void *thread_function(void*arg)
{
    printf("thread_function is running.Argument was %s\n",(char*)arg);
    sleep(4);
    printf("second thread setting finished flag,and exiting now\n");
    thread_finished=1;
```

```

    pthread_exit(NULL);
}
//-----
Waiting for thread to say it's finished...
thread_function is running.Argument was Hello World
Waiting for thread to say it's finished...
Waiting for thread to say it's finished...
Waiting for thread to say it's finished...
second thread setting finished flag,and exiting now
Other thread finished,bye

```

设置分离状态属性方法2

1. pthread_detach(threadid)函数

功能：使线程ID为threadid的线程处于分离状态。

一旦线程处于分离状态，该线程终止时底层资源立即被回收。通常是主线程使用pthread_create()创建子线程以后，调用该函数分离刚刚创建的子线程。

2. pthread_detach(pthread_self())函数

功能：线程设置自己的分离状态。

被创建的线程也可以自己分离自己，pthread_self() 函数返回线程自己的ID。

设置分离状态属性2举例

例4-8thread_detach2.c

```

#include <stdio.h>
#include <pthread.h>
void *thread_handler(void *arg){
    pthread_detach(pthread_self());
    int count = *((int *)arg);
    while(count > 0){
        printf("thread...\n");
        sleep(1);
        count--;
    }
    return NULL;
}
int main(int argc, const char *argv[])
{
    pthread_t thread;
    int arg = 3;
    if(pthread_create(&thread, NULL, thread_handler, (void *)&arg) != 0)
    {
        perror("pthread_create1 error");
    }
    int temp = 0;
    sleep(1);
    if(pthread_join(thread, NULL) == 0)
    {
        printf("pthread wait success\n");
        temp = 0;
    }
    else
    {
        printf("pthread wait failed\n");
        temp = 0;
    }
    return temp;
}
//-----

```

```
thread...
pthread wait failed
```

3 设置取消线程属性

(1) 函数原型

```
#include<pthread.h>
int pthread_setcancelstate(int *state, int *oldstate);
```

(2) 参数与返回值

- 1) 成功返回0，否则返回错误编号
- 2) state : 取值为
 1. PTHREAD_CANCEL_ENABLE: 允许线程接收取消请求（默认设置）
 2. PTHREAD_CANCEL_DISABLE: 忽略取消请求

```
int pthread_setcanceltype(int *type, int *oldtype);
```

(1) 参数与返回值

- 1) 成功返回0，否则返回错误编号
- 2) type : 取值为
 1. PTHREAD_CANCEL_ASYNCHRONOUS: 收到取消请求后立即行动
 2. PTHREAD_CANCEL_DEFERRED: 收到取消请求后延迟行动（默认设置）

```
void pthread_testcancel(void);
```

- 1) 当线程取消功能处于启用状态且取消类型设置为延迟模式时，pthread_testcancel() 函数有效
- 2) 在执行到pthread_testcancel的位置时，线程才可能响应cancel退出进程

设置取消线程属性举例

4-9thread_cancel.c

```
#include <stdio.h>
#include <stdlib.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
int g1 = 0;  int g2 = 0;  pthread_t thrd1, thrd2;
void task1(int *counter)
{
    volatile int i;
    //pthread_setcancelstate(PTHREAD_CANCEL_DISABLE, NULL);
    pthread_setcanceltype(PTHREAD_CANCEL_ASYNCHRONOUS, NULL);
    while((*counter) < 5)
    {
        printf("task1 count: %d\n", *counter);
        (*counter)++;sleep(1);
    }
    printf("*counter in task1 is : %d\n",*counter);
}
void task2(int *counter)
{
    while(*counter < 5)
    {
        printf("task2 count: %d\n", *counter);
        (*counter)++;
    }
}
void cleanup(int counter1, int counter2)
{
    printf("total iterations: %d\n", counter1 + counter2);
}
```

```

}
int main(int argc, char *argv[])
{
    pthread_t thrd1, thrd2;
    int ret;
    ret = pthread_create(&thrd1, NULL, (void *)task1, (void *)&g1);
    if (ret)
    {
        perror("pthread_create: task1");
        exit(EXIT_FAILURE);
    }
    ret = pthread_create(&thrd2, NULL, (void *)task2, (void *)&g2);
    if (ret)
    {
        perror("pthread_create: task2");
        exit(EXIT_FAILURE);
    }
    sleep(2);
    pthread_join(thrd2, NULL);
    pthread_cancel(thrd1);
    printf("in main , after pthread_cancel\n");
    pthread_join(thrd1, NULL);
    cleanup(g1, g2);
    exit(EXIT_SUCCESS);
}
//-----
root@ubuntu:/home/liuxinyu# ./thread_cancel
task1 count: 0
task2 count: 0
task2 count: 1
task2 count: 2
task2 count: 3
task2 count: 4
task1 count: 1
in main , after pthread_cancel
total iterations: 7
root@ubuntu:/home/liuxinyu# ./thread_cancel
task1 count: 0
task2 count: 0
task2 count: 1
task2 count: 2
task2 count: 3
task2 count: 4
task1 count: 1
task1 count: 2
in main , after pthread_cancel
total iterations: 8

```

掌握的知识点

线程的创建、终止函数
 线程互斥锁、信号量的应用
 线程主要属性的设置

Ch7 进程通信

信号
管道
IPC通信（消息队列、共享内存、信号量）
网络通信

信号

信号概念
信号相关命令
signal 函数
信号发送
kill, raise
alarm和 pause 函数

1.1 信号概念

(1)信号
1)Software interrupt软件中断
2)Mechanism for handling asynchronous events异步事件
3)Having a name (beginning with SIG)
4)Defined as a positive integer (in <signal.h>)
(2)信号产生
1)按终端键，硬件异常，kill(2)函数，kill(1)命令，软件条件，...

kill -l

SIG信号（1-31）是从UNIX系统中继承下来的称为不可靠信号(也称为非实时信号)。
SIGRTMIN~SIGRTMAX是为了解决前面“不可靠信号”问题而进行更改和扩充的信号，称为可靠信号(也称为实时信号)。

可靠信号(实时信号)：支持排队，发送用户进程一次就注册一次，发现相同信号已经在进程中注册，也要再注册。
不可靠信号(非实时信号)：不支持排队，发送用户进程判断后注册，发现相同信号已经在进程中注册，就不再注册，忽略该信号。

Signals in Linux/UNIX—kill -l

名称	说明	默认操作
SIGABRT	进程异常终止（调用abort函数产生此信号）	
SIGALRM	超时（alarm）	终止
SIGFPE	算术运算异常（除以0，浮点溢出等）	
SIGHUP	连接断开	终止
SIGILL	非法硬件指令	
SIGINT	终端终端符(Clt-C)	终止
SIGKILL	立即结束程序运行（不能被捕捉、阻塞或忽略）	终止
SIGPIPE	向没有读进程的管道写数据	
SIGQUIT	终端退出符(Clt-^)	终止
SIGTERM	终止（由kill命令发出的系统默认终止信号）	退出
SIGUSR1	用户定义信号	退出
SIGUSR2	用户定义信号	退出

名称	说明	默认操作
SIGSEGV	无效存储访问（段违例）	
SIGCHLD	子进程停止或退出	忽略
SIGCONT	使暂停进程继续	继续/忽略
SIGSTOP	暂停一个进程（不能被捕捉、阻塞或忽略）	终止
SIGTSTP	终端挂起符(CIt-Z)	停止进程
SIGTTIN	后台进程请求从控制终端读	
SIGTTOU	后台进程请求向控制终端写	

信号处理

(1)忽略信号
不能忽略的信号：
1)SIGKILL，SIGSTOP
2)一些硬件异常信号
(2)执行系统默认动作
(3)捕捉信号

1.2 信号相关命令

Kill
暂停 kill -STOP <pid>
恢复 kill -CONT <pid>
终止 kill -KILL <pid>

1.3 信号相关函数

函数	功能
kill	发送信号给进程或进程组
raise	发送信号给进程自身
alarm	定时器时间到，向进程发送SIGALRM信号
pause	没有捕捉信号前一直将进程挂起
signal	捕捉信号SIGINT, SIGQUIT时执行信号处理函数

函数	功能
sigemptyset	初始化信号集合为空
sigfillset	初始化信号集合为所有信号集合
sigaddset	将指定信号加入到指定集合
sigdelset	将指定信号从信号集中删除
sigprocmask	判断检测或更改信号屏蔽字

1 信号发送——kill & raise

```
kill: send signal to a process
#include <sys/types.h>
#include <signal.h>
int kill(pid_t pid, int sig);
    (Returned Value: 0 if success, -1 if failure)
The "pid" parameter
pid > 0: 发送信号给进程id为pid的进程。
pid =0:发送信号给与自己同组的所有进程。
pid = -1: 发送系统内所有进程
pid < -1: 发送给进程组号为-pid的所有进程。
```

```
raise: send a signal to the current process
#include <signal.h>
int raise(int sig);
    (Returned Value: 0 if success, -1 if failure)
```

例5-1

设计一程序，要求用户进程复制出一个子进程，父进程向子进程发出SIGKILL信号，子进程收到此信号，结束子进程的运行。

分析：用户进程fork子进程后，子进程使用raise函数发送SIGSTOP信号，使自己暂停；父进程使用kill函数向子进程发送SIGKILL信号，子进程收到信号结束。

```
#include <stdio.h>
#include <stdlib.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <sys/types.h>
#include <signal.h>
int main ()
{
    pid_t result;
    int ret;
    int newret;
    result=fork();
    if(result<0)
    {
```

```

    perror("创建子进程失败");
    exit(1);
}
else if (result==0)
{
    raise(SIGSTOP);    /*调用raise函数，发送SIGSTOP 使子进程暂停*/
    exit(0);
}
else
{
    printf("子进程的进程号(PID)是: %d\n",result);
    if((waitpid(result,NULL,WNOHANG))==0) /*Waitpid (pid, NULL, WNOHANG) 没有子进程终止立即返回，返回值为0*/
    {
        if((ret=kill(result,SIGKILL))==0)
            printf("用kill函数返回值是: %d, 发出的SIGKILL信号结束的进程进程号: %d\n",ret,result);
        else
        {
            perror("kill函数结束子进程失败"); waitpid(result,NULL,0);
        }
    }
}

//-----
子进程的进程号(PID)是: 2451804
用kill函数返回值是: 0, 发出的SIGKILL信号结束的进程进程号: 2451804

```

由此例可知，系统调用kill函数和raise函数，都是简单地向某一进程发送信号。kill函数用于给特定的进程或进程组发送信号，raise函数用于向一个进程自身发送信号。

2 信号处理——“signal”函数

```

(1)Installs a new signal handler for the signal with number signum.
#include <signal.h>
typedef void (*sighandler_t)(int);
sighandler_t signal(int signum, sighandler_t handler);
    (Returned Value: the previous handler if success, SIG_ERR if error)
(2)The "handler" parameter
    a user specified function, or
    SIG_DFL, or
    SIG_IGN

```

The "signal" function (cont'd)

```

//Program example
static void sig_usr(int);
int main(void)
{
    if (signal(SIGUSR1, sig_usr) == SIG_ERR)
        err_sys("can't catch SIGUSR1");
    if (signal(SIGUSR2, sig_usr) == SIG_ERR)
        err_sys("can't catch SIGUSR2");

    for ( ; ; )
        pause();
}

```

3 alarm & pause 函数

```
//alarm: set an alarm clock for delivery of a signal
#include <unistd.h>
unsigned int alarm(unsigned int seconds);
    (Returned value: 0, or the number of seconds remaining of previous alarm)
//pause: wait for a signal
#include <unistd.h>
int pause(void);
    (Returned value: -1, errno is set to be EINTR)
```

alarm & pause 函数

```
//Program example
实现"sleep" 函数:
signal(SIGALRM,fun)
alarm()
pause()
```

4.信号阻塞

有时既不希望进程在接收到信号时立刻中断进程的执行，也不希望此信号完全被忽略掉，而是延迟一段时间再去调用信号处理函数，这个时候就需要信号阻塞来完成。

信号集处理函数

```
#include <signal.h>
int sigemptyset(sigset_t *set);
int sigfillset(sigset_t *set);
int sigaddset(sigset_t *set,int signum);
int sigdelset(sigset_t *set,int signum);
int sigismember(const sigset_t *set,int signum);
    参数: set 信号集 ; signum 信号
    返回值: 若成功返回0, 若出错返回-1。
    sigismember若真返回1, 若假返回0, 若出错返回-1。
```

例5-2 signalset1

```
#include <stdio.h>
#include <signal.h>
#include<stdlib.h>
main()
{
    sigset_t *set;
    set=(sigset_t*)malloc(sizeof(sigset_t));
    sigemptyset(set);
    sigaddset(set,SIGUSR1);
    sigaddset(set,SIGINT);
    if((sigismember(set,SIGUSR1))==1)    printf("SIGUSR1\n");
    if((sigismember(set,SIGUSR2))==1)    printf("SIGUSR2\n");
    if((sigismember(set,SIGINT))==1)    printf("SIGINT\n");
}

//-----
SIGUSR1
SIGINT
```

信号集处理函数sigprocmask

```
#include <signal.h>
```

功能：检测或更改信号屏蔽字

函数：int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);

参数：how 操作方式set

how决定函数的操作方式。

SIG_BLOCK：增加一个信号集合到当前进程的阻塞集合之中。

SIG_UNBLOCK：从当前的阻塞集合之中删除一个信号集合。

SIG_SETMASK：将当前的信号集合设置为信号阻塞集合。

返回值：若成功返回0，若出错返回-1。

例5-3 signalset2

```
#include <stdio.h>
#include <signal.h>
#include <stdlib.h>
main()
{
    sigset_t *set;
    set=(sigset_t*)malloc(sizeof(sigset_t));
    sigemptyset(set);
    sigaddset(set,SIGINT);
    sigprocmask(SIG_SETMASK,set,NULL);
    //也可以使用: sigprocmask(SIG_BLOCK,set,NULL);
    while(1);
}
```

程序先定义信号集set，然后把信号SIGINT添加到set信号集中，最后把set设置为信号阻塞集合。运行程序时，进程进入死循环。按“ctrl+c”系统并没有中断程序，SIGINT信号屏蔽掉了。按“ctrl+z”来结束程序。

信号实例

```
void handler()
{
    alarm(1);
}
main(int argc,char *argv[])
{
    char buf[BUFSIZE];
    signal(SIGALRM,handler);
    alarm(1);
    sfd = open(argv[1],O_RDONLY); //①
    while(1)
    {
        pause();
        len = read(sfd,buf,10); //②
        ret = write(1,buf,len); //③
    }
}

(1)
do{
    sfd = open(argv[1],O_RDONLY);
    if (sfd<0){
        if (errno!= EINTR){
            perror("open()");
            exit(1);
        }
    }
}
```

```

    }
}
}while(sfd<0);
(2)
while((len = read(sfd,buf,BUFSIZE))<0){
    if (errno == EINTR)
        continue;
    perror("read()");
    break;
}if (len==0) break;
(3)
while(len>0){
    ret = write(1,buf,len);
    if (ret <0){
        if (errno == EINTR)
            continue;
        perror("write");
        exit(1);
    }
    len -= ret;
}
}

```

掌握的知识点

信号的概念
 信号处理方式
 常见的信号及处理方式
 kill, signal, raise, pause, alarm
 信号集处理函数sigprocmask

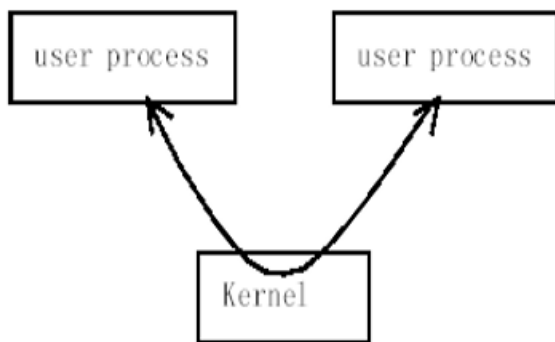
pipe管道

IPC 概念
 pipe
 popen pclose
 FIFO

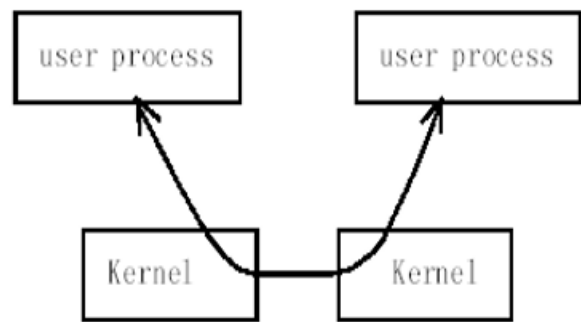
2.1 进程间通信

IPC: Inter-Process Communication
 IPC mechanisms
 shared file
 signal
 pipe, FIFO (named pipe), message queue, semaphore, shared memory
 socket

IPC illustrations



IPC on one host



IPC on different hosts:
Network Programming

Simple Client-Server or IPC model



Client reads filename from sdtin,
write it to the IPC channel, read the
file contents from the IPC channel

Server reads filename from the
IPC channel, opens file, write
the contents on the IPC channel

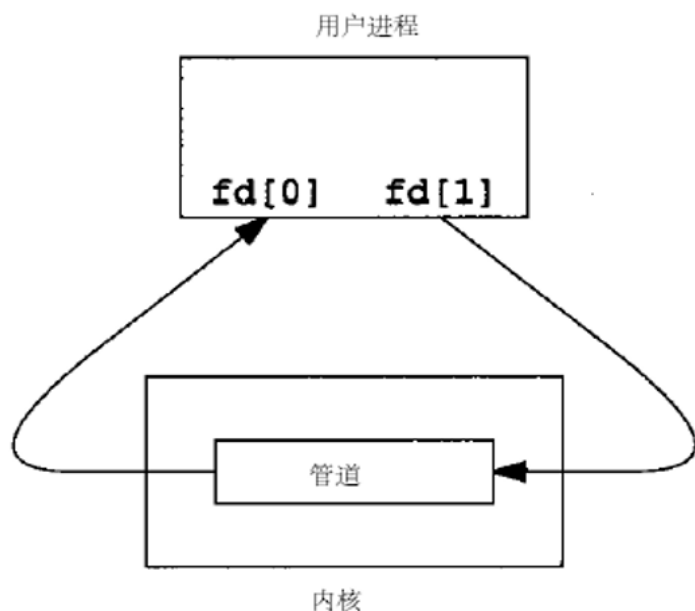
2.2 pipe 概念

Pipe
Pipe mechanism in a shell
e.g. `cmd1 | cmd2`
Pipe is half-duplex
管道只能在共同祖先的进程间使用
管道也是文件
命名管道(FIFO)

pipe 函数

```
//The pipe function: create a pipe
#include <unistd.h>
int pipe(int filedes[2]);
    (Returned value: 0 if success, -1 if failure)
A pipe: First In, First Out
    filedes[0]:read, filedes[1]: write
```

单个进程使用管道



父子进程使用管道

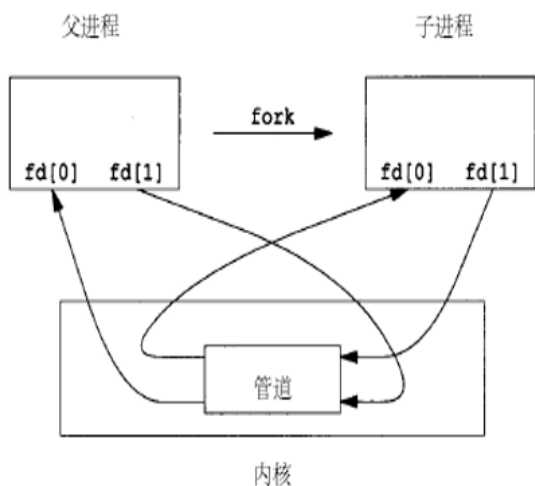


图14-2 fork之后的半双工管道

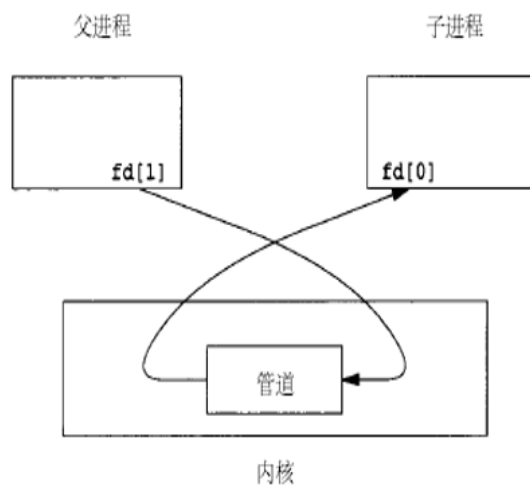


图14-3 从父进程到子进程的管道

管道使用(1): 单个进程

类似共享文件
pipe—write—read

管道使用 (2) 多进程

父-子进程/子-子进程
使用模式:
pipe—fork—write (写进程)
—read (读进程)
注意: pipe-fork顺序

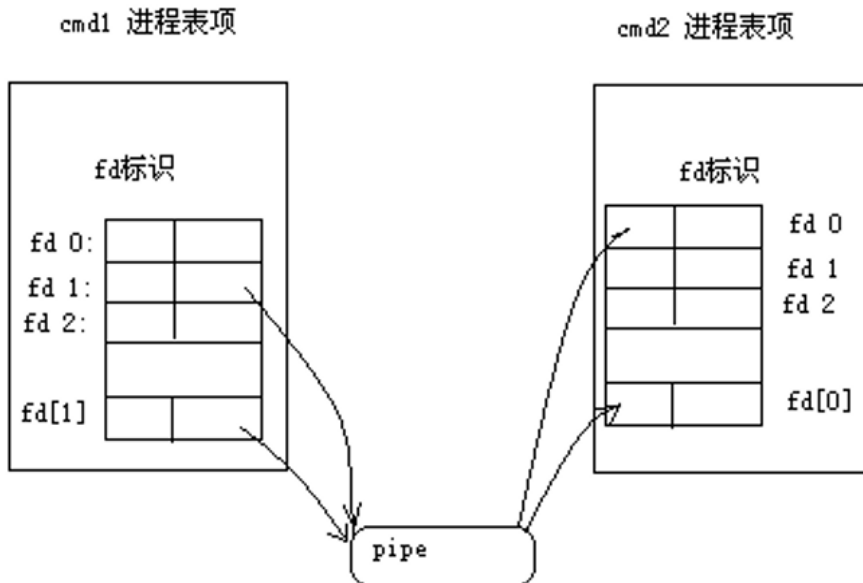
管道使用 (3) 用于标准输入输出

管道: shell中的形式
cmd1 | cmd2
重定向 cmd > file
实现代码
执行cmd1前
if (fd[1] != STDOUT_FILENO) {

```

    if (dup2(fd[1], STDOUT_FILENO) != STDOUT_FILENO)
        err_sys("dup2 error to stdout");
}
执行cmd2前
if (fd[0] != STDIN_FILENO) {
    if (dup2(fd[0], STDIN_FILENO) != STDIN_FILENO)
        err_sys("dup2 error to stdin");
}

```



例5-4

```

#include <stdio.h>
#include <stdlib.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <sys/types.h>
#include <signal.h>
int main()
{
    int data_processed;
    int file_pipes[2];
    const char some_data[] = "123";
    pid_t fork_result;
    if (pipe(file_pipes) == 0)
    {
        fork_result = fork();
        if (fork_result == (pid_t)-1)
        {
            fprintf(stderr, "Fork failure");
            exit(EXIT_FAILURE);
        }
        if (fork_result == (pid_t)0)
        {
            close(0);
            dup(file_pipes[0]);
            close(file_pipes[0]);
            close(file_pipes[1]);
            execlp("od", "od", "-c", (char *)0); //od -c file 以字符方式显示文件内容, 如果没
            exit(EXIT_FAILURE);
        }
    }
    else

```



```

    {
        close(file_pipes[0]);
        data_processed = write(file_pipes[1], some_data, strlen(some_data));
        close(file_pipes[1]);
        printf("%d - wrote %d bytes\n", (int)getpid(), data_processed);
    }
}
exit(EXIT_SUCCESS);
}

//-----
[root@VM-0-4-centos ~]# ./5_4
2459643 - wrote 3 bytes
[root@VM-0-4-centos ~]# 0000000 1 2 3
0000003

```

2.3 popen & pclose 函数

```

//popen, pclose: process I/O
#include <stdio.h>
FILE *popen(const char *command, const char *type);
int pclose(FILE *stream);

```



图14-5 fp=popen(command, "r")的结果

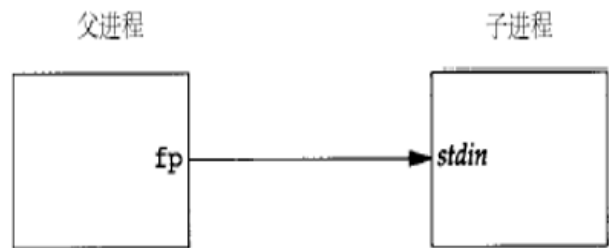


图14-6 fp=popen(command, "w")的结果

```

//fp=popen(cmd, "r")的pipe实现:
FILE * popen(char * cmd, char * type)
{
    int fd; int p;
    pipe(fd);
    p=fork();
    if(p==0)
    {
        close(1);
        dup(fd[1]);
        execl(cmd, cmd, NULL);
    }
    else
    {
        wait();
        fp=fdopen(fd[0], "r");
        return fp;
    }
}

```

popen 函数说明

所需头文件	#include<stdio.h>
函数功能	建立管道 I/O
函数原型	FILE * popen(const char * command,const char * type);
函数传入值	调用 fork()产生子进程，然后从子进程中调用/bin/sh -c 来执行参数 command 的指令。参数 type 可使用“r”代表读取，“w”代表写入。
函数返回值	若成功则返回文件指针，否则返回 NULL，错误原因存于 errno 中。
备注	在编写具 SUID/SGID 权限的程序时请尽量避免使用 popen()，popen()会继承环境变量，通过环境变量可能会造成系统安全的问题。

例5-5

设计一程序，要求用popen创建管道，实现“ls -l|grep fifo”的功能。

```
#include <stdio.h>
#include <stdlib.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <sys/types.h>
#include <signal.h>
int main ()
{
    FILE *fp;
    int num;
    char buf[500];
    memset(buf,0,sizeof(buf));
    printf("建立管道.....\n");
    fp=popen("ls -l","r");
    if(fp!=NULL)
    {
        num=fread(buf,sizeof(char),500,fp);
        if(num>0)
        {
            printf("第一个命令是“ls -l”，运行结果如下： \n");
            printf("%s\n",buf);
        }
        pclose(fp);
    }
    else
    {
        printf("用popen创建管道错误\n");
        return 1;
    }
    fp=popen("grep fifo 5_5.c","w");
    printf("第二个命令是'grep fifo 5_5.c'，运行结果如下： \n");
    fwrite(buf,sizeof(char),500,fp);
    pclose(fp);
    return 0;
}
```

```
//-----
建立管道.....
第一个命令是“ls -l”，运行结果如下：
total 164
-rwxr-xr-x 1 root root 18336 Jun 10 09:51 5_5
-rw-r--r-- 1 root root 702 Jun 10 09:50 5_5.c
-rwxr-xr-x 1 root ro
第二个命令是'grep fifo 5_5.c'，运行结果如下：
fp=popen("grep fifo 5_5.c", "w");
printf("第二个命令是'grep fifo'，运行结果如下： \n");
```

使用popen函数读写管道，实际上也是调用pipe函数建立一个管道，再调用fork函数建立子进程，接着会建立一个shell环境，并在这个shell环境中执行参数指定的进程。

popen & pclose应用

在程序中获得另一个程序的输出

例5-6 popen_two.c

```
gcc myucl.c -o myucl
```

```
gcc popen_two.c -o two
```

输入大写字母

按Ctrl+D结束程序

例5-6

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    char line[MAXLINE];
    FILE *fpin;
    fpin = popen("./myucl", "r");
    if( NULL == fpin )
        error_quit("popen error");
    while( 1 )
    {
        fputs("prompt> ", stdout);
        fflush(stdout);
        if( fgets(line, MAXLINE, fpin) == NULL )
            break;
        if( fputs(line, stdout) == EOF )
            perror("fputs error to pipe");
    }
    if( pclose(fpin) == -1 )
        perror ("pclose error");
    putchar('\n');
    return 0;
}
//-----
prompt> C
c
prompt> D
d
```

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    int c;
    while( 1 )
    {
        c = getchar();
        if( EOF == c )
            break;
        if( isupper(c) )
            c = tolower(c);
        putchar(c);
        if( '\n' == c )
            fflush(stdout);
    }
    return 0;
}

```

2.4 FIFO: named pipe命名管道

创建FIFO

命令行方式: `mknod filename p`
`mkfifo filename`
 程序方式: `mkfifo`: make a FIFO special file (a named pipe)
`#include <sys/types.h>`
`#include <sys/stat.h>`
`int mkfifo(const char *pathname, mode_t mode);`
 (Returned value: 0 if success, -1 if failure)
 例: `mkfifo("/tmp/myfifo",0666)`

访问FIFO

命令行方式
 (1) `cat < /tmp/myfifo` 读FIFO文件
 (2) `echo hello >/tmp/myfifo` 向FIFO写数据
 (3) `cat < /tmp/myfifo &`
`echo "hello" >/tmp/myfifo`

程序方式: 用open打开一个FIFO

Review: "open" system call
`int open(const char *pathname, int flags);`
 "flags" parameter
 必须指定的互斥模式:
`O_RDONLY`, `O_WRONLY`, `O_NONBLOCK`
`O_RDONLY`: 若无进程写方式打开FIFO, `open`阻塞
`O_RDONLY | O_NONBLOCK`: 若无进程写方式打开FIFO, `open`立即返回文件描述符
`O_WRONLY`: 若无进程读方式打开FIFO, `open`阻塞
`O_WRONLY | O_NONBLOCK`: 若无进程读方式打开FIFO, `open`返回ENXIO错误, -1

例5-7

两个程序通过FIFO传递数据，一个生产者程序创建并打开一个FIFO，向管道中写入数据。（5-7fifo_p.c）一个消费者程序，从FIFO中读取数据（5-7fifo_c.c）。

write.c

```
#define FIFO_NAME "/tmp/my_fifo"
#define BUFFER_SIZE PIPE_BUF
#define TEN_MEG (1024 * 1024 * 10)
int main()
{
    int pipe_fd;
    int res;
    int open_mode = O_WRONLY;
    int bytes_sent = 0;
    char buffer[BUFFER_SIZE + 1];
    if (access(FIFO_NAME, F_OK) == -1)
    {
        res = mkfifo(FIFO_NAME, 0777);
        if (res != 0)
        {
            fprintf(stderr, "Could not create fifo %s\n", FIFO_NAME);
            exit(EXIT_FAILURE);
        }
    }
    printf("Process %d opening FIFO O_WRONLY\n", getpid());
    pipe_fd = open(FIFO_NAME, open_mode);
    printf("Process %d result %d\n", getpid(), pipe_fd);
    if (pipe_fd != -1)
    {
        while(bytes_sent < TEN_MEG)
        {
            res = write(pipe_fd, buffer, BUFFER_SIZE);
            if (res == -1)
            {
                fprintf(stderr, "Write error on pipe\n"); exit(EXIT_FAILURE);
            }
            bytes_sent += res;
        }
        (void)close(pipe_fd);
    }
    else {
        exit(EXIT_FAILURE);
    }
    printf("Process %d finished\n", getpid());
    exit(EXIT_SUCCESS);
}
```

read.c

```
#define FIFO_NAME "/tmp/my_fifo"
#define BUFFER_SIZE PIPE_BUF
int main()
{
    int pipe_fd;
    int res;
    int open_mode = O_RDONLY;
    char buffer[BUFFER_SIZE + 1];
    int bytes_read = 0;
    memset(buffer, '\0', sizeof(buffer));
```

```

printf("Process %d opening FIFO O_RDONLY\n", getpid());
pipe_fd = open(FIFO_NAME, open_mode);
printf("Process %d result %d\n", getpid(), pipe_fd);
if (pipe_fd != -1)
{
    do
    {
        res = read(pipe_fd, buffer, BUFFER_SIZE);
        bytes_read += res;
    } while (res > 0);
    (void)close(pipe_fd)
}
else
{
    exit(EXIT_FAILURE);
}
printf("Process %d finished, %d bytes read\n", getpid(), bytes_read);
exit(EXIT_SUCCESS);
}

```

FIFO的应用(1)

● 用FIFO复制输出流

● 例

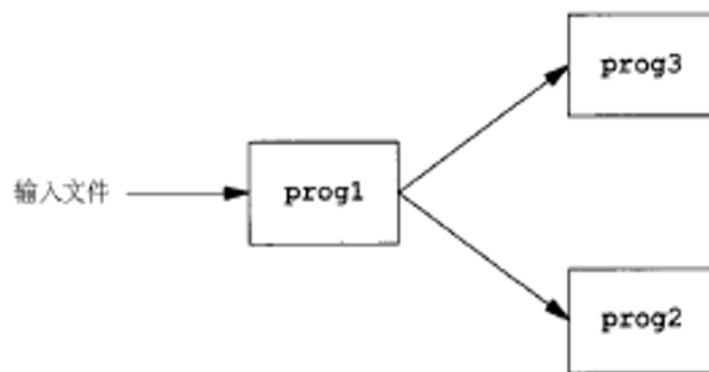


图14-9 对一个经过过滤的输入流进行两次处理

```

mkfifo fifo1
prog3 < fifo1 &
prog1 < infile | tee fifo1 | prog2

```

tee命令读取标准输入，把这些内容同时输出到标准输出和（多个）文件中

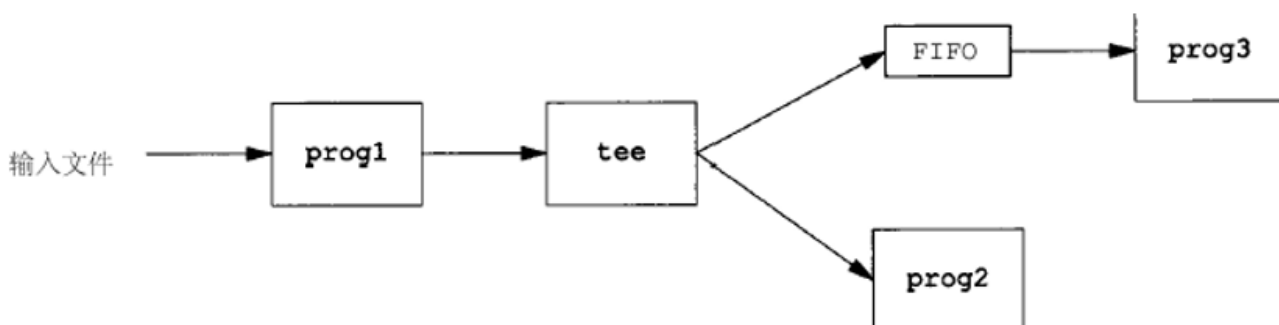


图14-10 使用FIFO和tee将一个流发送到两个不同的进程

FIFO的应用(2)

● C/S应用程序

● 例5-8 [client.c](#), [server.c](#)

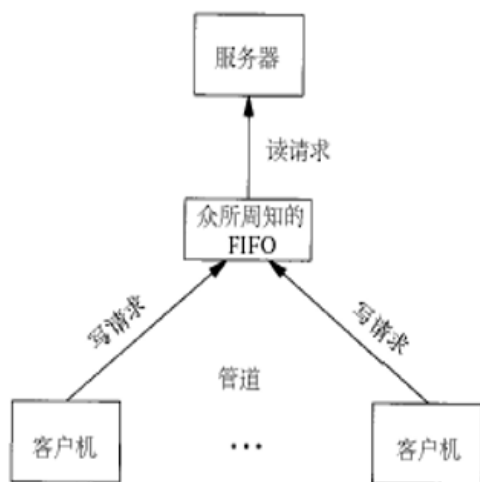


图14-11 客户机用FIFO向服务器发送请求

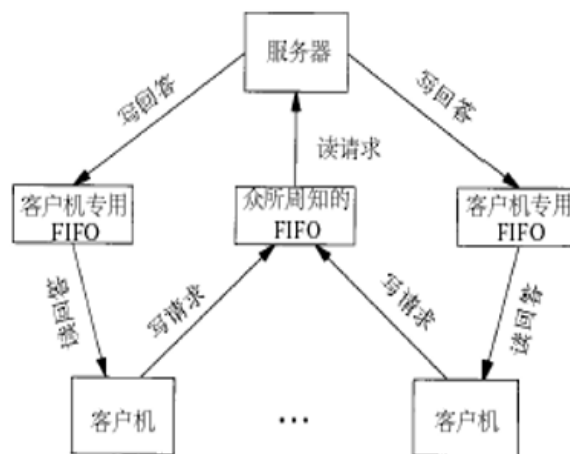


图14-12 客户机-服务器用FIFO进行通信

client.h

```
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <limits.h>
#include <sys/types.h>
#include <sys/stat.h>
#define SERVER_FIFO_NAME "/tmp/serv_fifo"
#define CLIENT_FIFO_NAME "/tmp/cli_%d_fifo"
#define BUFFER_SIZE 20
struct data_to_pass_st {
    pid_t client_pid;
    char some_data[BUFFER_SIZE - 1];
};
```

```
int main()    // 客户端程序
{
    int server_fifo_fd, client_fifo_fd;
    struct data_to_pass_st my_data;
    int times_to_send;
    char client_fifo[256];
    server_fifo_fd = open(SERVER_FIFO_NAME, O_WRONLY);
    if (server_fifo_fd == -1)
    {
        fprintf(stderr, "Sorry, no server\n");
        exit(EXIT_FAILURE);
    }
    my_data.client_pid = getpid();
    sprintf(client_fifo, CLIENT_FIFO_NAME, my_data.client_pid);
    if (mkfifo(client_fifo, 0777) == -1)
    {
        fprintf(stderr, "Sorry, can't make %s\n", client_fifo);
        exit(EXIT_FAILURE);
    }
}
```

```

for (times_to_send = 0; times_to_send < 5; times_to_send++)
{
    sprintf(my_data.some_data, "Hello from %d", my_data.client_pid);
    printf("%d sent %s, ", my_data.client_pid, my_data.some_data);
    write(server_fifo_fd, &my_data, sizeof(my_data));
    client_fifo_fd = open(client_fifo, O_RDONLY);
    if (client_fifo_fd != -1)
    {
        if (read(client_fifo_fd, &my_data, sizeof(my_data)) > 0)
        {
            printf("received: %s\n", my_data.some_data);
        }
        close(client_fifo_fd);
    }
}
close(server_fifo_fd);
unlink(client_fifo);
exit(EXIT_SUCCESS);
}

```

```

int main() //服务端程序
{
    int server_fifo_fd, client_fifo_fd;
    struct data_to_pass_st my_data;
    int read_res;
    char client_fifo[256];
    char *tmp_char_ptr;
    mkfifo(SERVER_FIFO_NAME, 0777);
    server_fifo_fd = open(SERVER_FIFO_NAME, O_RDONLY);
    if (server_fifo_fd == -1)
    {
        fprintf(stderr, "Server fifo failure\n");
        exit(EXIT_FAILURE);
    }
    sleep(10); /* lets clients queue for demo purposes */
    do {
        read_res = read(server_fifo_fd, &my_data, sizeof(my_data));
        if (read_res > 0)
        {
            tmp_char_ptr = my_data.some_data;
            while (*tmp_char_ptr)
            {
                *tmp_char_ptr = toupper(*tmp_char_ptr);
                tmp_char_ptr++;
            }
            sprintf(client_fifo, CLIENT_FIFO_NAME, my_data.client_pid);
            client_fifo_fd = open(client_fifo, O_WRONLY);
            if (client_fifo_fd != -1)
            {
                write(client_fifo_fd, &my_data, sizeof(my_data));
                close(client_fifo_fd);
            }
        }
    } while (read_res > 0);
    close(server_fifo_fd);
    unlink(SERVER_FIFO_NAME);
    exit(EXIT_SUCCESS);
}

```


设计两个程序，要求用命名管道FIFO实现简单的聊天功能。

Zhang.c

Li.c

Zhang.c

```
int main()
{
    int i, rfd,wfd,len=0,fd_in;
    char str[32];
    int flag,stdinflag;
    fd_set write_fd,read_fd;
    struct timeval net_timer;
    mkfifo("fifo1",S_IWUSR|S_IRUSR|S_IRGRP|S_IROTH);
    mkfifo("fifo2",S_IWUSR|S_IRUSR|S_IRGRP|S_IROTH);
    wfd =open("fifo1",O_WRONLY);
    rfd =open("fifo2",O_RDONLY);
    if(rfd<=0||wfd<=0)return 0;
    printf("这是张三端!\n");
    while(1)
    {
        FD_ZERO(&read_fd);
        FD_SET(rfd,&read_fd);
        FD_SET(fileno(stdin),&read_fd); /**/
        net_timer.tv_sec=5;
        net_timer.tv_usec=0;
        memset(str,0,sizeof(str));
        if(i=select(rfd+1, &read_fd,NULL, NULL, &net_timer) <= 0)
            continue;
        if(FD_ISSET(rfd,&read_fd))
        {
            read(rfd,str,sizeof(str));
            printf("-----\n");
            printf("李四:%s\n",str);
        }
        if(FD_ISSET(fileno(stdin),&read_fd))
        {
            printf("-----\n");
            fgets(str,sizeof(str),stdin);
            len=write(wfd,str,strlen(str));
        }
    }
    close(rfd);
    close(wfd);}
```

Li.c

```
int main()
{
    int i, rfd,wfd,len=0,fd_in;
    char str[32];
    int flag,stdinflag;
    fd_set write_fd,read_fd;
    struct timeval net_timer;
    mkfifo("fifo1",S_IWUSR|S_IRUSR|S_IRGRP|S_IROTH);
    mkfifo("fifo2",S_IWUSR|S_IRUSR|S_IRGRP|S_IROTH);
    rfd =open("fifo1",O_RDONLY);
    wfd =open("fifo2",O_WRONLY);
    if(rfd<=0||wfd<=0)
```

```

    return 0;
printf("这是李四端!\n");
while(1)
{
    FD_ZERO(&read_fd);
    FD_SET(rfd,&read_fd);
    FD_SET(fileno(stdin),&read_fd);
    net_timer.tv_sec=5;
    net_timer.tv_usec=0;
    memset(str,0,sizeof(str));
    if(i=select(rfd+1,&read_fd,NULL, NULL, &net_timer) <= 0)
        continue;
    if(FD_ISSET(rfd,&read_fd))
    {
        read(rfd,str,sizeof(str));
        printf("-----\n");
        printf("张三:%s\n",str);    }
    if(FD_ISSET(fileno(stdin),&read_fd))
    {
        printf("-----\n");
        fgets(str,sizeof(str),stdin);
        len=write(wfd,str,strlen(str)); /*写入管道*/
    }
}
close(rfd);
close(wfd);
}

```

掌握的知识点

管道两种实现方式的异同
 popen函数的实现及应用
 FIFO的原理和使用方法
 C-S通信程序设计

System V IPC

- (1)IPC objects
 - 1)信号量(semaphore set)
 - 2)消息队列(message queue)
 - 3)共享内存(shared memory)
- (2)shell命令
 - 1)ipcs -q/-m/-s
 - 2)ipcrm -q/-m/-s <id>
 - 3)ipcrm -Q/-M/-S <key>

3.1 System V IPC的共同特征

- (1)标识符与关键字
 - 1)创建IPC对象时指定关键字(key_t key;)
 key的选择：预定义常数IPC_PRIVATE；ftok函数
 - 2)引用IPC对象：标识符
 - 3)内核将关键字转换成标识符
- (2)许可权结构
 - 1)和文件类比
 - 2)struct ipc_perm

功能	消息队列	信号量	共享内存
分配一个IPC对象，获得对IPC的访问	msgget	semget	shmget
IPC操作: 发送/接收消息，信号量操作，连接/释放共享内存	msgsnd/ msgrcv	semop	shmat/ shmdt
IPC控制: 获得/修改状态信息，取消IPC	msgctl	semctl	shmctl

ftok函数

(1)创建函数

```
key_t ftok( char * filename, int id);
```

(2)功能说明

1)将一个已存在的文件名(该文件必须是存在而且可以访问的)和一个整数标识符id转换成一个key_t值

2)在Linux系统实现中，调用该函数时，系统将文件的索引节点号取出，并在前面加上子序号，从而得到key_t的返回值

创建IPC对象

key: 可由ftok()函数产生或定义为IPC_PRIVATE常量

flag: 包括读写权限，还可包含IPC_CREATE和IPC_EXCL标志位，组合效果如下

oflag标志	IPC对象不存在	IPC对象已存在
无特殊标志	出错， errno=ENOENT	成功，引用已存在对象
IPC_CREAT	成功，创建新对象	成功，引用已存在对象
IPC_CREAT IPC_EXCL	成功，创建新对象	出错， errno=EEXIST

3.2 Message queue

消息队列

1)消息队列是消息的链表，存放在内核中并由消息队列标识符标识。

2)First in, first out

3)message type: 优先级

4)块数据

消息队列——程序结构

```

proto.h:约定的消息队列通信格式
send.c
receive.c
proto.h:约定的消息队列通信格式:
#define KEYPATH "/etc/services"
#define KEYPROJ 'a'
struct msg_st {
    char * msg;
    long  type;
    ...
}

```

消息队列——系统函数

```

#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>

int msgget(key_t key, int flag);
int msgsnd(int msqid, const void *ptr, size_t nbytes, int flag);
int msgrcv(int msqid, void *ptr, size_t nbytes, long type, int flag);
int msgctl(int msqid, int cmd, struct shmid_ds *buf);

```

msgget

函数原型 `int msgget(key_t key, int flag);`

参数说明

- key:** 待创建/打开队列的键值
- flag:** 创建/打开方式
 - 常取 `IPC_CREAT | IPC_EXCL | 0666`
 - 若不存在 **key** 值的队列，则创建；否则，若存在，则打开队列
 - `0666` 表示与一般文件权限一样

返回值

- 成功返回消息队列描述字，否则返回 `-1`

说明

- `IPC_CREAT` 一般由服务器程序创建消息队列时使用
- 若是客户程序，须打开现有消息队列，而不用 `IPC_CREAT`

msgsnd

函数原型

```
int msgsnd(int msqid, struct msgbuf *msgp, size_t size, int flag);
```

说明

- flag** 有意义的标志为 `IPC_NOWAIT`，指明在消息队列没有足够空间容纳要发送的消息时，`msgsnd` 是否等待内核须对 `msgsnd()` 函数完成的工作
- 检查消息队列描述符、许可权及消息长度
 - 若合法，继续执行
 - 否则，返回
- 内核为消息分配数据区，将消息正文拷贝到消息数据区
- 分配消息首部，并将它链入消息队列的末尾
- 修改消息队列头数据，如队列中的消息数、字节总数等
- 唤醒等待消息的进程

msgrcv

```
int msgrcv(int msqid, void *ptr, size_t nbytes, long type, int flag);
```

参数说明

msgid: 消息队列描述符

msgp: 消息存储位置

size: 消息内容的长度

type: 请求读取的消息类型

flag: 规定队列无消息时内核应做的操作

IPC_NOWAIT: 无满足条件消息时返回，此时`errno=ENOMSG`

IPC_EXCEPT: `type>0`时使用，返回第一个类型不为`type`的消息

IPC_NOERROR: 如果队列中满足条件的消息内容大于所请求的`size`字节，则把该消息截断，截断部分将丢失

`msgctl`: message control operations

函数原型

```
int msgctl(int msqid, int cmd, struct msqid_ds *buf);
```

"cmd" 参数

IPC_STAT: 把`msqid_ds`结构中的数据置为消息队列的当前关联值

IPC_SET: 在进程有足够权限的前提下，把消息队列的当前关联值置为`msqid_ds`结构给出的值

IPC_RMID: 删除消息队列

消息队列——Example

A C/S application

One server, several clients: only one queue is required.

Compared with FIFO implementation

消息队列属性

(1)消息队列创建后，操作系统在内核中分配了一个名称为`msqid_ds`的数据结构用于管理该消息队列。

(2)在程序中可以通过函数`msgctl`对该结构进行读写，从而实现对消息队列的控制功能。

(3)成员说明：

1) `msg_perm`: IPC许可权限结构。

2) `msg_stime`: 最后一次向该消息队列发送消息（`msgsnd`）的时间。

3) `msg_rtime`: 最后一次从该消息队列接收消息（`msgrcv`）的时间。

4) `msg_ctime`: 最后一次调用`msgctl`的时间。

5) `msg_cbytes`: 当前该消息队列中的消息长度，以字节为单位。

6) `msg_qnum`: 当前该消息队列中的消息条数。

7) `msg_qbytes`: 该消息队列允许存储的最大长度，以字节为单位。

8) `msg_lspid`: 最后一次向该消息队列发送消息（`msgsnd`）的进程ID。

9) `msg_lrpid`: 最后一次从该消息队列接收消息（`msgrcv`）的进程ID。

(4)使用：

```
msg_sinfo.msg_qbytes = 1666;
```

```
msgctl(msgqid,IPC_SET,&msg_sinfo)
```

例5-10 `msg_stat.c`

```

struct msgbuf{
    int mtype;
    char mtext[1];
}msg_sbuf;
struct msgmbuf {
    int mtype;
    char mtext[10];
}msg_rbuf;

main( ){
    int gflags,sflags,rflags;
    key_t key;
    int msgid;
    int reval;

    struct msqid_ds msg_ginfo,msg_sinfo;

/*创建一个消息队列,输出消息队列缺省属性*/
char* msgpath="/UNIX/msgqueue";
key=ftok(msgpath,'a');
gflags=IPC_CREAT|IPC_EXCL;
msgid=msgget(key,gflags|00666);
if(msgid== -1) {
    printf("msg create error\n");
    return;
}
msg_stat(msgid,msg_ginfo);
sflags=IPC_NOWAIT;
msg_sbuf.mtype=10;
msg_sbuf.mtext[0]='a';

```

```

/*发送一个消息,输出消息队列属性*/
reval=msgsnd(msgid,&msg_sbuf,sizeof(msg_sbuf.mtext),sflags);
if(reval== -1) {
    printf("message send error\n");
}
msg_stat(msgid,msg_ginfo);
rflags=IPC_NOWAIT|MSG_NOERROR;
reval=msgrcv(msgid,&msg_rbuf,4,10,rflags);
if(reval== -1)
    printf("read msg error\n");
else
    printf("read from msg queue %d bytes\n",reval);

/*从消息队列中读出消息后,输出消息队列属性*/
msg_stat(msgid,msg_ginfo);
msg_sinfo.msg_perm.uid=8;
msg_sinfo.msg_perm.gid=8;
msg_sinfo.msg_qbytes=16388;
/*此处验证超级用户可以更改消息队列的缺省msg_qbytes*/
/*注意这里设置的值大于缺省值*/
reval=msgctl(msgid,IPC_SET,&msg_sinfo);
if(reval== -1) {
    printf("msg set info error\n");
    return;
}
msg_stat(msgid,msg_ginfo);
/*验证设置消息队列属性*/
reval=msgctl(msgid,IPC_RMID,NULL);/*删除消息队列*/
if(reval== -1) {
    printf("unlink msg queue error\n");
    return;
}

```

```

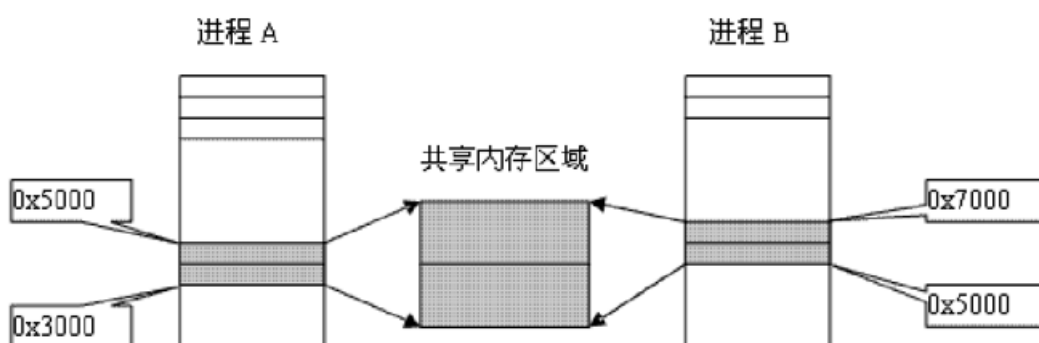
void msg_stat(int msgid, struct msqid_ds msg_info) {
    int reval;
    sleep(1); /*只是为了后面输出时间的方便*/
    reval=msgctl(msgid, IPC_STAT, &msg_info);
    if(reval== -1) {
        printf("get msg info error\n");
        return;
    }
    printf("\n");
    printf("current number of bytes on queue is %d\n", msg_info.msg_cbytes);
    printf("number of messages in queue is %d\n", msg_info.msg_qnum);
    printf("max number of bytes on queue is %d\n", msg_info.msg_qbytes);
    /*每个消息队列的容量（字节数）都有限制MSGMNB, 值的大小因系统而异*/
    /*在创建新的消息队列时, msg_qbytes的缺省值就是MSGMNB*/
    printf("pid of last msgsnd is %d\n", msg_info.msg_lspid);
    printf("pid of last msgrcv is %d\n", msg_info.msg_lrpid);
    printf("last msgsnd time is %s", ctime(&(msg_info.msg_stime)));
    printf("last msgrcv time is %s", ctime(&(msg_info.msg_rtime)));
    printf("last change time is %s", ctime(&(msg_info.msg_ctime)));
    printf("msg uid is %d\n", msg_info.msg_perm.uid);
    printf("msg gid is %d\n", msg_info.msg_perm.gid);
}

```

3.3 Shared memory

共享内存

- 1)共享内存是内核为进程创建的一个特殊内存段，它可连接(attach)到自己的地址空间，也可以连接到其它进程的地址空间
- 2)最快的进程间通信方式
- 3)不提供任何同步功能



共享内存实现途径

mmap() 系统调用

- 将普通文件在不同的进程中打开并映射到内存
- 不同进程通过访问映射来达到共享内存目的

POSIX共享内存机制 (Linux 2.6未实现)

System V共享内存

- 在内存文件系统tmpfs中建立文件
- 文件映射到不同进程空间

共享内存 (1)

mmap() 系统调用使得进程之间通过映射同一个普通文件实现共享内存。普通文件被映射到进程地址空间后，进程可以像访问普通内存一样对文件进行访问，不必再调用read(), write()等操作。

mmap()共享内存

```
#include <unistd.h>
#include <sys/mman.h>

void* mmap ( void * addr , size_t len , int prot , int flags , int fd , off_t offset )
int munmap( void * addr, size_t len )
int msync ( void * addr , size_t len, int flags)
```

相关函数 mmap

```
void* mmap ( void * addr , size_t len , int prot , int flags , int fd , off_t offset )
```

addr:指定文件应被映射到进程空间的起始地址，一般被指定一个空指针，此时选择起始地址的任务留给内核来完成。

len:映射到调用进程地址空间的字节数，从被映射文件开头**offset**个字节开始算起。

prot :指定共享内存的访问权限。可取如下几个值的或：PROT_READ（可读） ， PROT_WRITE （可写），PROT_EXEC（可执行）， PROT_NONE（不可访问）。

flags:指定映射对象的类型，映射选项和映射页是否可以共享。由以下几个常值指定：MAP_SHARED ， MAP_PRIVATE 必选其一。

fd:为即将映射到进程空间的文件描述符，一般由open()返回，同时，fd可以指定为-1，此时须指定flags参数中MAP_ANONYMOUS， 表明进行的是匿名映射（不涉及具体的文件名，避免了文件的创建及打开，只能用于具有亲缘关系的进程间通信）。

offset参数一般设为0，表示从文件头开始映射。

函数的返回值为最后文件映射到进程空间的地址，进程可直接操作起始地址为该值的有效地址。

mmap()用于共享内存的两种方式：

- (1) 使用普通文件提供的内存映射：适用于任何进程之间；需要打开或创建一个文件，然后再调用mmap()；

调用代码如下：

```
fd=open(name, flag, mode);
if(fd>0)
ptr=mmap(NULL, len , PROT_READ|PROT_WRITE,MAP_SHARED , fd , 0);
```

- (2) 使用特殊文件提供匿名内存映射：适用于具有亲缘关系的进程之间；

调用代码如下：

```
ptr=mmap(NULL, len , PROT_READ|PROT_WRITE, MAP_SHARED|MAP_ANONYMOUS , -1 , 0);
```

由于父子进程特殊的亲缘关系，在父进程中先调用mmap()，然后调用fork()。在调用fork()之后，子进程继承父进程匿名映射后的地址空间，也继承mmap()返回的地址

不必指定具体的文件，只要设置相应的标志即可

```
int munmap( void * addr, size_t len )
```

该调用在进程地址空间中解除一个映射关系，addr是调用mmap()时返回的地址，len是映射区的大小。当映射关系解除后，对原来映射地址的访问将导致段错误发生。

```
int msync ( void * addr , size_t len, int flags)
```

一般说来，进程在映射空间的对共享内容的改变并不直接写回到磁盘文件中，往往在调用munmap（）后才执行该操作。可以通过调用msync()实现磁盘上文件内容与共享内存区的内容一致。

例5-11mmap.c

设计一个程序，要求复制进程，父子进程通过匿名映射实现共享内存。

```
typedef struct
{
    char name[4];
    int age;
}people;
main(int argc, char** argv)
```

```

{
    pid_t result;
    int i;
    people *p_map;
    char temp;
    p_map=
    (people*)mmap(NULL, sizeof(people)*10, PROT_READ|PROT_WRITE, MAP_SHARED|MAP_ANONYMOUS, -1, 0);
    result=fork();
    if(result<0)
    {
        perror("创建子进程失败");
        exit(0);    }
    else if (result==0)
    {
        sleep(2);
        for(i = 0;i<5;i++)
            printf("子进程读取：第 %d 个人的年龄是： %d\n", i+1, (*(p_map+i)).age);
        (*p_map).age = 110;
        munmap(p_map, sizeof(people)*10); /*解除内存映射关系*/
        exit(0);
    }
    else {
        temp = 'a';
        for(i = 0;i<5;i++)
        {
            temp += 1;
            memcpy((*(p_map+i)).name, &temp, 1);
            (*(p_map+i)).age=20+i;
        }
        sleep(5);
        printf("父进程读取：五个人的年龄和是： %d\n", (*p_map).age );
        printf("解除内存映射.....\n");
        munmap(p_map, sizeof(people)*10);
        printf("解除内存映射成功! \n");
    }
}

```

使用特殊文件提供匿名内存映射，适用于具有亲缘关系的进程之间。一般而言，子进程单独维护从父进程继承下来的一些变量。而mmap函数的返回地址，由父子进程共同维护。

System V IPC共享内存的实现

通过映射到tmpfs中的shm文件对象实现共享主存

每个共享主存区对应tmpfs中的一个文件（通过shmid_kernel关联）

创建过程

从主存申请共享主存管理结构，初始化相应shmid_kernel结构

在tmpfs中创建并打开一个同名文件

在主存中建立与该文件对应的dentry及inode结构

返回相应标识符

共享内存（2）System V shared memory

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sm.h>

int shmget(key_t key, int size, int flag);
void *shmat(int shmid, void *addr, int flag);
int shmdt(void *addr);
int shmctl(int shmid, int cmd, struct shmid_ds *buf);
```

mmap和System V

System V共享内存中的数据，从来不写入到实际磁盘文件中去；而通过mmap()映射普通文件实现的共享内存通信可以指定何时将数据写入磁盘文件中。

System V共享内存是随内核持续的，即使所有访问共享内存的进程都已经正常终止，共享内存区仍然存在（除非显式删除共享内存），在内核重新引导之前，对该共享内存区域的任何改写操作都将一直保留。

3.4 信号量

用来协调进程对共享资源的访问

相关函数semget, semop, semctl

所需头文件

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
```

相关函数——semget

创建一个新信号量或取得一个已有信号量，原型为：int semget(key_t key, int num_sems, int sem_flags);

参数key：整数值

参数num_sems：指定需要的信号量数目，几乎总是1。

参数sem_flags：一组标志，信号量不存在时创建一个新的信号量，指定IPC_CREAT做按位或操作。设置了IPC_CREAT标志后，即使 给出的键是一个已有信号量的键，也不会产生错误。

指定IPC_CREAT | IPC_EXCL，创建一个新的，唯一的信号量，如果信号量已存在，返回一个错误。

返回值：成功返回一个相应信号标识符（非零），失败返回-1。

相关函数——semop

操作一个或一组信号，原型为：

```
int semop(int sem_id, struct sembuf *sem_opa, size_t nsops);
```

semid：信号集的识别码，可通过semget获取。

sem_opa：指向存储信号操作结构的数组指针，信号操作结构的原型如下

```
struct sembuf{
    short sem_num;    //信号量集中的信号量编号0, 1.....
    short sem_op;     //信号量在一次操作中需要改变的数据，通常是两个数，一个是-1，一个是+1。
    short sem_flg;    //通常为SEM_UNDO,使操作系统跟踪信号并在进程没有释放该信号量而终止时，操作系统释放
```

信号量

```
};
```

nsops：信号量操作结构的数量，大于或等于1

相关函数——semctl

该函数用来直接控制信号量信息，它的原型为

```
int semctl(int sem_id, int sem_num, int command, ...);
```

第四个参数，它通常是一个union semun结构，定义如下：

```
union semun{
    int val;
    struct semid_ds *buf;
    unsigned short *array;
};
```

使用举例——例5-12sem1.c

```
int main(int argc, char *argv[])
{
    char message = 'X';
    int i = 0;
    if(argc > 1)
        message = argv[1][0];
    for(i = 0; i < 10; ++i)
    {
        printf("%c", message);
        fflush(stdout);
        sleep(rand() % 3);
        printf("%c", message);
        fflush(stdout);
        sleep(rand() % 2);
    }
    sleep(10);
    printf("\n%d - finished\n", getpid());
    exit(EXIT_SUCCESS);
}

//-----
XXXXXXXXXXXXXXXXXXXXX
2490159 - finished
```

使用举例——例5-13sem2.c

```
static int set_semvalue()
{
    union semun sem_union;
    sem_union.val = 1; //用于初始化信号量
    if(semctl(sem_id, 0, SETVAL, sem_union) == -1)
        return 0;
    return 1;
}

static void del_semvalue()
{
    union semun sem_union;
    if(semctl(sem_id, 0, IPC_RMID, sem_union) == -1)
        fprintf(stderr, "Failed to delete semaphore\n");
}

static int semaphore_p()
{
    //对信号量做减1操作，即等待P(sv)
    struct sembuf sem_b;
    sem_b.sem_num = 0;
    sem_b.sem_op = -1; //P()
    sem_b.sem_flg = SEM_UNDO;
    if(semop(sem_id, &sem_b, 1) == -1)
```

```

{
    fprintf(stderr, "semaphore_p failed\n");
    return 0;
}
return 1;
}
for(i = 0; i < 10; ++i)
{
    if(!semaphore_p()) //进入临界区 前执行P操作
        exit(EXIT_FAILURE);
    printf("%c", message);
    fflush(stdout);
    sleep(rand() % 3);
    printf("%c", message); //离开临界区前再一次向屏幕输出数据
    fflush(stdout);
    if(!semaphore_v())
        exit(EXIT_FAILURE); //离开临界区，休眠随机时间后继续循环
    sleep(rand() % 2);
}
sleep(10);
printf("\n%d - finished\n", getpid());
if(argc > 1) //如果程序是第一次被调用，则在退出前删除信号量
{
    sleep(3);
    del_semvalue();
}
exit(EXIT_SUCCESS);
}
static int semaphore_v()
{
    //这是一个释放操作，它使信号量变为可用，即发送信号V（sv）
    struct sembuf sem_b;
    sem_b.sem_num = 0;
    sem_b.sem_op = 1; //V()
    sem_b.sem_flg = SEM_UNDO;
    if(semop(sem_id, &sem_b, 1) == -1)
    {
        fprintf(stderr, "semaphore_v failed\n");
        return 0;
    }
    return 1;
}
union semun
{
    int val;
    struct semid_ds *buf;
    unsigned short *array;
};
static int sem_id = 0;
static int set_semvalue();
static void del_semvalue();
static int semaphore_p();
static int semaphore_v();
int main(int argc, char *argv[])
{
    char message = 'X';
    int i = 0;
    sem_id = semget((key_t)1234, 1, 0666 | IPC_CREAT); //创建信号量
    if(argc > 1)
    {
        if(!set_semvalue()) //程序第一次被调用，初始化信号量
        {

```

```

        fprintf(stderr, "Failed to initialize semaphore\n");
        exit(EXIT_FAILURE);
    }
    message = argv[1][0]; //设置要输出到屏幕中的信息，即其参数的第一个字符
    sleep(2);
}
}

```

掌握的知识点

IPC通信的处理方式
消息队列和共享内存的特点
信号量的应用

网络编程

Socket的基本概念

如何实现面向连接的TCP编程

如何实现非连接的UDP编程

复杂网络程序的实现

网络程序的应用

4.0 服务器和客户机的信息函数

1. 字节序列转换

每一台机器内部对变量的字节存储顺序不同，而网络传输的数据是一定要统一顺序的。所以对内部字节表示顺序与网络字节顺序（大端）不同的机器，一定要对数据进行转换

以0x12345678数据为例，两种模式在内存中的存储情况。

内存地址	小端模式存储内容	大端模式存储内容
0x0000	0x78	0x12
0x0001	0x56	0x34
0x0002	0x34	0x56
0x0003	0x12	0x78

头文件: include <arpa/inet.h>

unsigned short int htons (unsigned short int hostshort) : 主机字节顺序转换成网络字节顺序，对无符号短型进行操作 4bytes
 unsigned long int htonl (unsigned long int hostlong) : 主机字节顺序转换成网络字节顺序，对无符号长型进行操作 8bytes
 unsigned short int ntohs (unsigned short int netshort) : 网络字节顺序转换成主机字节顺序，对无符号短型进行操作 4bytes
 unsigned long int ntohl (unsigned long int netlong) : 网络字节顺序转换成主机字节顺序，对无符号长型进行操作 8bytes

2. 地址格式转换

```

头文件: #include <sys/types.h>
        #include <sys/socket.h>
        #include <arpa/inet.h>
int inet_pton(int family, const char *strptr, void *addrptr);
将点分文本的IP地址转换为二进制网络字节序的IP地址
返回: 1成功; -1出错
const char* inet_ntop(int family, const void *addrptr, char *strptr, size_t len);
转换网络二进制结构到ASCII类型的地址
返回: 成功, 指向结果的指针; 出错, NULL

```

4.1 Socket 基础

(1)socket是网络编程的一种接口, 它是一种特殊的I/O, 用socket函数建立一个Socket连接, 此函数返回一个整型的socket描述符, 随后进行数据传输。

(2)一个IP地址, 一个通讯端口, 就能确定一个通讯程序的位置。为此开发人员专门设计了一个套接结构, 就是把网络程序中所用到的网络地址和端口信息放在一个结构体中。

(3)UDP是一个无连接协议, TCP是个可靠的端对端协议。传输UDP数据包时, LINUX不知道也不关心它们是否已经安全到达目的地, 而传输TCP数据包时, 则应先建立连接以保证传输的数据被正确接收。

三种类型套接字

(1)流套接字(SOCK_STREAM)

可靠的、面向连接的通信。

使用TCP协议

(2)数据报套接字(SOCK_DGRAM)

无连接服务

使用UDP协议

(3)原始套接字(SOCK_RAW)

允许对底层协议如IP、ICMP直接访问

1、socket套接字的数据结构

两个重要的数据类型: sockaddr和sockaddr_in, 这两个结构类型都是用来保存socket信息的, 如IP地址、通信端口等。

sockaddr—虚拟定义的地址(取决于协议):

```

struct sockaddr
{  unsigned short sa_family;
    /*可以为AF_INET, 代表TCP/IP地址族*/
    char sa_data[14];
    /*14个字节, 包含IP地址和端口号*/
};

```

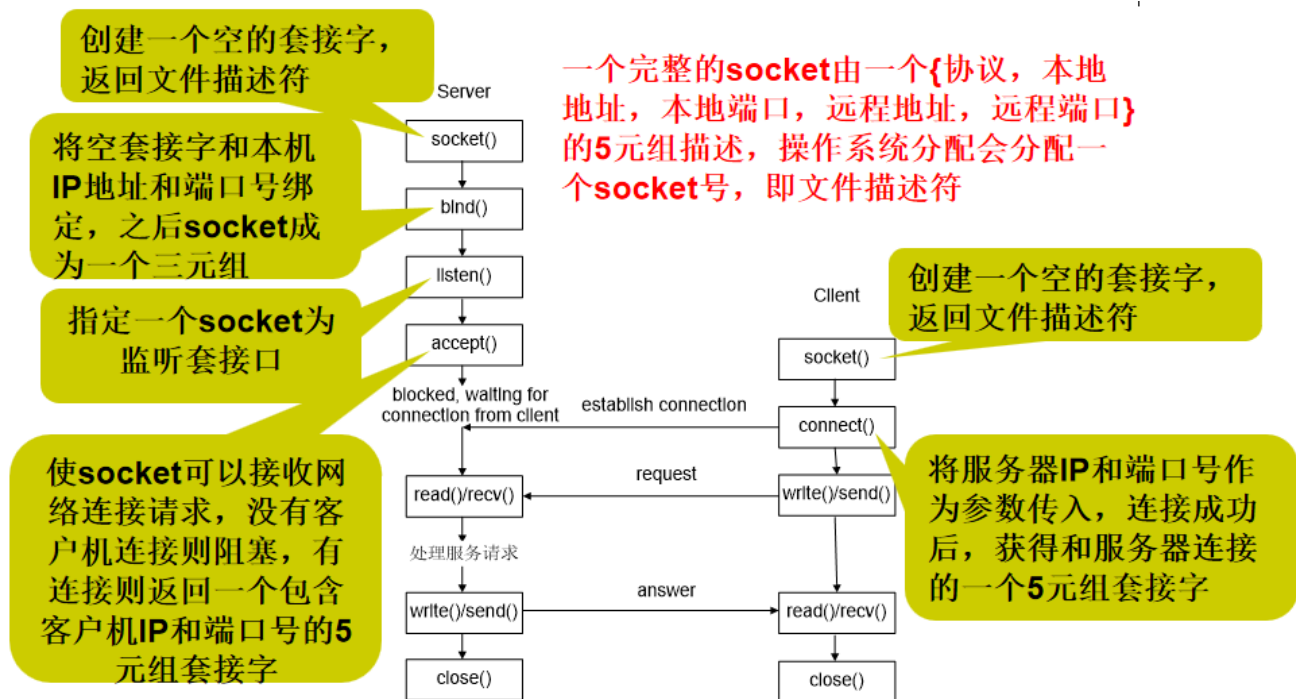
sockaddr_in (AF_INET中使用的地址定义):

```

struct sockaddr_in
{  short sin_family;          /*AF_INET (地址族)*/
    unsigned short sin_port;
    /*端口号(必须要采用网络数据格式)*/
    struct in_addr sin_addr;
    /*网络字节序的IP地址*, 只有一个成员s_addr*/
    unsigned char sin_zero[8];
    /*与SOCKADDR结构保持同样大小*/
};

```

2、基于连接的服务



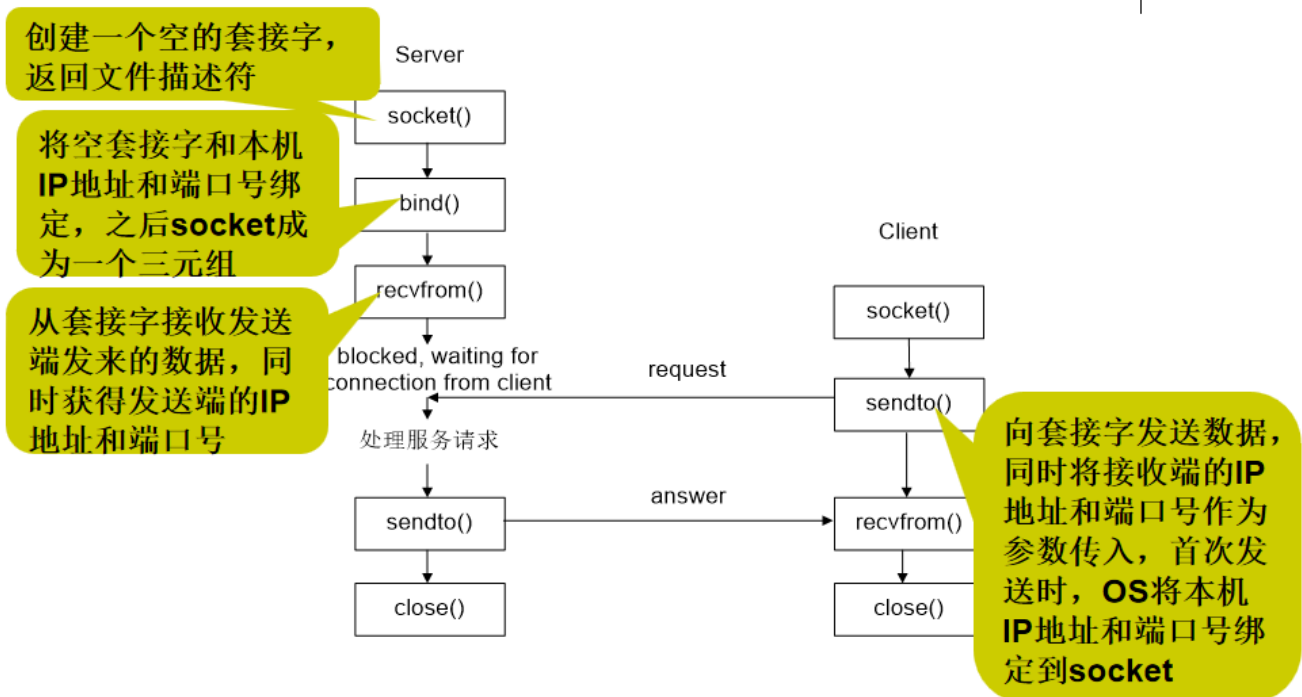
Server程序的作用

程序初始化
 持续监听一个固定的端口
 收到Client的连接后建立一个socket连接
 与Client进行通信和信息处理
 接收Client通过socket连接发送来的数据，进行相应处理并返回处理结果
 通过socket连接向Client发送信息
 通信结束后中断与Client的连接

Client程序的作用

程序初始化
 连接到某个Server上，建立socket连接
 与Server进行通信和信息处理
 接收Server通过socket连接发送来的数据，进行相应处理
 通过socket连接向Server发送请求信息
 通信结束后中断与Server的连接

3、无连接的服务



UDP编程的适用范围

部分满足以下几点要求时，应该用UDP

- 面向数据报
- 网络数据大多为短消息
- 拥有大量Client
- 对数据安全性无特殊要求
- 网络负担非常重，但对响应速度要求高

例子：ICQ、视频点播

具体编程时的区别

socket()的参数不同

- UDP Server不需要调用listen和accept
- UDP收发数据用sendto/recvfrom函数
- TCP: 地址信息在connect/accept时确定
- UDP: 在sendto/recvfrom函数中每次均需指定地址信息
- UDP: shutdown函数无效

4.2 TCP编程

基于TCP协议的编程，其最主要的特点是建立完连接后，才进行通信。

常用的基于TCP网络编程函数及功能

函数名	功能
bind	将 socket 与本机上的一个端口绑定，随后就可以在该端口监听服务请求
connect	面向连接的客户端程序使用 connect 函数来配置 socket 并与远端服务器建立一个 TCP 连接
listen	listen 函数使 socket 处于被动的监听模式，并为该 socket 建立一个输入数据队列，将到达的服务请求保存在此队列中，直到程序处理它们
accept	accept() 函数让服务器接收客户的连接请求
close	停止在该 socket 上的任何数据操作
send	数据发送函数
recv	数据接收函数

socket

```
socket: creates an endpoint for communication and returns a descriptor.
int socket(int domain, int type, int protocol);
(1)"domain" parameter
    Specifies a communication domain, that is, selects a protocol family, such as AF_INET,AF_INET6
...
(2)"type" parameter
    Specifies the communication semantics. Three main types are: SOCK_STREAM, SOCK_DGRAM, SOCK_RAW
(3)"protocol" parameter
    usually 0 (default).
```

bind

```
(1)bind: binds a name to a socket
int bind(int sockfd, struct sockaddr *my_addr, socklen_t addrlen);
(2)struct sockaddr {
    sa_family_t sa_family;
    char sa_data[14];
}
```

listen

```
listen: listen for connections on a socket
int listen(int s, int backlog);
参数说明:
s: socket()返回的套接口文件描述符。
backlog: 进入队列中允许的连接个数。进入的连接请求在使用系统调用accept()应答之前要在进入队列中等待。这个值是队列中最多可以拥有的请求的个数。大多数系统的缺省设置为20。
```

accept

```
accept()函数将响应连接请求，建立连接
int accept(int sockfd,struct sockaddr *addr,int *addrlen);
accept缺省是阻塞函数，阻塞直到有连接请求
sockfd: 被动(倾听)的socket描述符
如果成功，返回一个新的socket描述符(connected socket descriptor)来描述该连接。这个连接用来与特定的Client交换信息
addr将在函数调用后被填入连接对方的地址信息，如对方的IP、端口等。
```

connect

connect: initiate a connection on a `socket` (connect to a server).
`int connect(int sockfd, struct sockaddr *servaddr, int addrlen);`
主动的socket
`servaddr`是事先填写好的结构，Server的IP和端口都在该数据结构中指定。

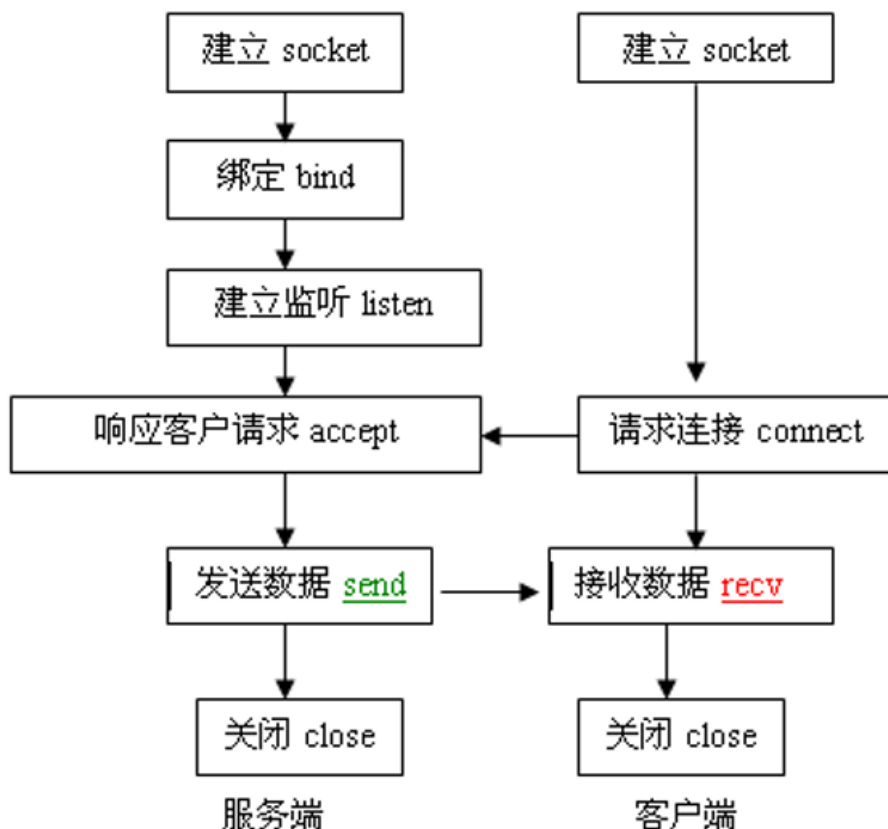
send/recv

send/recv: connection-oriented
`int send(int s, const void *msg, size_t len, int flag);`
s: 发送数据的套接口文件描述符。
msg: 发送的数据的指针
len: 数据的字节长度
flag: 标志设置为0。
`int recv(int s, void *buf, size_t len, int flag);`
s: 读取的套接口文件描述符。
buf: 保存读入信息的地址。
len: 缓冲区的最大长度。
flag: 设置为0。

close & shutdown

close
`int close(int sockfd);`
shutdown
`int shutdown(int sockfd, int how);`
how: SHUT_RD, SHUT_WR, SHUT_RDWR
shutdown直接对TCP连接进行操作，close只是对套接字描述符操作。

例5-14：服务器通过socket连接后，向客户端发送字符串“连接上了”。在服务器上显示客户端的IP地址或域



例5-14程序中的主要语句说明：

服务端

```
建立socket: socket(AF_INET, SOCK_STREAM, 0);
绑定bind: bind(sockfd, (struct sockaddr *)&my_addr, sizeof(struct sockaddr));
建立监听listen: listen(sockfd, BACKLOG);
响应客户请求: accept(sockfd, (struct sockaddr *)&remote_addr, &sin_size);
发送数据send: send(client_fd, "连接上了 \n", 26, 0);
关闭close: close(client_fd);
```

客户端:

```
建立socket: socket(AF_INET, SOCK_STREAM, 0);
请求连接connect: connect(sockfd, (struct sockaddr *)&serv_addr, sizeof(struct sockaddr));
接收数据recv: recv(sockfd, buf, MAXDATASIZE, 0);
关闭close: close(sockfd);
```

服务端源程序代码:

```
#include<stdio.h>
#include<stdlib.h>
#include<errno.h>
#include <string.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <sys/socket.h>
#include <sys/wait.h>
#include <arpa/inet.h>
#define SERVPORT 3333
#define BACKLOG 10
int main()
{
    int sockfd,client_fd;
    int sin_size;
    struct sockaddr_in my_addr;
    struct sockaddr_in remote_addr;
    sockfd = socket(AF_INET, SOCK_STREAM, 0) ;
    my_addr.sin_family=AF_INET;
    my_addr.sin_port=htons(SERVPORT);
    my_addr.sin_addr.s_addr = INADDR_ANY;
    bzero(&(my_addr.sin_zero),8);
    if (bind(sockfd, (struct sockaddr *)&my_addr, sizeof(my_addr)) == -1)
    {
        perror("bind 出错! ");
        exit(1);
    }
    if (listen(sockfd, BACKLOG) == -1)
    {
        perror("listen 出错! ");
        exit(1);
    }
    while(1)
    {
        sin_size = sizeof(struct sockaddr_in);
        if ((client_fd = accept(sockfd, (struct sockaddr *)&remote_addr, &sin_size)) == -1)
        {
            perror("accept error");
            continue;
        }
        printf("收到一个连接来自: %s\n", inet_ntoa(remote_addr.sin_addr));
        if (!fork())
        {
```

```

        if (send(client_fd, "连接上了 \n", 26, 0) == -1)
            error("send 出错! ");
        close(client_fd);
        exit(0);
    }
    close(client_fd);
}
}

```

客户端源程序代码：

```

#include<stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>
#include <netdb.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#define SERVPORT 3333
#define MAXDATASIZE 100
int main(int argc, char *argv[]){
    int sockfd, recvbytes;
    char buf[MAXDATASIZE];
    struct hostent *host;
    struct sockaddr_in serv_addr;
    if (argc < 2)
    {
        fprintf(stderr, "Please enter the server's hostname!\n");
        exit(1);
    }
    if((host=gethostbyname(argv[1]))==NULL)
    {
        perror("gethostbyname error! ");
        exit(1);
    }
    if ((sockfd = socket(AF_INET, SOCK_STREAM, 0)) == -1)
    {
        perror("socket create error! ");
        exit(1);
    }
    serv_addr.sin_family=AF_INET;
    serv_addr.sin_port=htons(SERVPORT);
    serv_addr.sin_addr = *((struct in_addr *)host->h_addr);
    bzero(&(serv_addr.sin_zero),8);
    if (connect(sockfd, (struct sockaddr *)&serv_addr, sizeof(struct sockaddr)) == -1)
    {
        perror("connect error! ");
        exit(1);
    }
    if ((recvbytes=recv(sockfd, buf, MAXDATASIZE, 0)) == -1)
    {
        perror("connect 出错! ");
        exit(1);
    }
    buf[recvbytes] = '\0';
    printf("收到: %s", buf);
    close(sockfd);
}

```

4.3 UDP编程

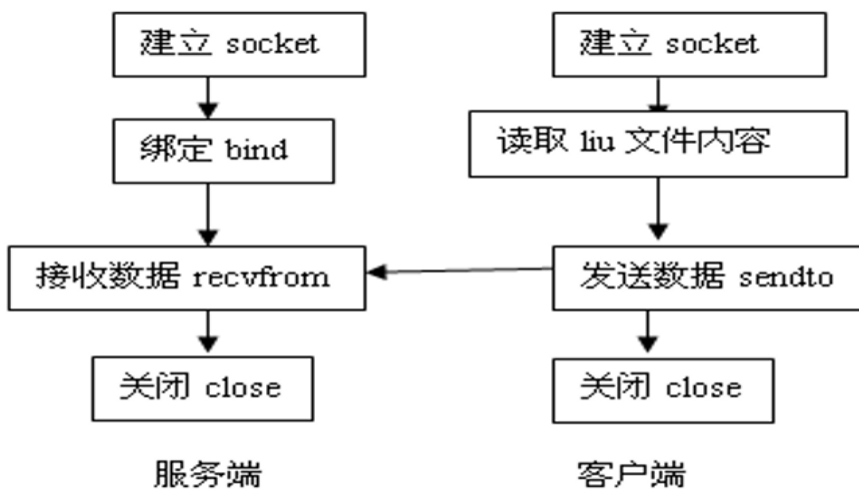
- 1)基于UDP协议的编程，其最主要的特点是不需要用函数bind把本地IP地址与端口号进行绑定，也能进行通信。
- 2)常用的基UDP网络编程函数及功能：

函数名	功能
bind	将 socket 与本机上的一个端口绑定，随后就可以在该端口监听服务请求
close	停止在该 socket 上的任何数据操作
sendto	数据发送函数
recvfrom	数据接收函数

sendto/recvfrom

```
int sendto(int s, const void *msg, size_t len, int flags, const struct sockaddr *to, socklen_t tolen);
int recvfrom(int s, void *buf, size_t len, int flags, struct sockaddr *from, socklen_t *fromlen);
```

例5-15：服务器端接受客户端发送的字符串。客户端将打开liu文件，读取文件中的3个字符串，传送给服务器端，当传送给服务端的字符串为”stop”时，终止数据传送并断开连接。



主要语句说明：

服务端：

- 1)建立socket: `socket(AF_INET,SOCK_DGRAM,0)`
- 2)绑定bind: `bind(sockfd,(struct sockaddr *)&adr_inet,sizeof(adr_inet));`
- 3)接收数据recvfrom: `recvfrom(sockfd,buf,sizeof(buf),0,(struct sockaddr *)&adr_clnt,&len);`
- 4)关闭close: `close(sockfd);`

客户端：

- 1)建立socket: `socket(AF_INET, SOCK_STREAM, 0);`
- 2)读取liu文件: `fopen("liu","r");`
- 3)发送数据sendto: `sendto(sockfd,buf,sizeof(buf),0,(struct sockaddr *)&adr_srvr,sizeof(adr_srvr));`
- 4)关闭close: `close(sockfd);`

服务端源程序代码：

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<sys/socket.h>
#include<netinet/in.h>
#include<arpa/inet.h>
#include<netdb.h>
#include<errno.h>
#include<sys/types.h>
int port=8888;
int main()
{
    int sockfd;
    int len;
    int z;
    struct sockaddr_in adr_inet;
    struct sockaddr_in adr_clnt; char buf[256];
    printf("等待客户端...\n");
    adr_inet.sin_family=AF_INET;
    adr_inet.sin_port=htons(port);
    adr_inet.sin_addr.s_addr =htonl(INADDR_ANY);
    bzero(&(adr_inet.sin_zero),8);
    len=sizeof(adr_clnt);
    sockfd=socket(AF_INET,SOCK_DGRAM,0);
    if(sockfd== -1)
    {
        perror("socket 出错");
        exit(1);
    }
    z=bind(sockfd,(struct sockaddr *)&adr_inet,sizeof(adr_inet));
    if(z== -1)
    {
        perror("bind 出错");
        exit(1);
    }
    while(1)
    {
        z=recvfrom(sockfd,buf,sizeof(buf),0,(struct sockaddr *)&adr_clnt,&len);
        if(z<0)
        {
            perror("recvfrom 出错");
            exit(1);
        }
        buf[z]=0;
        printf("接收:%s",buf);
        if(strncmp(buf,"stop",4)==0)
        {
            printf("结束...\n");
            break;
        }
    }
    close(sockfd);
    exit(0);
}
```

客户端源程序代码：

```
#include<stdio.h>
```

```

#include<stdlib.h>
#include<string.h>
#include<sys/socket.h>
#include<netinet/in.h>
#include<arpa/inet.h>
#include<netdb.h>
#include<errno.h>
#include<sys/types.h>
int port=8888;
int main()
{
    int sockfd;
    int i=0;
    int z;
    char buf[80],str1[80];
    struct sockaddr_in adr_srvr;
    FILE *fp;
    printf("打开文件.....\n");
    fp=fopen("liu","r");
    if(fp==NULL)
    {
        perror("打开文件失败");
        exit(1);
    }
    printf("连接服务端...\n");
    adr_srvr.sin_family=AF_INET;
    adr_srvr.sin_port=htons(port);
    adr_srvr.sin_addr.s_addr = htonl(INADDR_ANY);
    bzero(&(adr_srvr.sin_zero),8);
    sockfd=socket(AF_INET,SOCK_DGRAM,0);
    if(sockfd==-1)
    {
        perror("socket 出错");
        exit(1);
    }
    printf("发送文件 ....\n");
    for(i=0;i<3;i++)
    {
        fgets(str1,80,fp);
        printf("%d:%s",i,str1);
        sprintf(buf,"%d:%s",i,str1);
        z=sendto(sockfd,buf,sizeof(buf),0,(struct sockaddr *)&adr_srvr, sizeof(adr_srvr));
    }
    printf("发送.....\n");
    sprintf(buf,"stop\n");
    z=sendto(sockfd,buf,sizeof(buf),0,(struct sockaddr *)&adr_srvr,sizeof(adr_srvr));
    if(z<0)
    {
        perror("sendto 出错");
        exit(1);
    }
    fclose(fp);
    close(sockfd);
    exit(0);
}

```


4.4 网络高级编程

I/O Models

Block mode 阻塞模式

Non-block mode 非阻塞模式

阻塞方式

阻塞方式:在数据通信中,当服务器运行函数`accept()`时,假设没有客户机连接请求到来,那么服务器就会一直会停止在`accept()`语句上,等待客户机连接请求到来,出现这样的情况就称为阻塞。

非阻塞方式

阻塞与非阻塞方式的比较

`errno - EWOULDBLOCK`

非阻塞的实现

```
int flags;
```

```
if ( (flags=fcntl(sock_fd, F_GETFL, 0)) < 0)
```

```
    err_sys();
```

```
flags |= O_NONBLOCK;
```

```
if ( fcntl(sock_fd, F_SETFL, flags) < 0)
```

```
    err_sys();
```

掌握的知识点

TCP客户端和服务端的编程模型

UDP客户端和服务端的编程模型

服务器的多进程并发处理方式